DSA Pattern Recognition — Short Handbook

LAST-WEEK REVISION EDITION · 20 PATTERNS + 7 ALGORITHMS · JAVA

The condensed version of the full handbook. Every chapter has exactly three parts: **Concepts** (the invariant that makes the pattern work) → **Template** (reusable Java) → **Problems** (task, key insight, complete Java solution, complexity). All Java is identical to the full edition, where every block was compiled with JDK 17 and tested.

How to use it in the final week

1. Read the **key insight** and **recognition rule** at the top of the chapter.
2. Read Concepts, then write the Template from memory before looking.
3. For each problem: read the title, say the pattern + invariant + complexity aloud, then compare with the key insight. Re-code only those you got wrong.

Day	Chapters
1	P1–P4 · Two Pointers, Sliding Window, Fast/Slow, Prefix Sum
2	P5–P8 · Binary Search ×2, Intervals, Backtracking
3	P9–P11 · DFS, BFS, Topological Sort
4	P12–P14 · DP 1D, DP 2D, Greedy
5	P15–P17 · Heap, Bits, Union-Find
6	P18–P20 · Dijkstra, Trie, Segment/Fenwick
7	A1–A7 + redo every problem you missed

Before coding any problem

1. Input shape? (array, string, intervals, tree, graph, grid, operations)
2. Asked for? (exists, count, min/max, order, connectivity, shortest path)
3. Constraint budget: $n \le 20 \rightarrow 2^n$; $\le 5 \cdot 10^3 \rightarrow n^2$; $\le 10^5$–$10^6 \rightarrow n \log n$ / n; values $\le 10^9 \rightarrow \log V$.
4. Ordering, monotonicity, invariant or overlapping subproblems to exploit?
5. State the invariant in one sentence. Name the plausible wrong pattern and why it fails.
6. Edge cases: empty, single, duplicates, negatives, overflow, disconnected, cycles.

Signal → first hypothesis

Sorted pair / triplet sum	P1 Two Pointers
Contiguous range, monotone validity	P2 Sliding Window
Cycle / middle of a linked sequence	P3 Fast & Slow
Sum = k, divisible by k, negatives	P4 Prefix Sum + map
Sorted array boundary	P5 Binary Search
Minimise the max / smallest feasible	P6 BS on Answer
Overlaps, rooms, free time	P7 Merge Intervals
Enumerate all configurations	P8 Backtracking
Tree aggregates, components	P9 DFS
Fewest steps, unit weights	P10 BFS
Dependency order, cycle = impossible	P11 Topo Sort
Count ways / optimise, overlapping subproblems	P12/P13 DP
Max compatible intervals, provable local choice	P14 Greedy
Top-K, running median, repeated extremes	P15 Heap
Appears k times except one; XOR / masks	P16 Bits
Incremental connectivity, merging groups	P17 Union-Find
Weighted shortest path, weights ≥ 0	P18 Dijkstra
Prefix lookups over many words	P19 Trie
Range queries + updates	P20 Segment / Fenwick
Next greater / smaller element	A1 Monotonic Stack
Connect everything at min cost	A4 MST
Max-sum subarray	A5 Kadane
Pattern in text, borders	A6 KMP
Negative weights, ≤ K edges	A7 Bellman–Ford

Hypotheses, not rules: confirm the structural property before committing.

PATTERN 01

Two Pointers: Opposite Direction / Left–Right

> **Key insight.** Two pointers is correct only when each move discards a provably useless set of candidates. If you cannot say "every pair containing the pointer I just moved is impossible or no better than one already seen", you do not have a valid two-pointer solution.

> **Recognition rule.** Pair over an ordered axis + a score that is monotone in each end → shrink from the side that can no longer help.

Concepts

The elimination invariant. For a sorted array `a` and target `t`, maintain:

```
Invariant: if a valid pair (i, j) with i < j exists, then l ≤ i and j
≤ r.

If a[l] + a[r] < t:  for every j ≤ r,  a[l] + a[j] ≤ a[l] + a[r] < t
⇒  no pair uses l  ⇒  l++
If a[l] + a[r] > t:  for every i ≥ l,  a[i] + a[r] ≥ a[l] + a[r] > t
⇒  no pair uses r  ⇒  r--
```

Each step shrinks the search rectangle by one row or column, so the scan ends after at most n − 1 moves.

The dominance invariant (optimisation version). In Container With Most Water, area(l, r) = min(h[l], h[r]) · (r − l). Suppose h[l] ≤ h[r]. Then for every r' < r, the width shrinks and min(h[l], h[r']) ≤ h[l], so area(l, r') ≤ h[l] · (r' − l) < area(l, r). Pointer `l` has achieved its best pair, so discard it. The argument does not show the move is safe for the taller side, which is why you must move the shorter one.

Bounded-side invariant (Trapping Rain Water). Water above index i is `min(maxLeft(i), maxRight(i)) − h[i]`. Track `leftMax` over a[0..l] and `rightMax` over a[r..n−1]. If `leftMax < rightMax`, then for index l the true right maximum is at least `rightMax > leftMax`, so its water level is exactly `leftMax`, even though the rest of the right side has not been seen.

When sorting changes the problem. Sorting is safe when the answer depends only on the multiset of values (3Sum returns values). It is unsafe when it depends on positions: Container With Most Water uses indices as widths, so it must not be sorted, and its two-pointer validity comes from dominance, not order. Two Sum I returns indices, so sort index–value pairs or use a hash map.

Template

Template A — pair search with elimination. Reusable: the loop and the move rules. Problem-specific: `score` and what "found" means.

JAVA TEMPLATE TwoPointerPairSearch.java

```java
import java.util.*;

/** Opposite-direction pair search on a sorted array. Interval: [l,
  r] inclusive. */
class TwoPointerPairSearch {
    /** Indices {i, j}, i < j, with a[i] + a[j] == target, or null.
      a must be sorted. */
    static int[] findPair(int[] a, int target) {
        int l = 0, r = a.length - 1;
        // pairs need two distinct indices
        while (l < r) {
            // long: no overflow near Integer.MAX_VALUE
            long sum = (long) a[l] + a[r];
            if (sum == target) return new int[]{l, r};
            // pairs (l, j<=r) too small: drop row
            if (sum < target) l++;
            // pairs (i>=l, r) too large: drop col
            else r--;
        }
        return null;
    }

    /** Counts pairs i < j with a[i] + a[j] < target (a sorted), a
      block at a time. */
    static long countPairsBelow(int[] a, long target) {
```

```
        long count = 0;
        int l = 0, r = a.length - 1;
        while (l < r) {
            // l+1..r all ok
            if ((long) a[l] + a[r] < target) { count += r - l; l++;
              }
            else r--;
        }
        return count;
    }
}
```

Template B — fix one, two-pointer the rest (k-sum with deduplication).

JAVA TEMPLATE KSumTemplate.java

```
import java.util.*;

/** 3Sum-style skeleton: sort, fix a[i], two-pointer the suffix,
  skip duplicate values. */
class KSumTemplate {
    static List<List<Integer>> threeSum(int[] a, int target) {
        Arrays.sort(a);
        List<List<Integer>> res = new ArrayList<>();
        int n = a.length;
        for (int i = 0; i < n - 2; i++) {
            // canonical first value
            if (i > 0 && a[i] == a[i - 1]) continue;
            int l = i + 1, r = n - 1;
            while (l < r) {
                long s = (long) a[i] + a[l] + a[r];
                if (s == target) {
                    res.add(Arrays.asList(a[i], a[l], a[r]));
                    // skip AFTER recording a hit
                    while (l < r && a[l] == a[l + 1]) l++;
                    while (l < r && a[r] == a[r - 1]) r--;
                    l++;
                    r--;
                } else if (s < target) {
                    l++;
                } else {
                    r--;
                }
            }
        }
        return res;
    }
}
```

Problems

LC 167 Two Sum II - Input Array Is Sorted — MEDIUM

Task. Given a 1-indexed array sorted in non-decreasing order, return the 1-based indices of the unique pair summing to `target`, using O(1) extra space.

KEY INSIGHT

The sum is monotone in both pointers, so a mismatch identifies exactly one pointer that cannot belong to the answer.

JAVA SOLUTION LeetCode 167 · Solution.java

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int l = 0, r = numbers.length - 1;
        while (l < r) {
            // |values| <= 1000, so int is safe here
            int sum = numbers[l] + numbers[r];
            // 1-indexed output
            if (sum == target) return new int[]{l + 1, r + 1};
            if (sum < target) l++;
            else r--;
        }
        // unreachable: exactly one solution is guaranteed
        return new int[]{-1, -1};
    }
}
```

Time O(n) **Space** O(1)

LC 15 3Sum — MEDIUM

Task. Return all unique triplets of values (distinct indices) that sum to 0.

KEY INSIGHT

Fix the smallest element; the remaining pair is a sorted Two Sum on the suffix. Uniqueness comes from canonical choices on sorted data, not from a set of lists.

JAVA SOLUTION LeetCode 15 · Solution.java

```
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        int n = nums.length;
        for (int i = 0; i < n - 2; i++) {
            // smallest value > 0: done
            if (nums[i] > 0) break;
            // first value already handled
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int l = i + 1, r = n - 1;
            while (l < r) {
                // |v| <= 1e5: fits in int
                int s = nums[i] + nums[l] + nums[r];
                if (s == 0) {
                    res.add(Arrays.asList(nums[i], nums[l],
                      nums[r]));
                    while (l < r && nums[l] == nums[l + 1]) l++;
                    while (l < r && nums[r] == nums[r - 1]) r--;
                    l++;
                    r--;
                } else if (s < 0) {
                    l++;
                } else {
                    r--;
                }
            }
        }
        return res;
    }
}
```

Time O(n^2) **Space** O(log n) sort stack (output excluded)

LC 11 Container With Most Water — MEDIUM

Task. Choose two lines `i < j` maximising `min(h[i], h[j]) · (j − i)`.

KEY INSIGHT

The shorter line has already met its best partner (the farthest one). Every narrower container using it is no larger, so it can be discarded.

JAVA SOLUTION LeetCode 11 · Solution.java

```
class Solution {
    public int maxArea(int[] height) {
        int l = 0, r = height.length - 1, best = 0;
        while (l < r) {
            int h = Math.min(height[l], height[r]);
            // <= 1e4 * 1e5 = 1e9 < 2^31 - 1
            best = Math.max(best, h * (r - l));
            // l is the limiting side
            if (height[l] <= height[r]) l++;
            else r--;
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

LC 42 Trapping Rain Water — HARD

Task. Given bar heights, compute the total water trapped after rain.

KEY INSIGHT

`water[i] = min(maxLeft[i], maxRight[i]) − h[i]`. You do not need both maxima exactly, only to know which one is smaller. If `leftMax < rightMax`, the left index's level is settled at `leftMax`, because its true right maximum is at least `rightMax`.

JAVA SOLUTION LeetCode 42 · Solution.java

```
class Solution {
    public int trap(int[] height) {
        int l = 0, r = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (l < r) {
            // Invariant: leftMax = max(height[0..l-1]), rightMax =
              max(height[r+1..]).
            if (height[l] < height[r]) {
                // Inductively leftMax <= max(height[r..]): l's
                  water level is leftMax.
                leftMax = Math.max(leftMax, height[l]);
                water += leftMax - height[l];
                l++;
            } else {
                rightMax = Math.max(rightMax, height[r]);
                water += rightMax - height[r];
                r--;
            }
        }
        return water;
    }
}
```

Time O(n) **Space** O(1)

PATTERN 02

Sliding Window: Fixed and Variable Size

Key insight. A variable window is correct only when validity is *monotone under containment*: every sub-window of a valid window is valid (for "longest" problems), or every super-window of a valid window is valid (for "shortest" problems). Without that property, a greedy shrink can throw away the answer.

Concepts

Fixed window. The state equals the aggregate of exactly the last k elements:

```
S(r) = S(r−1) + a[r] − a[r−k]          for r ≥ k; window = [r−k+1, r]
```

The aggregate must be *invertible* (sum, count, xor, frequency map) to remove `a[r−k]` in O(1). Max and min are not invertible, which is why they need a deque.

Variable window, "longest valid". Invariant after processing r: `[l, r]` is the longest valid window ending at r. When adding `a[r]` breaks validity, shrink until valid. Correctness: if `[l, r]` is invalid then every `[l', r']` with `l' ≤ l, r' ≥ r` is invalid (it contains an invalid window), so no future answer starts at or before the discarded `l`.

Variable window, "shortest valid". Invariant: after shrinking, `[l−1, r]` was the last valid window. Expand r until valid, then shrink while still valid, recording the answer at each valid state. Correctness: once `[l, r]` is valid, any window `[l, r']` with `r' > r` is longer, so `l` never needs to pair with a larger r.

```
Window length          = r − l + 1      (inclusive [l, r])
Longest with ≤ k bad    : grow r; while bad > k: drop a[l++]; ans = max(ans, r − l + 1)
Shortest with condition : grow r; while valid:  ans = min(ans, r − l + 1); drop a[l++]
Exactly k               = atMost(k) − atMost(k − 1)    (counting problems)
```

Window state design. Keep a frequency array (`int[26]`, `int[128]`) when the alphabet is small, and a `HashMap` otherwise. Pair it with a single scalar that summarises validity: `distinct`, `bad`, or `missing`. Checking the scalar is O(1); re-scanning the frequency table per step is O(σ) and turns O(n) into O(nσ).

Template

Template A — fixed size k. Reusable: the add/remove rhythm. Problem-specific: what the state is and what is recorded.

JAVA TEMPLATE FixedWindow.java

```java
/** Fixed-size window over a[0..n-1]; window = [r-k+1, r]. Requires
  1 <= k <= n. */
class FixedWindow {
    static long maxWindowSum(int[] a, int k) {
        long sum = 0;
        // first full window [0, k-1]
        for (int i = 0; i < k; i++) sum += a[i];
        long best = sum;
        for (int r = k; r < a.length; r++) {
            // enter a[r], leave a[r-k]
            sum += a[r] - a[r - k];
            best = Math.max(best, sum);
        }
        return best;
    }
}
```

Template B — variable size, longest valid window (shrink while invalid).

JAVA TEMPLATE LongestValidWindow.java

```java
import java.util.*;

/** Longest subarray with at most k distinct values; validity is
  shrink-monotone. */
class LongestValidWindow {
    static int longestAtMostKDistinct(int[] a, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        int l = 0, best = 0;
        for (int r = 0; r < a.length; r++) {
            // expand
            freq.merge(a[r], 1, Integer::sum);
            // invalid: shrink from the left
            while (freq.size() > k) {
                if (freq.merge(a[l], -1, Integer::sum) == 0)
                  freq.remove(a[l]);
                l++;
            }
            // [l, r] valid and maximal
            best = Math.max(best, r - l + 1);
        }
        return best;
    }

    /** Exactly-k counting via the at-most difference. */
    static long countExactlyKDistinct(int[] a, int k) {
        return countAtMost(a, k) - countAtMost(a, k - 1);
    }

    private static long countAtMost(int[] a, int k) {
        if (k < 0) return 0;
        Map<Integer, Integer> freq = new HashMap<>();
        long count = 0;
        int l = 0;
        for (int r = 0; r < a.length; r++) {
            freq.merge(a[r], 1, Integer::sum);
            while (freq.size() > k) {
                if (freq.merge(a[l], -1, Integer::sum) == 0)
                  freq.remove(a[l]);
                l++;
            }
            // every window [l'..r], l <= l' <= r, is valid
            count += r - l + 1;
        }
        return count;
    }
}
```

Recognition rule. Contiguous range + a constraint that only gets worse as the window grows (or only better) → expand right, shrink left, never move back.

Assumption. Shrink-while-valid is correct only because all values are non-negative, so shrinking never makes an invalid window valid again. With negative values, use prefix sums with a monotonic deque instead.

Problems

LC 643 Maximum Average Subarray I — EASY

Task. Return the maximum average of any contiguous subarray of length exactly k.

KEY INSIGHT

Because k is fixed, maximising the average is the same as maximising the sum. Divide once at the end.

JAVA SOLUTION LeetCode 643 · Solution.java

```java
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        // |sum| <= 1e5 * 1e4: int also fits
        long sum = 0;
        for (int i = 0; i < k; i++) sum += nums[i];
        long best = sum;
        for (int r = k; r < nums.length; r++) {
            sum += nums[r] - nums[r - k];
            best = Math.max(best, sum);
        }
        // divide once, avoid rounding drift
        return (double) best / k;
    }
}
```

Time O(n) **Space** O(1)

LC 3 Longest Substring Without Repeating Characters — MEDIUM

Task. Return the length of the longest substring with all distinct characters. Characters are English letters, digits, symbols and spaces (ASCII).

KEY INSIGHT

"No repeats" is monotone under shrinking: a sub-window of a repeat-free window is repeat-free. When `s[r]` repeats, the window must start just after that character's previous occurrence, so `l` can jump there directly.

JAVA SOLUTION LeetCode 3 · Solution.java

```java
import java.util.*;

class Solution {
    public int lengthOfLongestSubstring(String s) {
        // ASCII alphabet per constraints
        int[] last = new int[128];
        Arrays.fill(last, -1);
        int l = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            // max: last[c] may be left of window
            l = Math.max(l, last[c] + 1);
            last[c] = r;
```

```
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

Time O(n) **Space** O(σ), σ = 128

LC 1493 Longest Subarray of 1's After Deleting One Element

MEDIUM

Task. In a binary array you *must* delete exactly one element. Return the length of the longest non-empty run of 1s that can result (0 if none).

KEY INSIGHT

Reframe as the longest window containing at most one zero. Deleting one element from that window (the zero, or any element if there is none) leaves all 1s, so the answer is the window length − 1.

JAVA SOLUTION LeetCode 1493 · Solution.java

```
class Solution {
    public int longestSubarray(int[] nums) {
        int l = 0, zeros = 0, best = 0;
        for (int r = 0; r < nums.length; r++) {
            if (nums[r] == 0) zeros++;
            // window may hold at most one zero
            while (zeros > 1) {
                if (nums[l] == 0) zeros--;
                l++;
            }
            // window length - 1 deleted element
            best = Math.max(best, r - l);
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

LC 76 Minimum Window Substring

HARD

Task. Return the shortest substring of `s` that contains every character of `t` with multiplicity, or `""`. Both strings use upper- and lowercase English letters.

KEY INSIGHT

"Covers t" is monotone under extension: a super-window of a covering window still covers. That makes it a shortest-valid window. A single `missing` counter (characters of t not yet covered) makes the validity check O(1).

JAVA SOLUTION LeetCode 76 · Solution.java

```
class Solution {
    public String minWindow(String s, String t) {
        if (t.length() > s.length()) return "";
        // need[c] > 0: still required copies
        int[] need = new int[128];
        for (int i = 0; i < t.length(); i++) need[t.charAt(i)]++;
        // required characters not yet covered
        int missing = t.length();
        int l = 0, bestL = 0, bestLen = Integer.MAX_VALUE;
        for (int r = 0; r < s.length(); r++) {
            // this copy was actually needed
            if (need[s.charAt(r)]-- > 0) missing--;
            // [l, r] covers t: shrink while valid
            while (missing == 0) {
                if (r - l + 1 < bestLen) {
                    bestLen = r - l + 1;
                    bestL = l;
                }
                // removed a required copy
                if (++need[s.charAt(l)] > 0) missing++;
                l++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" :
          s.substring(bestL, bestL + bestLen);
    }
}
```

Time O(|s| + |t|) **Space** O(σ), σ = 128

PATTERN 03

Fast and Slow Pointers: Cycle Detection / Linked Lists

Key insight. Fast/slow works on any structure where each state has *exactly one* next state (a functional graph). Its paths look like the Greek letter ρ: a tail leading into one cycle. It does not apply to general graphs, where a node can have several neighbours.

Recognition rule. One-successor sequence + a question about looping or relative position + O(1) memory → tortoise and hare.

Concepts

Convergence. Once both pointers are in the cycle (length L), let the gap be d = (fast − slow) mod L. Each iteration fast moves 2 and slow moves 1, so the gap changes by exactly one each step. It therefore passes through 0 within L steps and the two meet, with no risk of fast "jumping over" slow. Detection costs O(μ + L), where μ is the tail length.

Entry point derivation. Let μ = distance from head to entry, and suppose they meet k steps into the cycle.

```
slow travelled  s = μ + k  (+ whole loops, absorbed below)
fast travelled 2s = μ + k + m·L            for some integer m ≥ 1
⇒  s = m·L     ⇒   μ + k = m·L     ⇒   μ = m·L − k = (m − 1)·L + (L − k)
```

From the meeting point, walking μ steps lands on `k + μ ≡ 0 (mod L)`, which is the entry. So reset one pointer to the head and move both one step at a time; they meet exactly at the entry after μ steps.

Midpoint conventions. With `slow = fast = head` and the loop guard `fast != null && fast.next != null`, slow ends at index ⌊n/2⌋ (0-based): the *second* middle for even n. With the guard `fast.next != null && fast.next.next != null`, slow ends at ⌊(n−1)/2⌋: the *first* middle, which is what you need to split a list before reversing its second half.

Template

Template A — middle node (both conventions).

JAVA TEMPLATE ListMiddle.java (uses LeetCode ListNode)

```
/** Middle of a singly linked list in one pass. ListNode is the
  LeetCode definition. */
class ListMiddle {
    /** Second middle for even length: index n/2. */
    static ListNode secondMiddle(ListNode head) {
        ListNode slow = head, fast = head;
        // fast moves 2, slow moves 1
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    /** First middle for even length: index (n-1)/2. Use it to split
      before reversing. */
    static ListNode firstMiddle(ListNode head) {
        if (head == null) return null;
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

Template B — Floyd on any functional sequence. The same two phases work for list nodes, array indices (`i -> a[i]`) or numbers (`x -> f(x)`). Only `next` changes.

JAVA TEMPLATE FloydCycle.java

```
import java.util.function.IntUnaryOperator;

/** Floyd's cycle detection on x0, f(x0), f(f(x0)), ... over a
  finite state space. */
class FloydCycle {
    /** Returns the first state on the cycle (the entry); requires a
      finite state space. */
    static int cycleEntry(int x0, IntUnaryOperator f) {
        int slow = f.applyAsInt(x0), fast =
          f.applyAsInt(f.applyAsInt(x0));
        // phase 1: meet inside the cycle
        while (slow != fast) {
            slow = f.applyAsInt(slow);
            fast = f.applyAsInt(f.applyAsInt(fast));
        }
        // phase 2: mu steps from both ends
        slow = x0;
        while (slow != fast) {
            slow = f.applyAsInt(slow);
            fast = f.applyAsInt(fast);
        }
        return slow;
    }
```

```
    /** Cycle length: walk once around from any state known to be on
      the cycle. */
    static int cycleLength(int onCycle, IntUnaryOperator f) {
        int len = 1;
        for (int x = f.applyAsInt(onCycle); x != onCycle; x =
          f.applyAsInt(x)) len++;
        return len;
    }
}
```

Problems

LC 876 Middle of the Linked List — EASY

Task. Return the middle node of a non-empty list; for even length, the second of the two middles.

KEY INSIGHT

When fast has taken 2t steps and runs out, slow has taken t steps, so slow is at index ⌊n/2⌋.

JAVA SOLUTION LeetCode 876 · Solution.java

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // index n/2: second middle if n even
        return slow;
    }
}
```

Time O(n) **Space** O(1)

LC 142 Linked List Cycle II — MEDIUM

Task. Return the node where the cycle begins, or `null` if the list is acyclic. Do not modify the list. The follow-up asks for O(1) memory.

KEY INSIGHT

From the derivation μ = (m − 1)L + (L − k), a pointer restarted at the head and a pointer left at the meeting point reach the entry at the same time.

JAVA SOLUTION LeetCode 142 · Solution.java

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            // inside the cycle, k steps past entry
            if (slow == fast) {
                ListNode p = head;
                // both are mu steps from the entry
                while (p != slow) {
                    p = p.next;
                    slow = slow.next;
                }
                return p;
            }
        }
        // fast fell off: no cycle
        return null;
    }
}
```

Time O(n) **Space** O(1)

LC 547 Number of Provinces — MEDIUM

Task. Given an n × n adjacency matrix `isConnected` (undirected, symmetric, diagonal = 1), return the number of connected components.

KEY INSIGHT

Each traversal started from an unvisited city marks exactly one whole component, so the number of traversal starts equals the number of provinces.

JAVA SOLUTION LeetCode 547 · Solution.java (iterative BFS)

```
import java.util.*;

class Solution {
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, provinces = 0;
        boolean[] visited = new boolean[n];
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int s = 0; s < n; s++) {
            if (visited[s]) continue;
            // s starts a new component
            provinces++;
            visited[s] = true;
            queue.add(s);
            while (!queue.isEmpty()) {
                int u = queue.poll();
                for (int v = 0; v < n; v++) {
                    if (isConnected[u][v] == 1 && !visited[v]) {
                        // mark on enqueue: no duplicates
                        visited[v] = true;
                        queue.add(v);
                    }
                }
            }
        }
        return provinces;
    }
}
```

Time O(n^2) (the matrix is read once per row) **Space** O(n) visited + queue

LC 202 Happy Number — EASY

Task. Repeatedly replace n by the sum of the squares of its digits. Return true if the process reaches 1.

KEY INSIGHT

n → f(n) is a functional sequence in a bounded state space (see the termination note in 3.3), so it is eventually periodic. 1 is a fixed point, so n is happy exactly when the cycle Floyd finds is {1}.

JAVA SOLUTION LeetCode 202 · Solution.java

```
class Solution {
    public boolean isHappy(int n) {
        int slow = next(n), fast = next(next(n));
        while (slow != fast) {
            slow = next(slow);
            fast = next(next(fast));
        }
        // the cycle reached is {1} iff happy
        return slow == 1;
    }

    private int next(int x) {
        int sum = 0;
        while (x > 0) {
            int d = x % 10;
            // <= 81 * 10 digits: no overflow
            sum += d * d;
            x /= 10;
        }
        return sum;
    }
}
```

Time O(log n) per step; O(log n) steps before reaching the bounded range, then O(1) (states < 1000)

Space O(1)

PATTERN 04

Prefix Sum / Cumulative Sum

Key insight. Prefix sums make no monotonicity assumption, which is exactly why they work with negative values where a sliding window fails. The price is O(n) memory for the map of earlier prefixes.

Recognition rule. Exact sum, balance or divisibility on a contiguous range, especially with negatives → prefix sums plus a hash map of earlier prefixes.

Concepts

Indexing convention. Use a length-(n+1) array with an empty prefix:

```
P[0] = 0,   P[i+1] = P[i] + a[i]          P[i] = sum of a[0..i−1]
sum(a[l..r]) = P[r+1] − P[l]              inclusive l..r, 0 ≤ l ≤ r < n
2D:  S[i+1][j+1] = S[i][j+1] + S[i+1][j] − S[i][j] + g[i][j]
     sum(r1..r2, c1..c2) = S[r2+1][c2+1] − S[r1][c2+1] − S[r2+1][c1]
+ S[r1][c1]
```

The extra slot `P[0] = 0` represents the empty prefix. It is what allows subarrays starting at index 0 to be counted, and it is the reason hash-map solutions seed the map with `{0: 1}` (counting) or `{0: −1}` (index-based).

Hash-map invariant. Process j = 0..n−1 with running prefix `p = P[j+1]`. Invariant: before handling j, the map contains exactly the prefixes P[0..j], which are the starts of subarrays ending at j. Therefore:

1. **Query first**: count += map[p − k] counts subarrays (i..j) with i ≤ j.
2. **Then insert** p, so it can serve as a start for *later* ends.

Inserting before querying makes `P[j+1]` available as its own start, which pairs it with itself and counts the empty subarray whenever k = 0. That is the classic wrong-order bug.

First vs last occurrence. For the longest subarray, store a prefix's *first* index and never overwrite it (use `putIfAbsent`). For the shortest, keep the *latest* index (overwrite). For counting, store frequencies.

Difference arrays (the inverse operation). To add v to every element of a[l..r] for many updates, then read the array once: `D[l] += v; D[r+1] -= v`, and the final array is the prefix sum of D. This gives O(1) per update and O(n) to materialise.

Template

Template A — static range sums (1D and 2D).

JAVA TEMPLATE PrefixSums.java

```java
/** Static prefix sums. Queries are inclusive [l, r]; arrays carry
  an empty prefix. */
class PrefixSums {
    // p[i] = a[0] + ... + a[i-1]
    private final long[] p;

    PrefixSums(int[] a) {
        p = new long[a.length + 1];
        for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    }

    // 0 <= l <= r < n
    long sum(int l, int r) { return p[r + 1] - p[l]; }

    /** 2D: s[i][j] = sum of g[0..i-1][0..j-1]. */
    static long[][] build2D(int[][] g) {
        int m = g.length, n = g[0].length;
        long[][] s = new long[m + 1][n + 1];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                s[i + 1][j + 1] = s[i][j + 1] + s[i + 1][j] - s[i]
                  [j] + g[i][j];
        return s;
    }

    // inclusive corners
    static long sum2D(long[][] s, int r1, int c1, int r2, int c2) {
        return s[r2 + 1][c2 + 1] - s[r1][c2 + 1] - s[r2 + 1][c1] +
          s[r1][c1];
    }
}
```

Template B — prefix + hash map (count, longest, modular). Reusable: the scan, the seeding, and query-before-insert. Problem-specific: the key transform and the stored value.

JAVA TEMPLATE PrefixHashMap.java

```java
import java.util.*;

class PrefixHashMap {
    /** Number of subarrays with sum exactly k. Works with negative
      values. */
    static long countSumK(int[] a, long k) {
        Map<Long, Integer> freq = new HashMap<>();
        // empty prefix P[0] = 0
        freq.put(0L, 1);
        long p = 0, count = 0;
        for (int x : a) {
            p += x;
            // query: starts among P[0..j]
            count += freq.getOrDefault(p - k, 0);
            // then insert P[j+1]
            freq.merge(p, 1, Integer::sum);
        }
        return count;
    }

    /** Length of the longest subarray with sum exactly k (0 if
      none). */
    static int longestSumK(int[] a, long k) {
        Map<Long, Integer> first = new HashMap<>();
        // P[0] sits "before" index 0
        first.put(0L, -1);
        long p = 0;
        int best = 0;
        for (int j = 0; j < a.length; j++) {
            p += a[j];
            Integer i = first.get(p - k);
            // subarray (i+1 .. j)
            if (i != null) best = Math.max(best, j - i);
            // keep the earliest index only
            first.putIfAbsent(p, j);
        }
        return best;
    }

    /** Number of subarrays whose sum is divisible by k (k >= 1),
      values of any sign. */
    static long countDivisible(int[] a, int k) {
        long[] bucket = new long[k];
        // empty prefix has remainder 0
        bucket[0] = 1;
        long rem = 0, count = 0;
        for (int x : a) {
            // normalise Java's signed %
            rem = ((rem + x) % k + k) % k;
            count += bucket[(int) rem];
            bucket[(int) rem]++;
        }
        return count;
    }
}
```

Problems

LC 724 Find Pivot Index — EASY

Task. Return the leftmost index i where the sum strictly left of i equals the sum strictly right of i, or −1. Values may be negative.

KEY INSIGHT

The right sum is `total - left - nums[i]`, so a single running prefix and the total are enough. No array is needed.

JAVA SOLUTION LeetCode 724 · Solution.java

```java
class Solution {
    public int pivotIndex(int[] nums) {
        // |total| <= 1e4 * 1000: int is safe
        int total = 0;
        for (int x : nums) total += x;
        // sum of nums[0..i-1]
        int left = 0;
        for (int i = 0; i < nums.length; i++) {
            // first hit is leftmost
            if (left == total - left - nums[i]) return i;
            left += nums[i];
        }
        return -1;
    }
}
```

Time O(n) **Space** O(1)

LC 560 Subarray Sum Equals K — MEDIUM

Task. Count the contiguous non-empty subarrays whose sum equals k. Values are in [−1000, 1000].

KEY INSIGHT

sum(i..j) = k ⇔ P[i] = P[j+1] − k. For each end j, the number of valid starts is the number of earlier prefixes equal to `P[j+1] - k`.

JAVA SOLUTION LeetCode 560 · Solution.java

```java
import java.util.*;

class Solution {
    public int subarraySum(int[] nums, int k) {
        // |prefix| <= 2e4 * 1000: int fits
        Map<Integer, Integer> freq = new HashMap<>();
        // empty prefix
        freq.put(0, 1);
        int p = 0, count = 0;
        for (int x : nums) {
            p += x;
            // query earlier prefixes first
            count += freq.getOrDefault(p - k, 0);
            // then make p available to later ends
            freq.merge(p, 1, Integer::sum);
        }
        return count;
    }
}
```

Time O(n) expected **Space** O(n)

LC 523 Continuous Subarray Sum — MEDIUM

Task. Return true if some subarray of length at least 2 has a sum that is a multiple of k. Here 0 ≤ nums[i] ≤ 10^9 and 1 ≤ k ≤ 2^{31} − 1. Zero counts as a multiple.

KEY INSIGHT

The sum of (i..j) is a multiple of k ⇔ P[i] ≡ P[j+1] (mod k). The length constraint j − i + 1 ≥ 2 means the two equal-remainder prefixes must be at least 2 positions apart, so keep the *earliest* index of each remainder.

JAVA SOLUTION LeetCode 523 · Solution.java

```java
import java.util.*;

class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        // remainder -> earliest prefix index
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);
```

```
        // long: rem + nums[j] can exceed 2^31-1
        long rem = 0;
        for (int j = 0; j < nums.length; j++) {
            // nums are non-negative: no sign fix
            rem = (rem + nums[j]) % k;
            Integer i = first.get(rem);
            if (i != null) {
                // subarray (i+1 .. j), length >= 2
                if (j - i >= 2) return true;
            } else {
                // never overwrite: earliest wins
                first.put(rem, j);
            }
        }
        return false;
    }
}
```

Time O(n) expected **Space** O(min(n, k))

LC 325 Maximum Size Subarray Sum Equals k — MEDIUM — Premium

Task. Return the maximum length of a subarray summing to k, or 0 if none. Values may be negative ($|nums[i]| \le 10^4$, $|k| \le 10^9$, $n \le 2 \cdot 10^5$).

KEY INSIGHT

For end j, the longest valid subarray starts right after the *earliest* i with P[i] = P[j+1] − k. Keeping only first occurrences makes each lookup return the best start.

JAVA SOLUTION LeetCode 325 · Solution.java

```
import java.util.*;

class Solution {
    public int maxSubArrayLen(int[] nums, int k) {
        // prefix value -> earliest index
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);
        // long keeps p - k exact
        long p = 0;
        int best = 0;
        for (int j = 0; j < nums.length; j++) {
            p += nums[j];
            Integer i = first.get(p - k);
            if (i != null) best = Math.max(best, j - i);
            first.putIfAbsent(p, j);
        }
        return best;
    }
}
```

Time O(n) expected **Space** O(n)

PATTERN 05

Binary Search: Index / Sorted Array

Key insight. Correctness comes from one invariant: "the answer, if it exists, lies within my current interval". Each update must discard only indices that provably are not the answer, and the interval must strictly shrink.

Recognition rule. An index space plus a predicate that splits it into "no" then "yes" (or tells you which side holds an answer) → halve it.

Concepts

Two interval conventions. Pick one per problem and never mix them:

Convention	Loop	Shrink rules	Exit state
Inclusive `[lo, hi]`	`while (lo <= hi)`	`lo = mid + 1` / `hi = mid - 1`	Empty interval: lo = hi + 1. Use for exact match with early return.
Half-open `[lo, hi)`	`while (lo < hi)`	`lo = mid + 1` (pred false) / `hi = mid` (pred true)	lo == hi == first true (or n if none). Use for boundaries.

In the half-open form, `hi = mid` keeps a possible answer, and `mid = lo + (hi - lo) / 2 < hi` guarantees progress. If you need `lo = mid` (searching the *last* true), round the midpoint up (`lo + (hi - lo + 1) / 2`) or the loop can stall at a two-element interval.

```
lowerBound(x) = first i with a[i] ≥ x       (n if none)
upperBound(x) = first i with a[i] > x       (n if none)
count(x)      = upperBound(x) - lowerBound(x)
last index of x = upperBound(x) - 1         (valid iff count > 0)
mid = lo + (hi - lo) / 2   or   (lo + hi) >>> 1    never (lo + hi) / 2 when lo + hi may overflow
```

Duplicates. Exact-match search returns *an* occurrence, not a specific one. First/last occurrence must be phrased as a boundary (≥ x, > x), and then duplicates need no special handling.

Partition search. For the median of A (size m ≤ n) and B (size n), choose i elements from A and j = ⌈(m+n)/2⌉ − i from B for the left half. The cut is valid iff `A[i-1] ≤ B[j]` and `B[j-1] ≤ A[i]`. If `A[i-1] > B[j]`, i is too large; otherwise it is too small. So "i is too large" is a monotone predicate over i ∈ [0, m], and the search costs O(log min(m, n)).

Template

Template A — exact match, inclusive interval.

JAVA TEMPLATE ExactSearch.java

```
/** Exact match on a sorted array. Interval [lo, hi] inclusive;
  returns -1 if absent. */
class ExactSearch {
    static int indexOf(int[] a, int x) {
        int lo = 0, hi = a.length - 1;
        // invariant: if x exists, it is in [lo, hi]
        while (lo <= hi) {
            // overflow-safe midpoint
            int mid = lo + (hi - lo) / 2;
            // some occurrence (not necessarily first)
            if (a[mid] == x) return mid;
            if (a[mid] < x) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
}
```

Template B — boundary search on a predicate, half-open interval. This one shape covers lower bound, upper bound, rotated minimum and "first bad version".

JAVA TEMPLATE BoundarySearch.java

```
import java.util.function.IntPredicate;

/** First index in [lo, hi) where pred is true; pred must be
  false...false true...true. */
class BoundarySearch {
    static int firstTrue(int lo, int hi, IntPredicate pred) {
        // answer in [lo, hi]; hi means "none"
        while (lo < hi) {
            // mid < hi, so hi = mid always shrinks
            int mid = lo + (hi - lo) / 2;
            // mid may be the answer: keep it
            if (pred.test(mid)) hi = mid;
            // mid is false: discard it and everything left
            else lo = mid + 1;
        }
        // == original hi if pred is never true
        return lo;
    }

    static int lowerBound(int[] a, int x) { return firstTrue(0,
      a.length, i -> a[i] >= x); }

    static int upperBound(int[] a, int x) { return firstTrue(0,
      a.length, i -> a[i] > x); }

    /** Last index in [lo, hi] with pred true; pred is true...true
      false...false. */
    static int lastTrue(int lo, int hi, IntPredicate pred) {
        int ans = lo - 1;                          // sentinel: none
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            // record and look right
            if (pred.test(mid)) { ans = mid; lo = mid + 1; }
            else hi = mid - 1;
        }
        return ans;
    }
}
```

Problems

LC 704 Binary Search — EASY

Task. Return the index of `target` in an ascending array of distinct integers, or −1, in O(log n).

KEY INSIGHT

With distinct values, exact match has at most one answer, so the inclusive form with an early return is the cleanest fit.

JAVA SOLUTION LeetCode 704 · Solution.java

```java
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
}
```

Time O(log n) **Space** O(1)

LC 34 Find First and Last Position of Element in Sorted Array

MEDIUM

Task. In a non-decreasing array, return `[first, last]` indices of `target`, or `[-1, -1]`, in O(log n).

KEY INSIGHT

first = lowerBound(target), the first index with a[i] ≥ target. last = upperBound(target) − 1, using the first index with a[i] > target. Duplicates then need no special handling.

JAVA SOLUTION LeetCode 34 · Solution.java

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        // first a[i] >= target
        int first = firstIndex(nums, target, false);
        if (first == nums.length || nums[first] != target) return
          new int[]{-1, -1};
        // (first a[i] > target) - 1
        int last = firstIndex(nums, target, true) - 1;
        return new int[]{first, last};
    }

    /** First index in [0, n) with a[i] > target (strict) or a[i] >=
      target; n if none. */
    private int firstIndex(int[] a, int target, boolean strict) {
        // half-open [lo, hi)
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            boolean pred = strict ? a[mid] > target : a[mid] >=
              target;
            if (pred) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
}
```

Time O(log n) **Space** O(1)

LC 162 Find Peak Element

MEDIUM

Task. Return the index of any element strictly greater than its neighbours, with `nums[-1] = nums[n] = −∞` and adjacent elements distinct, in O(log n).

KEY INSIGHT

The slope predicate `nums[m] > nums[m+1]` ("descending at m") tells you a peak exists in [lo, m]; otherwise one exists in [m+1, hi]. The array is unsorted, but the invariant "a peak lies in [lo, hi]" is preserved (proof in 5.3).

JAVA SOLUTION LeetCode 162 · Solution.java

```java
class Solution {
    public int findPeakElement(int[] nums) {
        // invariant: a peak lies in [lo, hi]
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            // m < hi, so m + 1 is in range
            int m = lo + (hi - lo) / 2;
            // descending: peak in [lo, m]
            if (nums[m] > nums[m + 1]) hi = m;
            // ascending: peak in [m + 1, hi]
            else lo = m + 1;
        }
        return lo;
    }
}
```

Time O(log n) **Space** O(1)

LC 4 Median of Two Sorted Arrays

HARD

Task. Return the median of the union of two sorted arrays of sizes m and n in O(log(m + n)).

KEY INSIGHT

The median is defined by a *partition*: a left half of size ⌈(m+n)/2⌉ with every left element ≤ every right element. Choosing i elements from the shorter array fixes `j = half − i` from the other, so you are binary searching a single integer i ∈ [0, m].

JAVA SOLUTION LeetCode 4 · Solution.java

```java
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return
          findMedianSortedArrays(nums2, nums1);
        // a is the shorter array
        int[] a = nums1, b = nums2;
        int m = a.length, n = b.length;
        // size of the left half
        int half = (m + n + 1) / 2;
        // i = number taken from a, in [0, m]
        int lo = 0, hi = m;
        while (lo <= hi) {
            int i = lo + (hi - lo) / 2;
            // 0 <= j <= n because m <= n
            int j = half - i;
            // Sentinels for empty sides; values are within +-1e6,
              so long bounds are safe.
            long aLeft = i == 0 ? Long.MIN_VALUE : a[i - 1];
            long aRight = i == m ? Long.MAX_VALUE : a[i];
            long bLeft = j == 0 ? Long.MIN_VALUE : b[j - 1];
            long bRight = j == n ? Long.MAX_VALUE : b[j];
            if (aLeft > bRight) {
                // took too many from a
                hi = i - 1;
            } else if (bLeft > aRight) {
                // took too few from a
                lo = i + 1;
            } else {
                long leftMax = Math.max(aLeft, bLeft);
                if ((m + n) % 2 == 1) return leftMax;
                long rightMin = Math.min(aRight, bRight);
                return (leftMax + rightMin) / 2.0;
            }
        }
        throw new IllegalArgumentException("inputs must be sorted");
    }
}
```

Time O(log min(m, n)) **Space** O(1)

PATTERN 06

Binary Search on Answer: Parametric Search

Key insight. You do not need a formula for the optimum. You need (1) a range guaranteed to contain it, (2) a predicate that is monotone over that range, and (3) a fast, correct way to evaluate the predicate.

Recognition rule. "Smallest / largest value such that a budget is respected" + an O(n) way to test one value → binary search the value.

Concepts

Defining the answer space. Choose `lo` as a value that is certainly no larger than the answer and `hi` as a value that is certainly feasible. Good bounds come from the problem's extreme cases:

Problem	lo (lower bound)	hi (always feasible)	Why
Koko (875)	1	max(piles)	At speed max, each pile takes one hour and h ≥ n is guaranteed.
Smallest divisor (1283)	1	max(nums)	Every term ceil(v / max) = 1 and threshold ≥ n.
Ship within D days (1011)	max(weights)	sum(weights)	A package cannot be split, and one ship can take everything in one day.
Split array (410)	max(nums)	sum(nums)	Some part contains the largest element, and one part is always allowed.

Monotone predicate and the boundary search. For minimisation, the predicate over [lo, hi] looks like `F F F … F T T … T`. Search the first T with the half-open discipline:

```
Invariant: answer ∈ [lo, hi] and feasible(hi) is true.

mid = lo + (hi − lo) / 2          (rounds down, so mid < hi and the loop always shrinks)
feasible(mid) ⇒ hi = mid          (mid may be the answer: keep it)
otherwise     ⇒ lo = mid + 1      (mid is infeasible, so everything ≤ mid is too)
Loop while lo < hi; on exit lo == hi == answer.
```

For maximisation (`T T … T F F`), search the last T. Round up, `mid = lo + (hi − lo + 1) / 2`, with `lo = mid` on success and `hi = mid − 1` on failure. Rounding down here makes `lo = mid` loop forever when hi = lo + 1.

Integer ceiling without floating point. For positive a and b, `ceil(a / b) = (a + b − 1) / b`, or `(a − 1) / b + 1` for a ≥ 1. The second form avoids overflow when a is near `Integer.MAX_VALUE`. Never use `Math.ceil((double) a / b)` in interviews: it is slow and, for large longs, inexact.

Overflow. The checker's accumulator (total hours, total sum) can exceed $2^{31}-1$. For Koko, 10^4 piles × 10^9 hours each overflows an `int`, so use `long`. Bounds like sum(nums) can also overflow `int` in general; LeetCode 410's constraints keep it within 10^9, but writing `long` costs nothing.

Template

Template A — minimise x subject to a monotone feasibility predicate. Reusable: the loop. Problem-specific: bounds and `feasible`.

JAVA TEMPLATE MinFeasible.java

```java
import java.util.function.LongPredicate;

/** Smallest x in [lo, hi] with feasible(x) true. Precondition:
  feasible(hi) is true and
 *  feasible is monotone (false ... false true ... true). Interval
   [lo, hi] inclusive. */
class MinFeasible {
    static long minFeasible(long lo, long hi, LongPredicate
      feasible) {
        while (lo < hi) {
            // floor; never overflows, mid < hi
            long mid = lo + (hi − lo) / 2;
            // mid might be the answer
            if (feasible.test(mid)) hi = mid;
            // mid and everything below fails
            else lo = mid + 1;
        }
        return lo;
    }

    /** Example checker (Koko-style): hours needed at speed k,
      accumulated in long. */
    static long hoursNeeded(int[] piles, long k) {
        long hours = 0;
        // ceil(p / k) for p >= 0, k >= 1
        for (int p : piles) hours += (p + k − 1) / k;
        return hours;
    }
}
```

Template B — maximise x subject to a monotone predicate (max-min problems). Note the upward rounding.

JAVA TEMPLATE MaxFeasible.java

```java
import java.util.*;
import java.util.function.LongPredicate;

/** Largest x in [lo, hi] with ok(x) true. Precondition: ok(lo) is
  true and the predicate is
 *  true ... true false ... false. */
class MaxFeasible {
    static long maxFeasible(long lo, long hi, LongPredicate ok) {
        while (lo < hi) {
            // ceil: mid > lo, so lo = mid always progresses
            long mid = lo + (hi − lo + 1) / 2;
            if (ok.test(mid)) lo = mid;
            else hi = mid − 1;
        }
        return lo;
    }

    /** Example: maximise the minimum gap when placing m items on
      sorted positions. */
    static int maxMinGap(int[] pos, int m) {
        int[] p = pos.clone();
        Arrays.sort(p);
        return (int) maxFeasible(0, p[p.length − 1] − p[0], gap -> {
            int placed = 1;
            long last = p[0];
            for (int i = 1; i < p.length; i++) {
                // greedy: place ASAP
                if (p[i] − last >= gap) { placed++; last = p[i]; }
            }
            return placed >= m;
        });
    }
}
```

Problems

LC 1283 Find the Smallest Divisor Given a Threshold — MEDIUM

Task. Find the smallest positive integer d such that Σ ceil(nums[i] / d) ≤ threshold, where threshold ≥ nums.length.

KEY INSIGHT

Each term ceil(v / d) is non-increasing in d, so the sum is too. The condition "sum ≤ threshold" therefore holds on a suffix of d values.

JAVA SOLUTION LeetCode 1283 · Solution.java

```java
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int lo = 1, hi = 1;
        for (int v : nums) hi = Math.max(hi, v);
        while (lo < hi) {
            int mid = lo + (hi − lo) / 2;
            if (sumOfCeil(nums, mid) <= threshold) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private long sumOfCeil(int[] nums, int d) {
        long s = 0;
        // ceil(v / d) for v >= 1, no overflow
        for (int v : nums) s += (v − 1) / d + 1;
        return s;
    }
}
```

Time O(n log M), M = max(nums) **Space** O(1)

LC 875 Koko Eating Bananas — MEDIUM

Task. Koko eats at most k bananas per hour from a single pile. Find the minimum integer k that finishes all piles within h hours (h ≥ piles.length).

KEY INSIGHT

The hours needed at speed k are Σ ceil(p / k), which is non-increasing in k. The minimum feasible k is the first true in a monotone predicate.

JAVA SOLUTION LeetCode 875 · Solution.java

```java
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int lo = 1, hi = 1;
        for (int p : piles) hi = Math.max(hi, p);
        while (lo < hi) {
            int mid = lo + (hi − lo) / 2;
            if (hoursAt(piles, mid) <= h) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private long hoursAt(int[] piles, int k) {
        // up to 1e4 piles * 1e9 hours: needs long
        long hours = 0;
        for (int p : piles) hours += (p − 1) / k + 1;
        return hours;
    }
}
```

Time O(n log M) **Space** O(1)

LC 1011 Capacity To Ship Packages Within D Days — MEDIUM

Task. Ship packages in the given order, with each day's load at most the capacity. Find the minimum capacity that finishes within `days` days.

KEY INSIGHT

For a fixed capacity, greedily packing each day as full as possible minimises the number of days (see the exchange argument in 6.3), and the day count is non-increasing in capacity.

JAVA SOLUTION LeetCode 1011 · Solution.java

```java
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int lo = 0, hi = 0;
        // sum <= 5e4 * 500
        for (int w : weights) { lo = Math.max(lo, w); hi += w; }
        while (lo < hi) {
            int mid = lo + (hi − lo) / 2;
            if (daysNeeded(weights, mid) <= days) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    /** Greedy: fill the current day until the next package would
      overflow it. */
```

```
    private int daysNeeded(int[] weights, int cap) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d;
    }
}
```

Time O(n log(Σw)) **Space** O(1)

LC 410 Split Array Largest Sum — HARD

Task. Split `nums` (non-negative) into k non-empty contiguous subarrays to minimise the largest subarray sum.

KEY INSIGHT

"Can every part be kept ≤ S using at most k parts?" is monotone in S. If the greedy needs at most k parts, exactly k non-empty parts are also achievable, because any part with two or more elements can be split without raising the maximum (n ≥ k is guaranteed).

JAVA SOLUTION LeetCode 410 · Solution.java

```
class Solution {
    public int splitArray(int[] nums, int k) {
        long lo = 0, hi = 0;
        for (int v : nums) { lo = Math.max(lo, v); hi += v; }
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            // "at most k" is monotone in mid
            if (partsNeeded(nums, mid) <= k) hi = mid;
            else lo = mid + 1;
        }
        return (int) lo;
    }

    private int partsNeeded(int[] nums, long cap) {
        int parts = 1;
        long cur = 0;
        for (int v : nums) {
            if (cur + v > cap) { parts++; cur = 0; }
            cur += v;
        }
        return parts;
    }
}
```

Time O(n log Σ) **Space** O(1)

PATTERN 07

Merge Intervals: Sorting and Sweep

Key insight. Sorting by start converts a pairwise O(n²) overlap relation into a single running value, the end of the current block. Counting overlaps instead requires splitting each interval into a +1 event and a −1 event and sweeping them in time order.

Recognition rule. Intervals + "combine / gaps / how many at once" → sort by start and sweep, keeping one running end or a heap of ends.

Concepts

Overlap conditions. Closed intervals [a, b] and [c, d] overlap iff `max(a, c) ≤ min(b, d)`; they are disjoint iff `b < c` or `d < a`. For half-open [a, b) and [c, d), replace ≤ with <, so touching intervals do *not* overlap.

Problem	Convention	[1,4] with [4,5]	Code test
56 Merge Intervals	Closed	Overlap → [1,5]	`cur.start <= last.end`
57 Insert Interval	Closed	Overlap	`iv.end < new.start` → strictly before
253 Meeting Rooms II	Half-open (end frees the room)	Same room reusable	`minEnd <= start` → reuse
759 Employee Free Time	Free time must have positive length	No gap	`start > prevEnd` → gap

Merge invariant. While scanning intervals sorted by start, `last` is the union of everything seen that is connected to the most recent block, and every earlier output block ends strictly before `last.start`. Proof: if `cur.start > last.end`, then every later interval has start ≥ cur.start > last.end, so `last` is final.

Sweep-line invariant. Map each interval to events (start, +1) and (end, −1), and sort by time. At every point the running sum equals the number of intervals covering that instant, and the answer is its maximum. **Tie-breaking encodes the boundary convention**: for half-open intervals, process −1 before +1 at equal times, so a meeting ending at 10 frees its room for one starting at 10. For closed intervals, process +1 first.

```
rooms(t) = #{ i : start_i ≤ t < end_i }        (half-open)
answer   = max over event times t of rooms(t)
Heap view: keep the end times of active meetings; a new meeting at s
reuses a room iff min(end) ≤ s.
```

Template

Template A — sort and merge (closed intervals).

JAVA TEMPLATE MergeTemplate.java

```
import java.util.*;

/** Merge closed intervals [s, e]. Touching intervals ([1,4],[4,5])
  are merged. */
class MergeTemplate {
    static int[][] merge(int[][] intervals) {
        // do not reorder the caller's array
        int[][] a = intervals.clone();
        // no subtraction comparator
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
        List<int[]> out = new ArrayList<>();
        for (int[] iv : a) {
            int[] last = out.isEmpty() ? null : out.get(out.size() -
              1);
            // overlaps current block
            if (last != null && iv[0] <= last[1]) {
                // a contained interval keeps end
                last[1] = Math.max(last[1], iv[1]);
            } else {
                // copy: never alias the input
                out.add(new int[]{iv[0], iv[1]});
            }
        }
        return out.toArray(new int[0][]);
    }
}
```

Template B — event sweep for maximum overlap. The comparator's tie rule is the boundary convention.

JAVA TEMPLATE SweepTemplate.java

```
import java.util.*;

/** Maximum number of simultaneously active intervals. halfOpen =
  true means [s, e). */
class SweepTemplate {
    static int maxOverlap(int[][] intervals, boolean halfOpen) {
        int n = intervals.length;
        int[][] ev = new int[2 * n][];
        for (int i = 0; i < n; i++) {
            ev[2 * i] = new int[]{intervals[i][0], +1};
            ev[2 * i + 1] = new int[]{intervals[i][1], -1};
        }
        // Equal times: half-open -> end (-1) first; closed -> start
          (+1) first.
        Arrays.sort(ev, (x, y) -> x[0] != y[0] ?
          Integer.compare(x[0], y[0])
                : halfOpen ? Integer.compare(x[1], y[1]) :
                  Integer.compare(y[1], x[1]));
        int cur = 0, best = 0;
        for (int[] e : ev) {
            cur += e[1];
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Problems

LC 56 Merge Intervals — MEDIUM

Task. Merge all overlapping closed intervals and return the non-overlapping result covering the same points.

KEY INSIGHT

Sorted by start, each interval either extends the current block (start ≤ block end) or proves the block final.

JAVA SOLUTION LeetCode 56 · Solution.java

```
import java.util.*;
```

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0],
          b[0]));
        List<int[]> merged = new ArrayList<>();
        // 1 <= intervals.length
        int[] cur = intervals[0].clone();
        for (int i = 1; i < intervals.length; i++) {
            int[] iv = intervals[i];
            if (iv[0] <= cur[1]) {
                // contained intervals keep cur[1]
                cur[1] = Math.max(cur[1], iv[1]);
            } else {
                merged.add(cur);
                cur = iv.clone();
            }
        }
        // flush the final block
        merged.add(cur);
        return merged.toArray(new int[merged.size()][]);
    }
}
```

Time O(n log n) **Space** O(n) output + sort space

LC 57 Insert Interval — MEDIUM

Task. Insert `newInterval` into a sorted list of disjoint closed intervals, merging where necessary.

KEY INSIGHT

The input is already sorted and disjoint, so the result splits into three contiguous phases: intervals strictly before the new one, intervals overlapping it (absorbed into one), and intervals strictly after it. No sort is needed, which gives O(n).

JAVA SOLUTION LeetCode 57 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> out = new ArrayList<>();
        int i = 0, n = intervals.length;
        int start = newInterval[0], end = newInterval[1];
        // strictly before
        while (i < n && intervals[i][1] < start)
          out.add(intervals[i++]);
        // overlapping
        while (i < n && intervals[i][0] <= end) {
            start = Math.min(start, intervals[i][0]);
            end = Math.max(end, intervals[i][1]);
            i++;
        }
        out.add(new int[]{start, end});
        // strictly after
        while (i < n) out.add(intervals[i++]);
        return out.toArray(new int[out.size()][]);
    }
}
```

Time O(n) **Space** O(n) output

LC 253 Meeting Rooms II — MEDIUM — Premium

Task. Given meeting time intervals, return the minimum number of conference rooms required.

KEY INSIGHT

The answer is the maximum number of meetings in progress at once. Meetings are half-open: a room freed at time 10 can host a meeting starting at 10.

JAVA SOLUTION LeetCode 253 · Solution.java

```java
import java.util.*;

class Solution {
    public int minMeetingRooms(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0],
          b[0]));
        // end times of busy rooms
        PriorityQueue<Integer> roomEnds = new PriorityQueue<>();
        for (int[] m : intervals) {
            if (!roomEnds.isEmpty() && roomEnds.peek() <= m[0]) {
                // earliest room is free by m's start: reuse it
                roomEnds.poll();
            }
            roomEnds.offer(m[1]);
        }
        return roomEnds.size();
    }
}
```

Time O(n log n) **Space** O(n) heap

LC 759 Employee Free Time — HARD — Premium

Task. Each employee has a sorted list of non-overlapping working `Interval`s. Return the finite, positive-length intervals during which *all* employees are free, in sorted order.

KEY INSIGHT

Common free time is exactly the set of gaps in the *union* of all working intervals, restricted to the span between the first start and the last end. So: merge everything, then report the gaps.

JAVA SOLUTION LeetCode 759 · Solution.java (Interval is provided by LeetCode)

```java
import java.util.*;

class Solution {
    public List<Interval> employeeFreeTime(List<List<Interval>>
      schedule) {
        // Heap entries: {employee, index}; ordered by that
          interval's start.
        PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) ->
          Integer.compare(
                schedule.get(x[0]).get(x[1]).start,
                  schedule.get(y[0]).get(y[1]).start));
        for (int e = 0; e < schedule.size(); e++) {
            if (!schedule.get(e).isEmpty()) pq.offer(new int[]{e,
              0});
        }
        List<Interval> free = new ArrayList<>();
        // sentinel: no work seen yet
        int prevEnd = Integer.MIN_VALUE;
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            Interval iv = schedule.get(top[0]).get(top[1]);
            if (prevEnd != Integer.MIN_VALUE && iv.start > prevEnd)
              {
                // positive-length gap only
                free.add(new Interval(prevEnd, iv.start));
            }
            prevEnd = Math.max(prevEnd, iv.end);
            if (top[1] + 1 < schedule.get(top[0]).size()) {
                pq.offer(new int[]{top[0], top[1] + 1});
            }
        }
        return free;
    }
}
```

Time O(N log k), N intervals, k employees **Space** O(k) heap + output

PATTERN 08

Backtracking: DFS with Choices

Key insight. Correctness comes from two things: the choice tree generates every valid configuration *exactly once* (canonical ordering), and pruning removes only subtrees that provably contain no solution.

Recognition rule. "Return all valid configurations" with small n → a choice tree with undo. "How many" with repeated states → memoised DFS.

Concepts

Choose → explore → undo. Maintain the invariant "*on entry to `dfs`, the shared state equals exactly the choices on the current root-to-node path*". Every mutation before the recursive call must be reversed after it. Copy the path into the output only when recording (`new ArrayList<>(path)`), because the path object keeps changing.

Combinations vs permutations.

	Combination / subset	Permutation
Order matters?	No: {1,2} = {2,1}	Yes
Canonical generation	A `start` index: only choose elements at positions ≥ start (increasing indices)	A `used[]` array: any unused element at every level
Reuse allowed (39)	Recurse with `i` instead of `i + 1`	Drop `used[]`
Leaves / outputs	2^n subsets, C(n, k) of size k	n! permutations
Cost	O(n · 2^n) including copying	O(n · n!)

Duplicate avoidance with repeated values. Sort first. At one tree level (one loop over i), skip a value equal to its predecessor at that level:

```
Combinations (Subsets II, Combination Sum II):  if (i > start && a[i]
== a[i-1]) continue;
Permutations (Permutations II):                 if (i > 0 && a[i] ==
a[i-1] && !used[i-1]) continue;
```
Meaning: among equal values, always pick them left to right. Choosing a later copy first would produce the same multiset or sequence again.

Pruning. A prune is valid when it is *sound*, meaning no solution lies in the cut subtree. Typical sound prunes: remaining target < 0 (with positive candidates), sorted candidates with `a[i] > remain` → `break` (every later candidate is larger), and conflict checks in O(1) using auxiliary arrays (N-Queens columns and diagonals). Pruning does not change the worst case for output-bound problems, but it often decides whether a constraint-satisfaction search finishes at all.

Template

Template A — combinations and subsets (start index, optional duplicate skip).

JAVA TEMPLATE CombinationBacktracking.java

```java
import java.util.*;

/** All subsets of a (may contain duplicates; each distinct multiset
  reported once). */
class CombinationBacktracking {
    static List<List<Integer>> subsetsWithDup(int[] a) {
        int[] s = a.clone();
        // duplicates adjacent for skipping
        Arrays.sort(s);
        List<List<Integer>> out = new ArrayList<>();
        dfs(s, 0, new ArrayList<>(), out);
        return out;
    }

    private static void dfs(int[] a, int start, List<Integer> path,
                            List<List<Integer>> out) {
        // every node is a subset: record copy
        out.add(new ArrayList<>(path));
        for (int i = start; i < a.length; i++) {
            // same value at same level: skip
            if (i > start && a[i] == a[i - 1]) continue;
            path.add(a[i]);                                  // choose
            // explore (i + 1: no reuse)
            dfs(a, i + 1, path, out);
            path.remove(path.size() - 1);                    // undo
        }
    }
}
```

Template B — permutations (used[] array, duplicate-safe).

JAVA TEMPLATE PermutationBacktracking.java

```java
import java.util.*;

/** Distinct permutations of a (handles repeated values). */
class PermutationBacktracking {
    static List<List<Integer>> permuteUnique(int[] a) {
        int[] s = a.clone();
        Arrays.sort(s);
        List<List<Integer>> out = new ArrayList<>();
        dfs(s, new boolean[s.length], new ArrayList<>(), out);
        return out;
    }

    private static void dfs(int[] a, boolean[] used, List<Integer>
      path,
                            List<List<Integer>> out) {
        if (path.size() == a.length) { out.add(new ArrayList<>
          (path)); return; }
        for (int i = 0; i < a.length; i++) {
            if (used[i]) continue;
            // Equal values are used left to right: skip copy i
              while copy i-1 is unused.
            if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
            used[i] = true; path.add(a[i]);
            dfs(a, used, path, out);
            path.remove(path.size() - 1); used[i] = false;
        }
    }
}
```

Problems

LC 78 Subsets — MEDIUM

Task. Return all subsets (the power set) of an array of distinct integers.

KEY INSIGHT

Generating subsets in increasing index order makes each one appear exactly once. Every node of the choice tree, not just the leaves, is an answer.

JAVA SOLUTION LeetCode 78 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int[] nums, int start, List<Integer>
      path,
                           List<List<Integer>> res) {
        // each node of the tree is one subset
        res.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            backtrack(nums, i + 1, path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Time $O(n \cdot 2^n)$ **Space** O(n) recursion + path (output excluded)

LC 46 Permutations — MEDIUM

Task. Return all permutations of an array of distinct integers.

KEY INSIGHT

At depth d, any not-yet-used element can occupy position d. The `used[]` array is the set of choices remaining on the current path.

JAVA SOLUTION LeetCode 46 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(),
          res);
        return res;
    }

    private void backtrack(int[] nums, boolean[] used, List<Integer>
      path,
                           List<List<Integer>> res) {
        if (path.size() == nums.length) {
            res.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            path.add(nums[i]);
            backtrack(nums, used, path, res);
            // undo in reverse order of doing
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }
}
```

Time O(n · n!) **Space** O(n) recursion + used[] (output excluded)

LC 39 Combination Sum — MEDIUM

Task. Given distinct positive candidates and a target, return all unique combinations (each candidate reusable without limit) that sum to the target.

KEY INSIGHT

Reuse is allowed, so recurse with the same index `i`. Uniqueness of combinations is still enforced by never going back to smaller indices.

JAVA SOLUTION LeetCode 39 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int
      target) {
        // enables the break-prune below
        Arrays.sort(candidates);
        List<List<Integer>> res = new ArrayList<>();
        dfs(candidates, 0, target, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] c, int start, int remain, List<Integer>
      path,
                     List<List<Integer>> res) {
        if (remain == 0) {
            res.add(new ArrayList<>(path));
            return;
        }
        for (int i = start; i < c.length; i++) {
            // sorted: every later c[j] is larger too
            if (c[i] > remain) break;
            path.add(c[i]);
            // i, not i + 1: c[i] may be reused
            dfs(c, i, remain - c[i], path, res);
```

```
            path.remove(path.size() - 1);
        }
    }
}
```

Time $O(N^{T/m + 1})$ worst case (T target, m min candidate)
Space O(T/m) recursion depth

LC 51 N-Queens — HARD

Task. Return all distinct placements of n non-attacking queens on an n × n board, as lists of strings.

KEY INSIGHT

Place exactly one queen per row, so rows never conflict. A column, a "\" diagonal (`r - c` constant) or a "/" diagonal (`r + c` constant) is each a single index into a boolean array, which makes every conflict check O(1).

JAVA SOLUTION LeetCode 51 · Solution.java

```
import java.util.*;

class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        dfs(0, n, new int[n], new boolean[n], new boolean[2 * n -
          1],
            new boolean[2 * n - 1], res);
        return res;
    }

    private void dfs(int row, int n, int[] queenCol, boolean[] cols,
      boolean[] diag,
                     boolean[] anti, List<List<String>> res) {
        if (row == n) {
            res.add(render(queenCol, n));
            return;
        }
        for (int c = 0; c < n; c++) {
            // shift r - c from [-(n-1), n-1] to [0, 2n-2]
            int d = row - c + n - 1;
            // r + c in [0, 2n-2]
            int a = row + c;
            if (cols[c] || diag[d] || anti[a]) continue;
            cols[c] = diag[d] = anti[a] = true;
            queenCol[row] = c;
            dfs(row + 1, n, queenCol, cols, diag, anti, res);
            cols[c] = diag[d] = anti[a] = false;      // undo
        }
    }

    private List<String> render(int[] queenCol, int n) {
        List<String> board = new ArrayList<>(n);
        for (int r = 0; r < n; r++) {
            char[] line = new char[n];
            Arrays.fill(line, '.');
            line[queenCol[r]] = 'Q';
            board.add(new String(line));
        }
        return board;
    }
}
```

Time O(n!) upper bound (+ $O(n^2)$ per solution to render)
Space O(n) flags and recursion

PATTERN 09

Recursion / DFS on Trees and Graphs

Key insight. Define the function's contract in one sentence, for example "`gain(node)` = the best downward path sum starting at node". Then trust it for the children. Correctness follows by structural induction: if the contract holds for the children, the combination step makes it hold for the node.

Recognition rule. The answer at a node is composed from its children, or the question is about components or reachability, so use DFS with a clear return contract and a visited set.

Concepts

Traversal order is a modelling choice.

Order	Work happens...	Use when
Preorder	Before the children; passes information *down* (depth, path so far, bounds)	Path sum from the root, BST validation with (lo, hi) bounds, serialisation
Inorder	Between the children	BSTs: visits keys in sorted order (k-th smallest, validate BST)
Postorder	After the children; aggregates information *up*	Height, diameter, max path sum, subtree sums, tree DP

Return value vs global state. For Binary Tree Maximum Path Sum the parent can only extend a *downward* path, one that uses at most one child, so that is what `gain()` returns. The best path through a node may use both children, but it cannot be extended upward, so it is recorded in a global maximum. The same split appears in diameter, longest univalue path and house robber III, where the returned value is a pair.

```
gain(v)  = v.val + max(0, gain(v.left), gain(v.right))
(returned: extendable)
best     = max over v of  v.val + max(0, gain(v.left)) + max(0,
gain(v.right))   (recorded)
gain(null) = 0
```

Visitation rules on graphs. Mark a node visited *when you first reach it* (before or as you recurse), never after processing it. Otherwise two neighbours may both start on the same node, and cycles recurse forever. In grids you may mark in place (e.g. overwrite '1' with '0'), which saves memory but destroys the input. Say so, or copy the grid.

Connected components. The outer loop over all nodes launches one DFS per unvisited node. The number of launches equals the number of components, and each node and edge is touched O(1) times, so the total is O(V + E): $O(n^2)$ for an adjacency matrix, O(mn) for an m × n grid.

Template

Template A — tree DFS: return what the parent needs, record the global answer (illustrated with the tree diameter).

JAVA TEMPLATE TreeDfsTemplate.java (TreeNode provided by LeetCode)

```
/** Postorder skeleton. height() returns information for the parent;
  best records answers
 *  that combine both children (here: diameter in edges). */
class TreeDfsTemplate {
    private int best = 0;

    int diameter(TreeNode root) {
        best = 0;
        height(root);
        return best;
    }

    /** Contract: number of nodes on the longest downward path from
      node (0 for null). */
    private int height(TreeNode node) {
        if (node == null) return 0;
        // children first (postorder)
        int l = height(node.left), r = height(node.right);
        // path through node: both sides
        best = Math.max(best, l + r);
        // parent can extend one side only
        return 1 + Math.max(l, r);
    }
}
```

Template B — grid DFS, recursive and iterative. The iterative version is the safe default for very large grids.

JAVA TEMPLATE GridDfsTemplate.java

```
import java.util.*;

/** Count 4-connected regions of cells equal to target. Visited
  tracked separately
 *  (input not mutated). */
class GridDfsTemplate {
    private static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1},
      {0, -1}};

    static int countRegions(char[][] g, char target, boolean
      iterative) {
        int m = g.length, n = g[0].length, regions = 0;
        boolean[][] seen = new boolean[m][n];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (g[i][j] == target && !seen[i][j]) {
                    regions++;
                    if (iterative) fillIterative(g, i, j, target,
                      seen);
                    else fillRecursive(g, i, j, target, seen);
                }
        return regions;
    }
```

```
    private static void fillRecursive(char[][] g, int r, int c, char
      t, boolean[][] seen) {
        // bounds first
        if (r < 0 || c < 0 || r >= g.length || c >= g[0].length)
          return;
        if (seen[r][c] || g[r][c] != t) return;
        // mark on entry
        seen[r][c] = true;
        for (int[] d : DIRS) fillRecursive(g, r + d[0], c + d[1], t,
          seen);
    }

    private static void fillIterative(char[][] g, int r, int c, char
      t, boolean[][] seen) {
        Deque<int[]> stack = new ArrayDeque<>();
        // mark when pushed
        seen[r][c] = true;
        stack.push(new int[]{r, c});
        while (!stack.isEmpty()) {
            int[] cur = stack.pop();
            for (int[] d : DIRS) {
                int nr = cur[0] + d[0], nc = cur[1] + d[1];
                if (nr < 0 || nc < 0 || nr >= g.length || nc >=
                  g[0].length) continue;
                if (seen[nr][nc] || g[nr][nc] != t) continue;
                seen[nr][nc] = true;
                stack.push(new int[]{nr, nc});
            }
        }
    }
}
```

Problems

LC 104 Maximum Depth of Binary Tree — EASY

Task. Return the number of nodes on the longest root-to-leaf path.

KEY INSIGHT

depth(node) = 1 + max(depth(left), depth(right)), with depth(null) = 0. This is the smallest example of a postorder aggregate.

JAVA SOLUTION LeetCode 104 · Solution.java (TreeNode provided by LeetCode)

```
class Solution {
    public int maxDepth(TreeNode root) {
        // empty subtree contributes 0 nodes
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left),
          maxDepth(root.right));
    }
}
```

Time O(n) **Space** O(h) recursion; O(n) when skewed

LC 200 Number of Islands — MEDIUM

Task. Count the 4-directionally connected groups of '1' cells in a grid of '1'/'0' characters.

KEY INSIGHT

Each island is one connected component of an implicit graph. Every unvisited land cell found by the outer scan starts a new island, and the DFS consumes the whole island so it is never counted twice.

JAVA SOLUTION LeetCode 200 · Solution.java

```
class Solution {
    public int numIslands(char[][] grid) {
        int islands = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == '1') {
                    islands++;
                    // remove the whole island
                    sink(grid, i, j);
                }
            }
        }
        return islands;
    }

    private void sink(char[][] g, int r, int c) {
        if (r < 0 || c < 0 || r >= g.length || c >= g[0].length ||
          g[r][c] != '1') return;
        // mark visited in place (mutates input)
        g[r][c] = '0';
        sink(g, r + 1, c);
        sink(g, r - 1, c);
        sink(g, r, c + 1);
        sink(g, r, c - 1);
    }
}
```

Time O(m · n) **Space** O(m · n) worst-case recursion depth

LC 547 Number of Provinces — MEDIUM

Task. Given an n × n adjacency matrix `isConnected` of an undirected graph, return the number of connected components.

KEY INSIGHT

A province is a connected component. One DFS launch per unvisited city counts them. Because the input is a matrix, scanning a city's neighbours costs O(n), so the total is $O(n^2)$, which is optimal for this input format.

JAVA SOLUTION LeetCode 547 · Solution.java

```
class Solution {
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, provinces = 0;
        boolean[] visited = new boolean[n];
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                // i starts a new component
                provinces++;
                dfs(isConnected, i, visited);
            }
        }
        return provinces;
    }

    private void dfs(int[][] g, int i, boolean[] visited) {
        visited[i] = true;
        // undirected: matrix is symmetric
        for (int j = 0; j < g.length; j++) {
            if (g[i][j] == 1 && !visited[j]) dfs(g, j, visited);
        }
    }
}
```

Time $O(n^2)$ **Space** O(n) visited + recursion

LC 124 Binary Tree Maximum Path Sum — HARD

Task. A path is any sequence of adjacent nodes, each used at most once, with at least one node. Return the maximum path sum; values may be negative.

KEY INSIGHT

Every path has a unique highest node (its apex). At the apex the path is `val + left downward gain + right downward gain`, where a negative gain is replaced by 0 (don't take that branch). The parent, however, can only extend a single downward chain.

JAVA SOLUTION LeetCode 124 · Solution.java (TreeNode provided by LeetCode)

```
class Solution {
    private int best;

    public int maxPathSum(TreeNode root) {
        // all values may be negative
        best = Integer.MIN_VALUE;
        gain(root);
        return best;
    }

    /** Max sum of a downward path starting at node (may be just
      node); 0 for null. */
    private int gain(TreeNode node) {
        if (node == null) return 0;
        // drop a branch that only hurts
        int l = Math.max(0, gain(node.left));
        int r = Math.max(0, gain(node.right));
        // path with apex at node
        best = Math.max(best, node.val + l + r);
        // extendable: one side only
        return node.val + Math.max(l, r);
    }
}
```

Time O(n) **Space** O(h) recursion

PATTERN 10

BFS and Level Order: Shortest Path in Unweighted Graphs

Key insight. BFS is Dijkstra with a priority queue that happens to be FIFO, which works only because every edge adds exactly 1. Change one weight and the guarantee disappears.

Concepts

The layer invariant. At any moment the queue contains nodes from at most two consecutive layers, d and d + 1, with every layer-d node ahead of every layer-(d+1) node.

```
Claim: when node v is first enqueued from u, dist(v) = dist(u) + 1.
Proof sketch: nodes are dequeued in non-decreasing dist order (layer
invariant).
  If a shorter path to v existed, its predecessor w would have
dist(w) < dist(u)
  and was dequeued earlier, so it would already have enqueued v.
Contradiction.
```

Visited-state timing. Mark a node visited *when it is enqueued*, not when it is dequeued. Marking at dequeue lets the same node be enqueued many times by different neighbours in the same layer. Distances stay correct, but the queue can grow to O(E), or exponentially in implicit graphs, and lead to TLE or MLE. The exception is weighted search (Dijkstra), where the distance is final only at dequeue time.

Level boundaries. Two equivalent techniques:

- **Size snapshot:** `for (int k = q.size(); k > 0; k--)` processes exactly one layer, and the loop counter is the level.
- **Distance array:** store `dist[v] = dist[u] + 1` at enqueue time; no layer loop is needed. This is more flexible (multi-source, and the visited test becomes `dist[v] == -1`).

Multi-source BFS. Enqueue all sources with distance 0 before starting. This is the same as adding a virtual super-source S with a 0-weight edge to each real source, then running BFS from S. The result is `dist[v] = min over sources s of dist(s, v)` in a single O(V + E) pass, compared with O(k · (V + E)) for k separate BFS runs.

Template

Template A — single/multi-source BFS on a grid with a distance array. Reusable: queue, directions, bounds check, enqueue-time marking. Problem-specific: what counts as passable, and which cells are sources.

JAVA TEMPLATE GridBfs.java

```java
import java.util.*;

/** Multi-source grid BFS: dist[r][c] = fewest 4-directional steps
  to the nearest source. */
class GridBfs {
    static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    /** passable[r][c] says whether a cell may be entered; sources
      are {r, c} pairs. */
    static int[][] distances(boolean[][] passable, List<int[]>
      sources) {
        int m = passable.length, n = passable[0].length;
        int[][] dist = new int[m][n];
        // -1 means "not yet discovered"
        for (int[] row : dist) Arrays.fill(row, -1);
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int[] s : sources) {
            if (dist[s[0]][s[1]] == -1) {
                // all sources are layer 0
                dist[s[0]][s[1]] = 0;
                q.add(s);
            }
        }
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || r >= m || c < 0 || c >= n) continue;
                if (!passable[r][c] || dist[r][c] != -1) continue;
                // mark at ENQUEUE time
                dist[r][c] = dist[cur[0]][cur[1]] + 1;
                q.add(new int[]{r, c});
            }
        }
        return dist;
    }
}
```

Template B — level-order BFS with a size snapshot (adjacency list or implicit graph).

Recognition rule. Fewest equal-cost moves, or simultaneous spreading → BFS (multi-source when there are many starts).

JAVA TEMPLATE LevelBfs.java

```java
import java.util.*;

/** BFS with explicit levels. Unweighted graph; adj edges are
  directed as given. */
class LevelBfs {
    /** Returns the fewest edges from src to dst, or -1 if dst is
      unreachable. */
    static int shortest(List<List<Integer>> adj, int src, int dst) {
        if (src == dst) return 0;
        boolean[] seen = new boolean[adj.size()];
        ArrayDeque<Integer> q = new ArrayDeque<>();
        q.add(src);
        seen[src] = true;
        for (int level = 1; !q.isEmpty(); level++) {
            // exactly one layer
            for (int k = q.size(); k > 0; k--) {
                int u = q.poll();
                for (int v : adj.get(u)) {
                    if (seen[v]) continue;
                    // early exit on first discovery
                    if (v == dst) return level;
                    seen[v] = true;
                    q.add(v);
                }
            }
        }
        return -1;
    }
}
```

Problems

LC 733 Flood Fill — EASY

Task. Recolour the 4-connected region of pixels sharing the start pixel's colour with `color`.

KEY INSIGHT

The region is the connected component of the start pixel under the "same original colour" relation. Overwriting the colour can serve as the visited mark, but only when the new colour differs from the old one.

JAVA SOLUTION LeetCode 733 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int
      color) {
        int original = image[sr][sc];
        // also prevents infinite re-visiting
        if (original == color) return image;
        int m = image.length, n = image[0].length;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        ArrayDeque<int[]> q = new ArrayDeque<>();
        // recolouring doubles as the visited mark
        image[sr][sc] = color;
        q.add(new int[]{sr, sc});
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r >= 0 && r < m && c >= 0 && c < n && image[r]
                  [c] == original) {
                    // mark at enqueue time
                    image[r][c] = color;
                    q.add(new int[]{r, c});
                }
            }
        }
        return image;
    }
}
```

Time O(m · n) **Space** O(m · n) queue in the worst case

LC 994 Rotting Oranges — MEDIUM

Task. Each minute, rotten oranges (2) rot their fresh 4-neighbours (1). Return the minutes until no fresh orange remains, or −1 if that is impossible.

KEY INSIGHT

All rotten oranges spread simultaneously, so the time at which a fresh orange rots is its BFS distance to the *nearest* initially rotten orange. The answer is the maximum such distance.

JAVA SOLUTION LeetCode 994 · Solution.java

```java
import java.util.*;

class Solution {
    public int orangesRotting(int[][] grid) {
        int m = grid.length, n = grid[0].length, fresh = 0;
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                // every source at minute 0
                if (grid[r][c] == 2) q.add(new int[]{r, c});
                else if (grid[r][c] == 1) fresh++;
            }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int minutes = 0;
        // stop once nothing is left to rot
        while (!q.isEmpty() && fresh > 0) {
            minutes++;
            // one minute = one BFS layer
            for (int k = q.size(); k > 0; k--) {
                int[] cur = q.poll();
                for (int[] d : dirs) {
                    int r = cur[0] + d[0], c = cur[1] + d[1];
                    if (r < 0 || r >= m || c < 0 || c >= n ||
                      grid[r][c] != 1) continue;
                    // mark at enqueue time
                    grid[r][c] = 2;
                    fresh--;
                    q.add(new int[]{r, c});
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

Time O(m · n) **Space** O(m · n)

LC 542 01 Matrix — MEDIUM

Task. For each cell of a binary matrix, return the distance (4-directional steps) to the nearest 0. At least one 0 exists.

KEY INSIGHT

Reverse the question: rather than searching outward from every 1 (O((mn)²)), search outward from *all zeros at once*. Each cell is discovered first from its nearest zero.

JAVA SOLUTION LeetCode 542 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int m = mat.length, n = mat[0].length;
        int[][] dist = new int[m][n];
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                // all zeros are sources
                if (mat[r][c] == 0) q.add(new int[]{r, c});
                // undiscovered
                else dist[r][c] = -1;
            }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || r >= m || c < 0 || c >= n || dist[r][c]
                  != -1) continue;
                dist[r][c] = dist[cur[0]][cur[1]] + 1;
                q.add(new int[]{r, c});
            }
        }
        return dist;
    }
}
```

Time O(m · n) **Space** O(m · n) (queue; the result matrix is required output)

LC 127 Word Ladder — HARD

Task. Return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, where each step changes one letter and every intermediate word is in `wordList`. Return 0 if none exists.

KEY INSIGHT

Words are nodes of an implicit unweighted graph; edges connect words at Hamming distance 1. Generate neighbours by substituting each of the 26 letters at each position and checking a hash set. This is O(L · 26) per word, instead of comparing against all N words.

JAVA SOLUTION LeetCode 127 · Solution.java

```java
import java.util.*;

class Solution {
    public int ladderLength(String beginWord, String endWord,
      List<String> wordList) {
        Set<String> unvisited = new HashSet<>(wordList);
        if (!unvisited.contains(endWord)) return 0;
        ArrayDeque<String> q = new ArrayDeque<>();
        q.add(beginWord);
        unvisited.remove(beginWord);
        // words = sequence length so far
        for (int words = 1; !q.isEmpty(); words++) {
            for (int k = q.size(); k > 0; k--) {
                char[] w = q.poll().toCharArray();
                for (int i = 0; i < w.length; i++) {
                    char orig = w[i];
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        if (ch == orig) continue;
                        w[i] = ch;
                        String next = new String(w);
                        // not in dict, or seen
                        if (!unvisited.remove(next)) continue;
                        if (next.equals(endWord)) return words + 1;
                        q.add(next);
                    }
                    // undo the substitution
                    w[i] = orig;
                }
            }
        }
        return 0;
    }
}
```

Time O(N · L² · 26): N words, each with L positions × 26 letters, and each candidate costs O(L) to build and hash

Space O(N · L)

PATTERN 11

Topological Sort: DFS / Kahn's Algorithm

> **Key insight.** Ordering is possible if and only if there is no directed cycle. Kahn's algorithm detects a cycle when it emits fewer than n vertices; DFS detects one when it meets a *gray* (in-progress) vertex.

> **Recognition rule.** Directed "before/after" constraints + "find an order" or "is it possible" → topological sort (and its built-in cycle check).

Concepts

Kahn's invariant. The queue holds exactly the vertices whose indegree has dropped to 0 and which have not yet been emitted. Every emitted vertex has had all its predecessors emitted before it.

```
indeg(v) = number of edges (u → v) with u not yet emitted
Emit u  ⇒  for each u → v: indeg(v)--, and enqueue v when indeg(v)
becomes 0

Termination: emitted == n  ⇔  graph is a DAG
If a cycle exists, no vertex on it can ever reach indegree 0 (each
has a predecessor on the cycle).
```

DFS colouring. White = unvisited, gray = on the current recursion stack, black = finished (all descendants explored).

- An edge to a **gray** vertex is a back edge, which proves a directed cycle.
- An edge to a **black** vertex is safe: that vertex's subtree is complete and acyclic.
- When a vertex turns black, every vertex reachable from it is already black. Appending on finish gives *reverse* topological order, so reverse the list (or prepend).

Choosing between them. Kahn is iterative (no recursion-depth risk), naturally gives a lexicographically smallest order with a priority queue, and supports "process in parallel rounds" questions (the number of BFS layers is the minimum number of semesters). DFS is shorter when you already have a recursive traversal, and its colour states answer "which nodes can reach a cycle?" (Problem 802).

Template

Template A — Kahn's algorithm (BFS on indegrees). Directed graph: `edges[i] = {u, v}` means u → v (u before v). Returns an order, or an empty array if there is a cycle.

JAVA TEMPLATE KahnTopoSort.java

```java
import java.util.*;

/** Kahn's algorithm. Directed edges u -> v mean "u must come before
  v". */
class KahnTopoSort {
    static int[] order(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[] indeg = new int[n];
        for (int[] e : edges) {
            adj.get(e[0]).add(e[1]);
            indeg[e[1]]++;
        }
        // PriorityQueue -> lexicographic
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int v = 0; v < n; v++) if (indeg[v] == 0) q.add(v);
        int[] out = new int[n];
        int k = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            // all predecessors already emitted
            out[k++] = u;
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.add(v);
        }
        // k < n  <=>  a directed cycle exists
        return k == n ? out : new int[0];
    }
}
```

Template B — DFS with three colours and reverse postorder. Iterative alternatives exist, but recursion is the interview norm. Note that the depth can reach V.

JAVA TEMPLATE DfsTopoSort.java

```java
import java.util.*;

/** DFS topological sort with WHITE/GRAY/BLACK colouring. Directed
  edges u -> v. */
class DfsTopoSort {
    static final int WHITE = 0, GRAY = 1, BLACK = 2;

    static List<Integer> order(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) adj.get(e[0]).add(e[1]);
        int[] color = new int[n];
        List<Integer> post = new ArrayList<>();
        // cover every component
        for (int v = 0; v < n; v++)
            if (color[v] == WHITE && !dfs(v, adj, color, post))
              return new ArrayList<>();
        // reverse postorder = topological
        Collections.reverse(post);
        return post;
    }

    /** Returns false if a directed cycle is reachable from u. */
    static boolean dfs(int u, List<List<Integer>> adj, int[] color,
      List<Integer> post) {
        // u is on the recursion stack
        color[u] = GRAY;
        for (int v : adj.get(u)) {
            // back edge: cycle
            if (color[v] == GRAY) return false;
            if (color[v] == WHITE && !dfs(v, adj, color, post))
              return false;
        }
        // everything reachable is finished
        color[u] = BLACK;
        post.add(u);
        return true;
    }
}
```

Problems

LC 210 Course Schedule II — MEDIUM

Task. Given `numCourses` and pairs `[a, b]` ("take b before a"), return any valid course order, or an empty array if impossible.

KEY INSIGHT

The required output is literally a topological order of the graph with edges b → a, and "impossible" means it contains a directed cycle.

JAVA SOLUTION LeetCode 210 · Solution.java

```java
import java.util.*;

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>
          ());
        int[] indeg = new int[numCourses];
        // [a, b]: b must precede a  =>  edge b -> a
        for (int[] p : prerequisites) {
            adj.get(p[1]).add(p[0]);
            indeg[p[0]]++;
        }
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int c = 0; c < numCourses; c++) if (indeg[c] == 0)
          q.add(c);
        int[] order = new int[numCourses];
        int k = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            order[k++] = u;
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.add(v);
        }
        return k == numCourses ? order : new int[0];
    }
}
```

Time O(V + E) **Space** O(V + E)

LC 207 Course Schedule — MEDIUM

Task. Return whether all courses can be finished given the same `[a, b]` prerequisite pairs.

KEY INSIGHT

Only a yes/no verdict is needed: "can finish" ⇔ no directed cycle. DFS colouring answers it without building an order.

JAVA SOLUTION LeetCode 207 · Solution.java

```java
import java.util.*;

class Solution {
    private List<List<Integer>> adj;
    // 0 = white, 1 = gray (on stack), 2 = black (done)
    private int[] color;

    public boolean canFinish(int numCourses, int[][] prerequisites)
      {
        adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>
          ());
        // b -> a
        for (int[] p : prerequisites) adj.get(p[1]).add(p[0]);
        color = new int[numCourses];
        for (int v = 0; v < numCourses; v++)
            if (color[v] == 0 && hasCycle(v)) return false;
        return true;
    }

    private boolean hasCycle(int u) {
        color[u] = 1;
        for (int v : adj.get(u)) {
            // back edge
            if (color[v] == 1) return true;
            if (color[v] == 0 && hasCycle(v)) return true;
        }
        color[u] = 2;
        return false;
    }
}
```

Time O(V + E) **Space** O(V + E), including O(V) recursion depth

LC 802 Find Eventual Safe States — MEDIUM

Task. In a directed graph (`graph[i]` lists i's successors), a node is *safe* if every path from it ends at a terminal node (out-degree 0). Return all safe nodes in ascending order.

KEY INSIGHT

A node is safe ⇔ it cannot reach a cycle. Terminal nodes are safe; a node becomes safe once *all* its successors are known safe. That is Kahn's algorithm on the **reversed** graph, driven by out-degrees.

JAVA SOLUTION LeetCode 802 · Solution.java

```java
import java.util.*;

class Solution {
    public List<Integer> eventualSafeNodes(int[][] graph) {
        int n = graph.length;
        List<List<Integer>> rev = new ArrayList<>();
        for (int i = 0; i < n; i++) rev.add(new ArrayList<>());
        int[] outdeg = new int[n];
        for (int u = 0; u < n; u++) {
            outdeg[u] = graph[u].length;
            // reversed edge v -> u
            for (int v : graph[u]) rev.get(v).add(u);
        }
        ArrayDeque<Integer> q = new ArrayDeque<>();
        // terminals are safe
        for (int u = 0; u < n; u++) if (outdeg[u] == 0) q.add(u);
        boolean[] safe = new boolean[n];
        while (!q.isEmpty()) {
            int v = q.poll();
            safe[v] = true;
            // all successors safe
```

```
            for (int u : rev.get(v)) if (--outdeg[u] == 0) q.add(u);
        }
        List<Integer> res = new ArrayList<>();
        for (int u = 0; u < n; u++) if (safe[u]) res.add(u);
        return res;
    }
}
```

Time O(V + E) **Space** O(V + E)

LC 269 Alien Dictionary

HARD Premium

Task. Given words sorted lexicographically in an unknown alphabet, return a string of all letters that appear, in any order consistent with the sorting. Return "" if no valid order exists.

KEY INSIGHT

Each adjacent pair of words gives at most one edge: at the first differing position, `w1[j] → w2[j]`. A word followed by its own proper prefix is a contradiction no alphabet can fix.

JAVA SOLUTION LeetCode 269 · Solution.java

```
import java.util.*;

class Solution {
    public String alienOrder(String[] words) {
        final int A = 26;
        boolean[] present = new boolean[A];
        // dedupes repeated constraints
        boolean[][] edge = new boolean[A][A];
        int[] indeg = new int[A];
        for (String w : words) for (char c : w.toCharArray())
          present[c - 'a'] = true;

        for (int i = 0; i + 1 < words.length; i++) {
            String w1 = words[i], w2 = words[i + 1];
            int len = Math.min(w1.length(), w2.length()), j = 0;
            while (j < len && w1.charAt(j) == w2.charAt(j)) j++;
            if (j == len) {
                // "abc" before "ab": invalid
                if (w1.length() > w2.length()) return "";
                // prefix in order: no info
                continue;
            }
            int u = w1.charAt(j) - 'a', v = w2.charAt(j) - 'a';
            // only the first mismatch counts
            if (!edge[u][v]) {
                edge[u][v] = true;
                indeg[v]++;
            }
        }

        ArrayDeque<Integer> q = new ArrayDeque<>();
        int letters = 0;
        for (int c = 0; c < A; c++) {
            if (!present[c]) continue;
            letters++;
            if (indeg[c] == 0) q.add(c);
        }
        StringBuilder sb = new StringBuilder();
        while (!q.isEmpty()) {
            int u = q.poll();
            sb.append((char) ('a' + u));
            for (int v = 0; v < A; v++)
                if (edge[u][v] && --indeg[v] == 0) q.add(v);
        }
        // cycle => inconsistent
        return sb.length() == letters ? sb.toString() : "";
    }
}
```

Time O(C + 26^2), where C is the total number of characters

Space O(26^2) = O(1) alphabet-bounded

PATTERN 12

Dynamic Programming: 1D

Key insight. A correct state answers "what does the rest of the problem need to know about the past?". If two different histories with the same `i` can lead to different futures, the state is missing a dimension.

Concepts

The four parts of a DP. Write each down before coding:

1. **State:** a precise sentence, e.g. "`dp[i]` = number of ways to split the suffix `s[i..n-1]`".
2. **Transition:** case analysis on the *last* (or first) decision. The cases must be disjoint for counting and exhaustive for both counting and optimisation.
3. **Base case:** the empty prefix or suffix. In counting DP the empty problem has exactly **1** way, not 0.
4. **Order:** compute every dependency before its use (increasing i for prefix states, decreasing i for suffix states).

```
Counting   :  dp[i] = Σ over disjoint last choices c   dp[prev(i, c)]
(sum rule)
Optimising :  dp[i] = max over last choices c  ( dp[prev(i, c)] +
gain(c) )
House Robber: dp[i] = max(dp[i-1], dp[i-2] + a[i-1])    over prefix
length i
               (skip house i-1)   (rob house i-1, so house i-2 must
be skipped)
```

State dependencies and space optimisation. If `dp[i]` reads only `dp[i-1]` and `dp[i-2]`, keep two variables: O(1) space. If it reads a window of size w (a piece of at most w digits), keep a circular buffer of size w + 1. The rolling update must read the old values before overwriting them.

Counting vs optimisation. Counting needs *disjoint* cases (no double counting) and a modulus. Optimisation tolerates overlapping cases (max is idempotent) but needs a sentinel such as `Integer.MIN_VALUE / 2` or −∞ for impossible states, so that adding to it does not overflow.

Top-down vs bottom-up. Memoised recursion is quicker to derive and visits only reachable states, but costs O(depth) stack. Bottom-up allows space compression and avoids stack overflow. Interviewers accept either, as long as you can state the complexity as (#states × transition cost).

Template

Template A — prefix-indexed optimisation with rolling variables.

JAVA TEMPLATE LinearDp.java

```
/** dp over prefix length i: dp[i] = best sum of the first i
  elements, no two adjacent. */
class LinearDp {
    static long bestNonAdjacent(int[] a) {
        long prev2 = 0;          // dp[i-2]  (empty prefix = 0)
        long prev1 = 0;          // dp[i-1]
        for (int x : a) {
            // skip a[i-1]  vs  take it
            long cur = Math.max(prev1, prev2 + x);
            // shift: read old values first
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;            // dp[n]
    }
}
```

Recognition rule. Exponential recursion + small repeated arguments → memoise; the arguments are the state.

Template B — counting partitions of a sequence (suffix DP, modulo). Reusable: the suffix loop and modular sum. Problem-specific: which pieces `s[i..j]` are valid.

JAVA TEMPLATE PartitionCountDp.java

```
/** dp[i] = ways to split s[i..n-1] into valid pieces; dp[n] = 1
  (empty suffix). */
class PartitionCountDp {
    static final int MOD = 1_000_000_007;

    // piece s[i..j] inclusive
    interface Piece { boolean valid(String s, int i, int j); }

    static int count(String s, int maxPieceLen, Piece rule) {
        int n = s.length();
        long[] dp = new long[n + 1];
        // one way to split nothing
        dp[n] = 1;
        // dependencies dp[j+1] lie to the right
        for (int i = n - 1; i >= 0; i--) {
            long ways = 0;
            for (int j = i; j < n && j - i < maxPieceLen; j++)
                if (rule.valid(s, i, j)) ways += dp[j + 1];
            dp[i] = ways % MOD;
        }
        return (int) dp[0];
    }
}
```

Problems

LC 70 Climbing Stairs

EASY

Task. Count the distinct ways to climb n steps taking 1 or 2 steps at a time (1 ≤ n ≤ 45).

KEY INSIGHT

Classify by the *last* step: it was 1 (from n−1) or 2 (from n−2). The cases are disjoint, so `ways(n) = ways(n-1) + ways(n-2)`, a Fibonacci recurrence.

JAVA SOLUTION LeetCode 70 · Solution.java

```java
class Solution {
    public int climbStairs(int n) {
        int prev2 = 1, prev1 = 1;          // ways(0), ways(1)
        for (int i = 2; i <= n; i++) {
            // last move was 1 step or 2 steps
            int cur = prev1 + prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        // ways(45) = 1836311903 < 2^31 - 1
        return prev1;
    }
}
```

Time O(n) **Space** O(1)

LC 198 House Robber — MEDIUM

Task. Maximise the money robbed from houses in a row without robbing two adjacent houses.

KEY INSIGHT

For the prefix of the first i houses, the last house is either skipped (best = dp[i−1]) or robbed (then house i−2 is excluded: dp[i−2] + a[i−1]). The rest of the history does not matter.

JAVA SOLUTION LeetCode 198 · Solution.java

```java
class Solution {
    public int rob(int[] nums) {
        // dp[i-2], dp[i-1]; dp[0] = 0
        int prev2 = 0, prev1 = 0;
        for (int x : nums) {
            // skip this house, or rob it
            int cur = Math.max(prev1, prev2 + x);
            prev2 = prev1;
            prev1 = cur;
        }
        // <= 100 houses * 400 fits easily
        return prev1;
    }
}
```

Time O(n) **Space** O(1)

LC 1416 Restore The Array — HARD

Task. A digit string `s` (length ≤ 10^5) was formed by concatenating integers in [1, k] (k ≤ 10^9) with no leading zeros. Count the possible original arrays, mod 10^9+7.

KEY INSIGHT

Let `dp[i]` = number of ways to split the suffix `s[i..]`. The first number starts at i, cannot start with '0', and has at most `digits(k) ≤ 10` digits. So each state has at most 10 transitions, and the scan stops as soon as the number exceeds k.

JAVA SOLUTION LeetCode 1416 · Solution.java

```java
class Solution {
    private static final int MOD = 1_000_000_007;

    public int numberOfArrays(String s, int k) {
        int n = s.length();
        // dp[i] = ways to split s[i..n-1]
        long[] dp = new long[n + 1];
        dp[n] = 1;
        for (int i = n - 1; i >= 0; i--) {
            // no number may start with '0' (dp[i] = 0)
            if (s.charAt(i) == '0') continue;
            long num = 0, ways = 0;
            for (int j = i; j < n; j++) {
                // long: up to 11 digits
                num = num * 10 + (s.charAt(j) - '0');
                // longer pieces only get bigger
                if (num > k) break;
                ways += dp[j + 1];
            }
            // at most 10 terms < MOD: no long overflow
            dp[i] = ways % MOD;
        }
        return (int) dp[0];
    }
}
```

Time O(n · $\log_{10}$ k) ≤ about 10n

Space O(n); reducible to O(log k) with a circular buffer of size 11

LC 494 Target Sum — MEDIUM

Task. Assign + or − to each of `nums` (n ≤ 20, 0 ≤ nums[i], Σ ≤ 1000) and count the assignments whose total equals `target`.

KEY INSIGHT

Let P be the elements given +, N the elements given −. Then `P - N = target` and `P + N = total`, so `P = (total + target) / 2`. The problem reduces to counting subsets with sum P.

```
sum(P) - sum(N) = target
sum(P) + sum(N) = total          ⇒   sum(P) = (total + target) / 2
Feasible only if |target| ≤ total and (total + target) is even.
```

JAVA SOLUTION LeetCode 494 · Solution.java

```java
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int total = 0;
        for (int x : nums) total += x;
        if (Math.abs(target) > total || ((total + target) & 1) == 1)
          return 0;
        // required sum of the '+' subset, >= 0
        int p = (total + target) / 2;
        // dp[s]: subsets of items so far with sum s
        int[] dp = new int[p + 1];
        dp[0] = 1;
        for (int x : nums)
            // decreasing: each item used at most once
            for (int s = p; s >= x; s--)
                // x == 0 doubles: +0 and -0 both count
                dp[s] += dp[s - x];
        // <= 2^20 assignments: fits in int
        return dp[p];
    }
}
```

Time O(n · (total + target)/2) **Space** O(total)

PATTERN 13

Dynamic Programming: 2D

Key insight. Use prefix *lengths* (an (m+1) × (n+1) table), not indices. Row 0 and column 0 then represent empty prefixes, the base cases become explicit, and the boundary `if`s disappear.

Recognition rule. Two sequences compared position by position, or a grid with monotone moves → a (prefix × prefix) or (row × column) table.

Concepts

Grid DP. With moves right/down, every path to (i, j) enters from (i−1, j) or (i, j−1):

```
Count:    paths[i][j]  = paths[i-1][j] + paths[i][j-1]          (0
for blocked cells)
Minimise: cost[i][j]   = grid[i][j] + min(cost[i-1][j], cost[i][j-1])
Borders:  first row / column have a single predecessor
```

Two-sequence DP. On prefixes of lengths i and j:

```
LCS:   L[i][j] = L[i-1][j-1] + 1                    if a[i-1] ==
b[j-1]
               = max(L[i-1][j], L[i][j-1])           otherwise
       L[0][j] = L[i][0] = 0

Edit:  D[i][j] = D[i-1][j-1]                        if a[i-1] ==
b[j-1]
               = 1 + min( D[i-1][j],    // delete a[i-1]
                          D[i][j-1],    // insert b[j-1]
                          D[i-1][j-1] ) // replace a[i-1] by b[j-1]
       D[i][0] = i,  D[0][j] = j       // convert to / from the
empty string
```

Traversal order. Each cell must be computed after its dependencies. For dependencies on up, left and up-left, row-major order (i ascending, j ascending) works. For dependencies on down/right (such as Dungeon Game), fill from the bottom-right.

Space compression. If row i depends only on rows i and i−1, keep one row:

- Up and left only (grid DP): `dp[j] = f(dp[j] /*up, old*/, dp[j-1] /*left, new*/)`, with no extra variable.
- Also up-left (LCS, edit distance): the old `dp[j-1]` is overwritten before cell (i, j) is computed, so save it in a variable `diag` before each overwrite.
- Compress along the *shorter* dimension to get O(min(m, n)) space. You lose the ability to reconstruct the alignment unless you use Hirschberg's divide-and-conquer.

Template

Template A — grid DP with a rolling row (monotone moves right/down).

JAVA TEMPLATE GridDp.java

```java
/** Minimum-cost path from (0,0) to (m-1,n-1) moving right/down.
  O(n) space via one row. */
class GridDp {
    static long minPathCost(int[][] g) {
        int m = g.length, n = g[0].length;
        // dp[j] = best cost to reach (i, j)
        long[] dp = new long[n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (i == 0 && j == 0) dp[j] = g[0][0];
                // only from the left
                else if (i == 0) dp[j] = dp[j - 1] + g[i][j];
                // only from above
                else if (j == 0) dp[j] = dp[j] + g[i][j];
                // dp[j] = up (old row)
                else dp[j] = Math.min(dp[j], dp[j - 1]) + g[i][j];
            }
        }
        return dp[n - 1];
    }
}
```

Template B — two-sequence DP on prefix lengths (full table). Reusable: the table shape, the borders and the fill order. Problem-specific: the match and mismatch transitions.

JAVA TEMPLATE TwoSequenceDp.java

```java
/** dp[i][j] over prefixes a[0..i), b[0..j). Shown for LCS; swap
  transitions as needed. */
class TwoSequenceDp {
    static int lcs(String a, String b) {
        int m = a.length(), n = b.length();
        // row 0 / col 0 = empty prefix = 0
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] =
                  dp[i - 1][j - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j -
                  1]);
            }
        }
        return dp[m][n];
    }
}
```

Problems

LC 62 Unique Paths — MEDIUM

Task. Count the paths from the top-left to the bottom-right of an m × n grid moving only right or down.

KEY INSIGHT

Paths into a cell arrive from above or from the left, and the two sets are disjoint, so the counts add. Equivalently, a path is a sequence of (m−1) downs and (n−1) rights, giving C(m+n−2, m−1).

JAVA SOLUTION LeetCode 62 · Solution.java

```java
import java.util.*;

class Solution {
    public int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        // first row: exactly one path to each cell
        Arrays.fill(dp, 1);
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                // from above (old dp[j]) + from left (new)
                dp[j] += dp[j - 1];
        // problem guarantees answer <= 2 * 10^9
        return dp[n - 1];
    }
}
```

Time O(m · n) **Space** O(n)

LC 64 Minimum Path Sum — MEDIUM

Task. Find the minimum sum of a right/down path from the top-left to the bottom-right of a non-negative grid.

KEY INSIGHT

The best path to (i, j) extends the cheaper of the best paths to its two possible predecessors (optimal substructure, since costs add).

JAVA SOLUTION LeetCode 64 · Solution.java

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[] dp = new int[n];
        dp[0] = grid[0][0];
        // first row
        for (int j = 1; j < n; j++) dp[j] = dp[j - 1] + grid[0][j];
        for (int i = 1; i < m; i++) {
            // first column
            dp[0] += grid[i][0];
            for (int j = 1; j < n; j++)
                // up vs left
                dp[j] = Math.min(dp[j], dp[j - 1]) + grid[i][j];
        }
        // <= (200 + 200) * 200: fits in int
        return dp[n - 1];
    }
}
```

Time O(m · n) **Space** O(n), or O(1) by writing into `grid` (mutating the input)

LC 1143 Longest Common Subsequence — MEDIUM

Task. Return the length of the longest common subsequence of two strings (lengths ≤ 1000).

KEY INSIGHT

Look at the last characters of the two prefixes. If they match, they can be paired (the exchange argument above). Otherwise at least one of them is unused, so drop one and take the better result.

JAVA SOLUTION LeetCode 1143 · Solution.java

```java
class Solution {
    public int longestCommonSubsequence(String text1, String text2)
      {
        int m = text1.length(), n = text2.length();
        // LCS of text1[0..i) and text2[0..j)
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            char c = text1.charAt(i - 1);
            for (int j = 1; j <= n; j++) {
                if (c == text2.charAt(j - 1)) dp[i][j] = dp[i - 1][j
                  - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j -
                  1]);
            }
        }
        return dp[m][n];
    }
}
```

Time O(m · n)

Space O(m · n); O(min(m, n)) with a rolling row and a diagonal variable

LC 72 Edit Distance — MEDIUM

Task. Return the minimum number of single-character insertions, deletions and replacements to turn `word1` into `word2` (lengths ≤ 500).

KEY INSIGHT

In an optimal edit script, the last characters are either matched for free, or the last operation is a delete of `a[i-1]`, an insert of `b[j-1]`, or a replace. Each option leaves a smaller prefix pair. Borders are the cost of building from or reducing to the empty string.

JAVA SOLUTION LeetCode 72 · Solution.java

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        // delete all i characters
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        // insert all j characters
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    // match: free
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    // replace
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                    // delete word1[i-1]
                                    Math.min(dp[i - 1][j],
                                             // insert word2[j-1]
                                             dp[i][j - 1]));
                }
            }
        }
        return dp[m][n];
```

```
    }
}
```

Time O(m · n) **Space** O(m · n); O(min(m, n)) with a rolling row

PATTERN 14

Greedy: Interval Scheduling / Local Optimality

Key insight. A greedy algorithm is only as good as its proof. State the choice ("take the compatible interval with the smallest end") and the exchange ("replacing OPT's first interval with mine keeps OPT feasible and the same size"). If the exchange fails on some example, switch to DP.

Concepts

Exchange argument for earliest end. Sort intervals by end. Let g_1 be the first interval with the smallest end and let OPT = $o_1, o_2, \ldots$ be an optimal compatible set sorted by end.

```
end(g₁) ≤ end(o₁)                          (g₁ has the globally smallest end)
o₂ starts at or after end(o₁) ≥ end(g₁)   ⇒  {g₁, o₂, o₃, …} is still compatible
|{g₁, o₂, …}| = |OPT|                     ⇒  some optimal solution contains g₁
Induct on the intervals compatible with g₁  ⇒  greedy is optimal.
```

The same argument, phrased as "greedy stays ahead", says that after k picks the greedy's k-th end is ≤ OPT's k-th end, so greedy can never fall behind.

Why sort by end and not by start. Sorting by start and taking the earliest-starting interval fails on `[1,100], [2,3], [4,5]`: you take one long interval instead of two short ones. Sorting by length fails on `[1,5], [4,7], [6,10]`: the shortest, `[4,7]`, blocks both others. The end time is the only key whose exchange argument works.

Boundary convention. LeetCode 435 and 252 treat touching intervals (`[1,2]` and `[2,3]`) as compatible, which means half-open semantics. LeetCode 56 and 986 use closed intervals, where touching intervals overlap or intersect at a point. Write the comparison (`>=` vs `>`) as a deliberate decision.

When the greedy view is not the full story. Merging (56) and intersecting lists (986) are often filed under greedy because each step commits to something permanently. Their correctness comes from sortedness plus a sweep or two-pointer invariant, not from an exchange argument. The assigned problems below label this explicitly.

Template

Template A — maximum compatible subset (earliest end). Reusable: sort by end, keep `lastEnd`. Problem-specific: the compatibility test (`>=` for half-open, `>` for closed).

JAVA TEMPLATE IntervalScheduling.java

```java
import java.util.*;

/** Unweighted interval scheduling on [start, end): touching
  intervals are compatible. */
class IntervalScheduling {
    static int maxCompatible(int[][] intervals) {
        if (intervals.length == 0) return 0;
        // keep the caller's order
        int[][] a = intervals.clone();
        // by end; never x[1] - y[1]
        Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1]));
        int count = 1;
        int lastEnd = a[0][1];
        for (int i = 1; i < a.length; i++) {
            // closed intervals: use '>'
            if (a[i][0] >= lastEnd) {
                count++;
                // greedy choice: earliest end
                lastEnd = a[i][1];
            }
        }
        return count;
    }
}
```

Template B — neighbour conflict check after sorting by start.

JAVA TEMPLATE NeighbourConflict.java

```java
import java.util.*;

/** Sorted by start, any overlap implies an overlap between two
  adjacent intervals. */
class NeighbourConflict {
    static boolean hasOverlap(int[][] intervals, boolean
      touchingOverlaps) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
        for (int i = 1; i < a.length; i++) {
            boolean overlap = touchingOverlaps ? a[i][0] <= a[i - 1]
              [1]
                                               : a[i][0] < a[i - 1]
                                                 [1];
            if (overlap) return true;
        }
        return false;
    }
}
```

Recognition rule. Unweighted "keep as many as possible" over intervals → sort by end, take each one that fits.

Problems

LC 252 Meeting Rooms — EASY — Premium

Task. Given meeting intervals `[start, end]`, decide whether one person can attend all of them. A meeting ending at time t and another starting at t do not conflict.

KEY INSIGHT

After sorting by start, if any two meetings overlap then some *adjacent* pair overlaps. Checking neighbours is enough.

JAVA SOLUTION LeetCode 252 · Solution.java

```java
import java.util.*;

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (x, y) -> Integer.compare(x[0],
          y[0]));
        for (int i = 1; i < intervals.length; i++) {
            // touching is allowed
            if (intervals[i][0] < intervals[i - 1][1]) return false;
        }
        return true;
    }
}
```

Time O(n log n) **Space** O(n) worst case for the object-array sort

LC 435 Non-overlapping Intervals — MEDIUM

Task. Return the minimum number of intervals to remove so the rest do not overlap. Touching intervals (`[1,2]`, `[2,3]`) do not overlap.

KEY INSIGHT

Minimum removals = n − maximum number of compatible intervals, and the maximum is achieved by repeatedly keeping the compatible interval with the earliest end.

JAVA SOLUTION LeetCode 435 · Solution.java

```java
import java.util.*;

class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (x, y) -> Integer.compare(x[1],
          y[1]));
        int kept = 1;
        // n >= 1 by constraints
        int lastEnd = intervals[0][1];
        for (int i = 1; i < intervals.length; i++) {
            // compatible (touching allowed)
            if (intervals[i][0] >= lastEnd) {
                kept++;
                lastEnd = intervals[i][1];
            // else: removed; lastEnd unchanged
            }
        }
        return intervals.length - kept;
    }
}
```

Time O(n log n) **Space** O(n) worst case (sort)

LC 56 Merge Intervals — MEDIUM

Task. Merge all overlapping closed intervals (touching ones such as `[1,4]` and `[4,5]` merge) and return the disjoint result.

KEY INSIGHT

Sort starts and ends *independently*. In sorted order, a merged group ends exactly at position i when `starts[i+1] > ends[i]`: the i+1 smallest ends all come before the next start.

JAVA SOLUTION LeetCode 56 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        int n = intervals.length;
        int[] starts = new int[n], ends = new int[n];
        for (int i = 0; i < n; i++) {
            starts[i] = intervals[i][0];
            ends[i] = intervals[i][1];
        }
        // primitive sort: no comparator
        Arrays.sort(starts);
        Arrays.sort(ends);
        List<int[]> res = new ArrayList<>();
        int groupStart = starts[0];
        for (int i = 0; i < n; i++) {
            // Closed intervals: touching (starts[i+1] == ends[i])
              still merges, hence '>'.
            if (i == n - 1 || starts[i + 1] > ends[i]) {
                res.add(new int[]{groupStart, ends[i]});
                if (i + 1 < n) groupStart = starts[i + 1];
            }
        }
        return res.toArray(new int[0][]);
    }
}
```

Time O(n log n) **Space** O(n) for the two endpoint arrays

LC 986 Interval List Intersections — MEDIUM

Task. Two lists of closed intervals, each pairwise disjoint and sorted, are given. Return their intersection.

KEY INSIGHT

Two closed intervals intersect in `[max(starts), min(ends)]` when that range is non-empty. Whichever interval ends first is exhausted: every later interval in the other list starts after the current one, and the current one already started no later than this end.

JAVA SOLUTION LeetCode 986 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] intervalIntersection(int[][] firstList, int[][]
      secondList) {
        List<int[]> res = new ArrayList<>();
        int i = 0, j = 0;
        while (i < firstList.length && j < secondList.length) {
            int lo = Math.max(firstList[i][0], secondList[j][0]);
            int hi = Math.min(firstList[i][1], secondList[j][1]);
            // closed: [5,5] is a valid answer
            if (lo <= hi) res.add(new int[]{lo, hi});
            // earlier end: exhausted
            if (firstList[i][1] < secondList[j][1]) i++;
            else j++;
        }
        return res.toArray(new int[0][]);
    }
}
```

Time O(m + n) **Space** O(1) auxiliary (output excluded)

PATTERN 15

Heap / Priority Queue: Top-K and Streaming

Key insight. The heap type is the opposite of the goal for bounded top-k. The root must be the element you are ready to throw away. For k largest that is the smallest element kept, so use a min-heap.

Recognition rule. Repeatedly need the extreme of a changing set → heap. Need only the top k → bound the heap at k with the opposite order.

Concepts

Heap invariant. Every parent compares ≤ its children under the comparator, so the root is the extreme. Java's `PriorityQueue` is a binary min-heap by natural order. Pass a comparator for anything else.

Operation	`PriorityQueue` cost	Note
`offer` / `poll`	O(log n)	Sift up / sift down
`peek`, `size`	O(1)	
`remove(Object)`, `contains`	O(n)	Linear scan; avoid in hot loops
Build from a collection	O(n)	`new PriorityQueue<>(collection)` heapifies bottom-up
Iteration order	Not sorted	Only the root is guaranteed; poll repeatedly for sorted output

Bounded top-k invariant. After processing a prefix, the size-k min-heap holds the k largest elements of that prefix. A new element x enters only if `x > heap.peek()`, and then evicts the root. Cost: O(n log k) time, O(k) space.

Two-heaps median invariant.

```
lo = max-heap (smaller half),  hi = min-heap (larger half)
(1) every x in lo ≤ every y in hi      ⇔  lo.peek() ≤ hi.peek()
(2) |lo| = |hi|  or  |lo| = |hi| + 1
median = lo.peek()                        if sizes differ
       = (lo.peek() + hi.peek()) / 2.0    otherwise
```

Insert into `lo`, move `lo`'s maximum to `hi` (restores (1)), and if `hi` becomes larger move its minimum back (restores (2)). Each add is three heap operations, O(log n).

JAVA SNIPPET Comparator pitfalls

```java
// overflows
PriorityQueue<Integer> bad  = new PriorityQueue<>((a, b) -> b - a);
// max-heap
PriorityQueue<Integer> good = new PriorityQueue<>
  (Collections.reverseOrder());
PriorityQueue<int[]> byFreqThenValue = new PriorityQueue<>(
    (x, y) -> x[1] != y[1] ? Integer.compare(x[1], y[1]) :
      Integer.compare(x[0], y[0]));
```

Duplicates. Heaps are multisets: equal values coexist and are popped in an unspecified order among themselves. Count-based problems (top-k frequent) should push one entry per *distinct key*, not per occurrence.

Template

Template A — bounded top-k (k largest by a key). Reusable: the size-k min-heap and eviction rule. Problem-specific: the key and the comparator.

JAVA TEMPLATE TopK.java

```java
import java.util.*;

/** Keeps the k largest values seen; root = k-th largest. O(n log k)
  time, O(k) space. */
class TopK {
    static int[] kLargest(int[] nums, int k) {
        // min-heap: root is weakest
        PriorityQueue<Integer> heap = new PriorityQueue<>();
        for (int x : nums) {
            if (heap.size() < k) heap.offer(x);
            // x beats the weakest kept
            else if (x > heap.peek()) {
                heap.poll();
                heap.offer(x);
            }
        }
        int[] res = new int[heap.size()];
        // descending order
        for (int i = res.length - 1; i >= 0; i--) res[i] =
          heap.poll();
        return res;
    }
}
```

Template B — two heaps for a running median.

JAVA TEMPLATE RunningMedian.java

```java
import java.util.*;

/** lo: max-heap (smaller half); hi: min-heap (larger half); |lo| -
  |hi| in {0, 1}. */
class RunningMedian {
    private final PriorityQueue<Integer> lo =
        new PriorityQueue<>(Collections.reverseOrder());
    private final PriorityQueue<Integer> hi = new PriorityQueue<>();

    void add(int x) {
        lo.offer(x);
        // max of lower half moves up
        hi.offer(lo.poll());
        // rebalance sizes
```

```
        if (hi.size() > lo.size()) lo.offer(hi.poll());
    }

    // precondition: at least one element
    double median() {
        if (lo.size() > hi.size()) return lo.peek();
        // long: the int sum may overflow
        return ((long) lo.peek() + hi.peek()) / 2.0;
    }
}
```

Problems

LC 1046 Last Stone Weight — EASY

Task. Repeatedly smash the two heaviest stones. Equal stones are both destroyed; otherwise the difference remains. Return the last stone's weight, or 0 if none is left.

KEY INSIGHT

Each round needs the two current maxima and may insert a new stone. That is a dynamic-maximum simulation, which a max-heap handles directly.

JAVA SOLUTION LeetCode 1046 · Solution.java

```java
import java.util.*;

class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>
          (Collections.reverseOrder());
        for (int s : stones) pq.offer(s);
        while (pq.size() > 1) {
            int y = pq.poll(), x = pq.poll();  // y >= x
            if (y != x) pq.offer(y - x);
        }
        return pq.isEmpty() ? 0 : pq.peek();
    }
}
```

Time O(n log n) **Space** O(n)

LC 215 Kth Largest Element in an Array — MEDIUM

Task. Return the k-th largest element in sorted order (duplicates count separately), ideally without fully sorting.

KEY INSIGHT

A size-k min-heap's root is the k-th largest of everything seen. Alternatively, quickselect partitions around a pivot and recurses into one side only.

JAVA SOLUTION LeetCode 215 · Solution.java (heap)

```java
import java.util.*;

class Solution {
    public int findKthLargest(int[] nums, int k) {
        // min-heap of the k largest
        PriorityQueue<Integer> heap = new PriorityQueue<>();
        for (int x : nums) {
            heap.offer(x);
            // evict the weakest
            if (heap.size() > k) heap.poll();
        }
        // k-th largest overall
        return heap.peek();
    }
}
```

Time heap O(n log k); quickselect O(n) expected
Space heap O(k); quickselect O(1)

LC 347 Top K Frequent Elements — MEDIUM

Task. Return the k most frequent values in any order. The answer is guaranteed unique.

KEY INSIGHT

Count first, then run bounded top-k over the *distinct keys* using frequency as the priority. The heap stores keys, not occurrences.

JAVA SOLUTION LeetCode 347 · Solution.java

```java
import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        // Min-heap by frequency: the root is the least frequent of
          the current top-k.
        PriorityQueue<Integer> heap =
            new PriorityQueue<>((a, b) ->
              Integer.compare(freq.get(a), freq.get(b)));
        for (int key : freq.keySet()) {
            heap.offer(key);
            if (heap.size() > k) heap.poll();
        }
        int[] res = new int[k];
        // most frequent first
        for (int i = k - 1; i >= 0; i--) res[i] = heap.poll();
        return res;
    }
}
```

Time O(n + u log k), u = distinct values **Space** O(u)

LC 295 Find Median from Data Stream — HARD

Task. Design `MedianFinder` with `addNum(int)` and `findMedian()` returning the median of all numbers so far.

KEY INSIGHT

The median is determined by the boundary between the lower and upper halves. Keep the lower half in a max-heap and the upper half in a min-heap; the median is at one or both roots.

JAVA SOLUTION LeetCode 295 · MedianFinder.java

```java
import java.util.*;

class MedianFinder {
    // max-heap: smaller half
    private final PriorityQueue<Integer> lo;
    // min-heap: larger half
    private final PriorityQueue<Integer> hi;

    public MedianFinder() {
        lo = new PriorityQueue<>(Collections.reverseOrder());
        hi = new PriorityQueue<>();
    }

    public void addNum(int num) {
        lo.offer(num);
        // keeps max(lo) <= min(hi)
        hi.offer(lo.poll());
        // keeps |lo| in {|hi|, |hi| + 1}
        if (hi.size() > lo.size()) lo.offer(hi.poll());
    }

    public double findMedian() {
        if (lo.size() > hi.size()) return lo.peek();
        return ((long) lo.peek() + hi.peek()) / 2.0;
    }
}
```

Time `addNum` O(log n), `findMedian` O(1) **Space** O(n)

PATTERN 16

Bit Manipulation

Key insight. Ask what happens *in a single bit position*. If the answer's bit i depends only on the inputs' bit i (counts, parity, common prefix), solve the one-bit problem and apply it to all 32 lanes.

Recognition rule. Fixed multiplicities + O(1) space → XOR or per-bit counting. Range bitwise op → think about the common prefix.

Concepts

Identity	Meaning / use
`x ^ x = 0`, `x ^ 0 = x`, XOR is associative and commutative	The order of folding does not matter; pairs cancel.
`x & (x - 1)`	Clears the lowest set bit. Used in Kernighan popcount (loop runs once per set bit) and the power-of-two test `x > 0 && (x & (x-1)) == 0`.
`x & -x`	Isolates the lowest set bit, since `-x = ~x + 1`. Drives Fenwick trees (Pattern 20) and Single Number III.
`(x >> i) & 1`, `x | (1 << i)`, `x & ~(1 << i)`, `x ^ (1 << i)`	Get, set, clear and toggle bit i.
`for (s = m; s > 0; s = (s - 1) & m)`	Enumerates all non-empty submasks of m in decreasing order. Over all m this totals $O(3^n)$.

Java's fixed-width integers. `int` is 32-bit two's complement, `long` is 64-bit; there are no unsigned types.

- `>>` is an *arithmetic* shift (copies the sign bit): `-8 >> 1 == -4`. `>>>` is a *logical* shift (fills with 0): `-8 >>> 1 == 2147483644`. A loop `while (x != 0) x >>= 1` never terminates for negative x; use `>>>`.
- Shift distances are taken modulo the width: `1 << 32 == 1` for an int, and `1 << 31 == Integer.MIN_VALUE`. Use `1L << i` for 64-bit masks.
- `Integer.MIN_VALUE` is its own negation: `-MIN_VALUE == MIN_VALUE`. Therefore `MIN_VALUE & -MIN_VALUE == MIN_VALUE`, which is still the correct lowest set bit (bit 31).
- Built-ins: `Integer.bitCount`, `Integer.numberOfTrailingZeros`, `Integer.highestOneBit`, `Integer.lowestOneBit`, `Integer.toBinaryString` (shows negatives as 32 bits).

Template

Template A — bit toolkit. These are reusable as is.

JAVA TEMPLATE BitTricks.java

```java
import java.util.*;

/** Core bit identities for 32-bit ints (two's complement). */
class BitTricks {
    static boolean getBit(int x, int i)  { return ((x >>> i) & 1) == 1; }
    static int setBit(int x, int i)      { return x | (1 << i); }
    static int clearBit(int x, int i)    { return x & ~(1 << i); }
    static int toggleBit(int x, int i)   { return x ^ (1 << i); }
    // 0 if x == 0
    static int lowestSetBit(int x)       { return x & -x; }
    static int clearLowestSetBit(int x)  { return x & (x - 1); }
    static boolean isPowerOfTwo(int x)   { return x > 0 && (x & (x - 1)) == 0; }

    /** Kernighan: one iteration per set bit; correct for negatives (32 iterations max). */
    static int popCount(int x) {
        int c = 0;
        while (x != 0) { x &= x - 1; c++; }
        return c;
    }

    /** All non-empty submasks of mask, largest first. */
    static List<Integer> submasks(int mask) {
        List<Integer> out = new ArrayList<>();
        for (int s = mask; s > 0; s = (s - 1) & mask) out.add(s);
        return out;
    }
}
```

Template B — per-bit counting modulo k. Problem-specific: k, and whether a different multiplicity (other than 1) must be extracted.

JAVA TEMPLATE PerBitCount.java

```java
/** Every value appears k times except one that appears once:
  rebuild it bit by bit. */
class PerBitCount {
    static int singleAmongK(int[] nums, int k) {
        int result = 0;
        for (int bit = 0; bit < 32; bit++) {
            int count = 0;
            // >>> : sign bit is not smeared
            for (int x : nums) count += (x >>> bit) & 1;
            // bit 31 set => negative result
            if (count % k != 0) result |= 1 << bit;
        }
        return result;
    }
}
```

Problems

LC 136 Single Number — EASY

Task. Every element appears twice except one. Find it in O(n) time and O(1) extra space.

KEY INSIGHT

XOR of all elements: each pair contributes `x ^ x = 0`, and order does not matter, so only the single value survives.

JAVA SOLUTION LeetCode 136 · Solution.java

```java
class Solution {
    public int singleNumber(int[] nums) {
        int x = 0;
        // pairs cancel; order irrelevant
        for (int v : nums) x ^= v;
        return x;
    }
}
```

Time O(n) **Space** O(1)

LC 137 Single Number II — MEDIUM

Task. Every element appears three times except one, which appears once. Find it in linear time and constant space.

KEY INSIGHT

In each bit lane, the count of ones is 3c + b, where b is the answer's bit. Keep a counter modulo 3 per lane. Two ints `ones` and `twos` store the counter's two bits for all 32 lanes simultaneously.

JAVA SOLUTION LeetCode 137 · Solution.java

```java
class Solution {
    public int singleNumber(int[] nums) {
        // per-bit counter mod 3, stored as two bit-planes
        int ones = 0, twos = 0;
        for (int x : nums) {
            ones = (ones ^ x) & ~twos;
            twos = (twos ^ x) & ~ones;
        }
        return ones;              // lanes with count % 3 == 1
    }
}
```

Time O(n) **Space** O(1)

LC 260 Single Number III — MEDIUM

Task. Exactly two elements appear once and all others appear twice. Return the two singles, in any order, in O(n) time and O(1) space.

KEY INSIGHT

The XOR of everything is `a ^ b ≠ 0`. Any set bit of it, for example the lowest via `d & -d`, is a position where a and b differ. Splitting the array by that bit puts a and b in different groups, while each duplicate pair stays together. Each group is then a Single Number I.

JAVA SOLUTION LeetCode 260 · Solution.java

```java
class Solution {
    public int[] singleNumber(int[] nums) {
        int diff = 0;
        // = a ^ b, non-zero because a != b
        for (int v : nums) diff ^= v;
        // lowest bit where a and b differ
        int mask = diff & -diff;
        // diff == MIN_VALUE still works
        int a = 0, b = 0;
        for (int v : nums) {
            // parenthesised: != binds tighter than &
            if ((v & mask) != 0) a ^= v;
            else b ^= v;
        }
        return new int[]{a, b};
    }
}
```

Time O(n) **Space** O(1)

LC 201 Bitwise AND of Numbers Range — MEDIUM

Task. Given `0 ≤ left ≤ right ≤ 2^31 − 1`, return the AND of every integer in `[left, right]`.

KEY INSIGHT

The result is the common binary prefix of `left` and `right` followed by zeros. Below the highest bit where they differ, the range contains a number ending in `0111…` followed by one ending in `1000…`, so every lower bit is 0 somewhere in the range.

JAVA SOLUTION LeetCode 201 · Solution.java

```java
class Solution {
    public int rangeBitwiseAnd(int left, int right) {
        int shift = 0;
        // differing bits remain below the common prefix
        while (left < right) {
            // inputs are non-negative, so >> and >>> agree
            left >>= 1;
            right >>= 1;
            shift++;
        }
        // common prefix followed by zeros
        return left << shift;
    }
}
```

Time O(log right) ≤ 31 iterations **Space** O(1)

PATTERN 17

Union-Find / Disjoint Set Union

Key insight. DSU does not store *how* elements are connected, only *whether* they are. If you need paths, distances or edge directions, traverse the graph instead. If you only need connectivity under insertions, DSU is the simplest correct tool.

Concepts

Representation. `parent[x]` is x's parent and roots satisfy `parent[r] == r`. Invariant: following parents from any element reaches its component's unique root.

Two heuristics, used together.

- **Union by size (or rank)**: attach the smaller tree under the larger root. A node's depth increases only when its tree at least doubles in size, so depth ≤ $\log_2 n$ even without compression.
- **Path compression**: during `find`, point visited nodes (all of them, or every other one with path halving) closer to the root, flattening future queries.

```
With both heuristics: m operations on n elements cost O(m · α(n))
(Tarjan)
α = inverse Ackermann; α(n) ≤ 4 for any n that fits in memory  →
"effectively constant"
Union by size alone: O(log n) per find.   Compression alone: O(log n)
amortised.
```

Cycle detection. Process undirected edges in order. If `find(u) == find(v)` before the union, a u–v path already exists, so adding (u, v) closes a cycle. This is also Kruskal's rejection rule.

Component counting. `components = n − (number of successful unions)`, because each successful union merges exactly two components.

Template

Template A — array DSU over 0..n−1. Reusable as is. Problem-specific: what counts as an edge, and what you do when `union` returns false.

JAVA TEMPLATE DSU.java

```java
/** Disjoint Set Union with path halving + union by size. Elements
  are 0..n-1. */
class DSU {
    private final int[] parent, size;
    private int components;

    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    int find(int x) {
        while (parent[x] != x) {
            // path halving: skip a level each step
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    /** Returns false if a and b were already connected (=> the edge
      a–b closes a cycle). */
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        // smaller under larger
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        components--;
        return true;
    }

    boolean connected(int a, int b) { return find(a) == find(b); }
    int componentSize(int x)        { return size[find(x)]; }
    int components()                { return components; }
}
```

Template B — DSU over arbitrary keys. For strings or sparse integers (emails, coordinates), map keys to ids lazily.

JAVA TEMPLATE KeyedDSU.java

```java
import java.util.*;

/** DSU over hashable keys; ids are assigned on first sight. */
class KeyedDSU<K> {
    private final Map<K, Integer> id = new HashMap<>();
    private final List<Integer> parent = new ArrayList<>();
    private final List<Integer> size = new ArrayList<>();

    int idOf(K key) {
        Integer i = id.get(key);
        if (i != null) return i;
        int n = parent.size();
        id.put(key, n);
        parent.add(n);
        size.add(1);
        return n;
    }

    int find(int x) {
        while (parent.get(x) != x) {
            parent.set(x, parent.get(parent.get(x)));
            x = parent.get(x);
        }
        return x;
    }

    boolean union(K a, K b) {
        int ra = find(idOf(a)), rb = find(idOf(b));
        if (ra == rb) return false;
        if (size.get(ra) < size.get(rb)) { int t = ra; ra = rb; rb =
          t; }
        parent.set(rb, ra);
        size.set(ra, size.get(ra) + size.get(rb));
        return true;
    }

    boolean connected(K a, K b) { return find(idOf(a)) ==
      find(idOf(b)); }
}
```

Note: `parent.get(x) != x` compares an `Integer` with an `int`, which unboxes, so the comparison is numeric and correct. Comparing two `Integer` objects with `!=` would compare references and break above 127.

Recognition rule. Undirected relations merging over time + "same group?" or "how many groups?" → DSU.

Problems

LC 1971 Find if Path Exists in Graph — EASY

Task. Given an undirected graph on vertices 0..n−1 as an edge list, decide whether `source` and `destination` are connected.

KEY INSIGHT

A path exists iff both endpoints end up in the same component after all edges are unioned. No adjacency list is needed.

JAVA SOLUTION LeetCode 1971 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public boolean validPath(int n, int[][] edges, int source, int
      destination) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        // undirected, unweighted edges
        for (int[] e : edges) union(e[0], e[1]);
        // source == destination: trivially true
        return find(source) == find(destination);
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
    }
}
```

Time O(n + E·α(n)) **Space** O(n)

LC 684 Redundant Connection — MEDIUM

Task. A tree on n nodes (labelled 1..n) had one extra edge added. Return an edge whose removal leaves a tree; if several qualify, return the one that occurs last in the input.

KEY INSIGHT

Scan edges in order. The first edge whose endpoints are already connected closes the (unique) cycle. Every other cycle edge appears earlier, so this edge is the last cycle edge in the input, which is exactly the required answer.

JAVA SOLUTION LeetCode 684 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int[] findRedundantConnection(int[][] edges) {
        // tree + 1 edge => edges == nodes
        int n = edges.length;
        // labels are 1..n
        parent = new int[n + 1];
        size = new int[n + 1];
        for (int i = 0; i <= n; i++) { parent[i] = i; size[i] = 1; }
        for (int[] e : edges) {
            // already connected: closes the cycle
            if (!union(e[0], e[1])) return e;
        }
        // unreachable for valid input
        return new int[0];
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(n·α(n)) **Space** O(n)

LC 547 Number of Provinces — MEDIUM

Task. Given an n × n symmetric adjacency matrix `isConnected`, count the connected components.

KEY INSIGHT

Start with n components and decrement on each successful union over the upper triangle of the matrix.

JAVA SOLUTION LeetCode 547 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        int components = n;
        for (int i = 0; i < n; i++) {
            // symmetric: upper triangle only
            for (int j = i + 1; j < n; j++) {
                if (isConnected[i][j] == 1 && union(i, j))
                    components--;
            }
        }
        return components;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(n²·α(n)) **Space** O(n)

LC 721 Accounts Merge — MEDIUM

Task. Each account is `[name, email1, email2, …]`. Two accounts belong to the same person if they share any email. Merge them and return each person's name followed by their emails in sorted order; accounts may be returned in any order.

KEY INSIGHT

Sharing an email is a transitive equivalence between *accounts*. Map each email to the first account index that contained it, and union every later account containing that email with that index.

JAVA SOLUTION LeetCode 721 · Solution.java

```java
import java.util.*;

class Solution {
    private int[] parent, size;

    public List<List<String>> accountsMerge(List<List<String>>
      accounts) {
        int n = accounts.size();
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }

        // email -> first account index
        Map<String, Integer> owner = new HashMap<>();
        for (int i = 0; i < n; i++) {
            List<String> acc = accounts.get(i);
            // index 0 is the name
            for (int j = 1; j < acc.size(); j++) {
                Integer prev = owner.putIfAbsent(acc.get(j), i);
                // shared email => same person
                if (prev != null) union(prev, i);
            }
        }

        Map<Integer, TreeSet<String>> groups = new HashMap<>();
        for (Map.Entry<String, Integer> e : owner.entrySet()) {
            int root = find(e.getValue());
            groups.computeIfAbsent(root, k -> new TreeSet<>
              ()).add(e.getKey());
        }

        List<List<String>> res = new ArrayList<>();
        for (Map.Entry<Integer, TreeSet<String>> g :
          groups.entrySet()) {
            List<String> merged = new ArrayList<>();
            // the group's shared name
            merged.add(accounts.get(g.getKey()).get(0));
            // sorted, no duplicates
            merged.addAll(g.getValue());
            res.add(merged);
        }
        return res;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
    }
}
```

Time O(L log L), L = total emails (sorting dominates; DSU adds O(L·α(n)))
Space O(L)

PATTERN 18

Shortest Path: Dijkstra's Algorithm

Key insight. Dijkstra is BFS with a priority queue instead of a FIFO queue. It needs exactly one property: *extending a path never makes it cheaper*. Sums of non-negative weights have it; so do the maximum edge on a path and the product of probabilities ≤ 1. Negative edges do not.

Concepts

Relaxation. For edge (u, v, w): if `dist[u] + w < dist[v]`, set `dist[v] = dist[u] + w` and push `(dist[v], v)`. Every algorithm in this family is a schedule of relaxations. Dijkstra's schedule relaxes the outgoing edges of each vertex exactly once, in order of final distance.

```
Invariant: when u is popped with d == dist[u], dist[u] = δ(s, u)
(true shortest distance)

Proof sketch: let P be a shortest s→u path, and let (x, y) be its
first edge with x settled, y not.
  dist[y] ≤ δ(s, x) + w(x, y) = δ(s, y) ≤ δ(s, u)        (prefix of a
shortest path; w ≥ 0)
  u was chosen as the minimum  ⇒  dist[u] ≤ dist[y] ≤ δ(s, u) ≤
dist[u]   ⇒   equality.
The step δ(s, y) ≤ δ(s, u) is exactly where non-negative weights are
used.
```

Stale entries (lazy deletion). `java.util.PriorityQueue` has no efficient decrease-key, so push a new entry on every improvement. Old entries stay in the heap. When popped, an entry whose `d > dist[u]` is stale and is skipped. The heap holds at most O(E) entries, so each operation costs O(log E) = O(log V), because $E \le V^2$.

```
Total: O((V + E) log V) time,  O(V + E) space (adjacency list + up to
E heap entries)
Dense graphs (E ≈ V²): the array version (scan for the minimum) is
O(V²) and beats the heap.
```

Why negative weights break it. Take the edges s→a (2), s→b (5) and b→a (−4). Dijkstra settles a at distance 2, but the true distance is 1 via b. Once a is settled, its outgoing edges were relaxed with the wrong value, so the error propagates. Adding a constant to every edge does *not* fix it, because paths with more edges are penalised more.

Generalised path costs. Dijkstra works for any cost that satisfies (1) *monotonicity*: cost(P + e) ≥ cost(P), and (2) *isotonicity*: if cost(P) ≤ cost(Q) for two paths to the same vertex, then cost(P + e) ≤ cost(Q + e). Bottleneck `max(cost, w)` (LC 1631) and the probability product with factors in [0, 1] under a max-heap (A2) both qualify.

State expansion. When a constraint makes "cheapest so far" insufficient (for example, a cheap path that has used too many edges), encode the constraint in the state: vertex v becomes (v, edgesUsed). The expanded graph has V·(K+2) states, and Dijkstra applies unchanged. The first time the target appears at the top of the heap, in *any* layer, its cost is optimal.

Template

Template A — adjacency-list Dijkstra with lazy deletion. Reusable: everything. Problem-specific: graph construction (directed or undirected, index base) and what you read from `dist`.

JAVA TEMPLATE Dijkstra.java

```java
import java.util.*;

/** Single-source shortest paths, non-negative weights. Vertices
  0..n-1, edges {u, v, w}. */
class Dijkstra {
    // headroom: INF + w never overflows
    static final long INF = Long.MAX_VALUE / 4;

    static long[] shortest(int n, int[][] edges, int src, boolean
      directed) {
        List<int[]>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] e : edges) {
            adj[e[0]].add(new int[]{e[1], e[2]});
            if (!directed) adj[e[1]].add(new int[]{e[0], e[2]});
        }
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        // Entry {distance, vertex}; Long.compare avoids subtraction
          overflow.
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
          Long.compare(a[0], b[0]));
        pq.add(new long[]{0, src});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
            long d = top[0];
            int u = (int) top[1];
            // stale entry: u was settled with a smaller d
            if (d > dist[u]) continue;
            for (int[] e : adj[u]) {
                int v = e[0];
                long nd = d + e[1];
                if (nd < dist[v]) {                // relaxation
                    dist[v] = nd;
                    pq.add(new long[]{nd, v});
                }
            }
        }
        // INF marks unreachable vertices
        return dist;
    }
}
```

Template B — grid Dijkstra with a pluggable path-cost rule. The `combine` line is the only problem-specific part: `d + w` for sums, `Math.max(d, w)` for bottleneck costs.

JAVA TEMPLATE GridDijkstra.java

```java
import java.util.*;

/** Dijkstra on an R x C grid, 4-directional; edge weight computed
  from the two cells. */
class GridDijkstra {
    static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};

    static int minCost(int[][] g, boolean bottleneck) {
        int R = g.length, C = g[0].length;
        int[] dist = new int[R * C];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0],
                  b[0]));
        // {cost, cellId}
        pq.add(new int[]{0, 0});
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int d = top[0], id = top[1];
            if (d > dist[id]) continue;
            // target popped: final
            if (id == R * C - 1) return d;
            int r = id / C, c = id % C;
            for (int k = 0; k < 4; k++) {
                int nr = r + DR[k], nc = c + DC[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C)
                  continue;
                // problem-specific weight
                int w = Math.abs(g[nr][nc] - g[r][c]);
                // problem-specific combine
                int nd = bottleneck ? Math.max(d, w) : d + w;
                int nid = nr * C + nc;
                if (nd < dist[nid]) {
                    dist[nid] = nd;
                    pq.add(new int[]{nd, nid});
                }
            }
        }
        return dist[R * C - 1];
    }
}
```

Recognition rule. Weighted edges + non-negative costs + "minimum total" from a source → Dijkstra. Before coding, ask whether the vertex alone is a sufficient state.

Problems

LC 1971 Find if Path Exists in Graph — EASY

Task. Given an undirected graph on `n` vertices (0-indexed edge list), decide whether `source` can reach `destination`.

KEY INSIGHT

Reachability depends only on connectivity. The first time BFS reaches `destination` is enough; distances are irrelevant.

JAVA SOLUTION LeetCode 1971 · Solution.java

```java
import java.util.*;

class Solution {
    public boolean validPath(int n, int[][] edges, int source, int
      destination) {
        if (source == destination) return true;
        List<Integer>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] e : edges) {                    // undirected
            adj[e[0]].add(e[1]);
```

```
            adj[e[1]].add(e[0]);
        }
        boolean[] seen = new boolean[n];
        Deque<Integer> q = new ArrayDeque<>();
        q.add(source);
        // mark on enqueue: each vertex enters once
        seen[source] = true;
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : adj[u]) {
                if (seen[v]) continue;
                if (v == destination) return true;
                seen[v] = true;
                q.add(v);
            }
        }
        return false;
    }
}
```

Time O(V + E) **Space** O(V + E)

LC 743 Network Delay Time — MEDIUM

Task. Directed edges `times[i] = [u, v, w]` on nodes 1..n. A signal starts at `k`. Return the time until all nodes have received it, or −1 if some node is unreachable.

KEY INSIGHT

A node receives the signal at its shortest-path distance from `k`, and all nodes are done at the *maximum* of those distances.

JAVA SOLUTION LeetCode 743 · Solution.java

```
import java.util.*;

class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        // nodes are 1-indexed
        List<int[]>[] adj = new List[n + 1];
        for (int i = 1; i <= n; i++) adj[i] = new ArrayList<>();
        // directed
        for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});
        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0],
                  b[0]));
        pq.add(new int[]{0, k});
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int d = top[0], u = top[1];
            if (d > dist[u]) continue;                  // stale
            for (int[] e : adj[u]) {
                // d is finite here: no overflow
                int nd = d + e[1];
                if (nd < dist[e[0]]) {
                    dist[e[0]] = nd;
                    pq.add(new int[]{nd, e[0]});
                }
            }
        }
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            ans = Math.max(ans, dist[i]);
        }
        return ans;
    }
}
```

Time O((V + E) log V) **Space** O(V + E)

LC 1631 Path With Minimum Effort — MEDIUM

Task. In a height grid, a route's effort is the maximum absolute height difference between consecutive cells. Minimise the effort of a route from the top-left to the bottom-right cell (4-directional moves).

KEY INSIGHT

Replace `dist[v] = dist[u] + w` with `dist[v] = max(dist[u], w)`. The first time the target is popped, no other route can have a smaller bottleneck.

JAVA SOLUTION LeetCode 1631 · Solution.java

```
import java.util.*;

class Solution {
    private static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1,
      -1};

    public int minimumEffortPath(int[][] heights) {
        int R = heights.length, C = heights[0].length, target = R *
          C - 1;
        int[] effort = new int[R * C];
        Arrays.fill(effort, Integer.MAX_VALUE);
        effort[0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0],
                  b[0]));
        // {effort, cellId}
        pq.add(new int[]{0, 0});
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int e = top[0], id = top[1];
            // stale
            if (e > effort[id]) continue;
            // settled: optimal bottleneck
            if (id == target) return e;
            int r = id / C, c = id % C;
            for (int k = 0; k < 4; k++) {
                int nr = r + DR[k], nc = c + DC[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C)
                  continue;
                int w = Math.abs(heights[nr][nc] - heights[r][c]);
                // bottleneck combine
                int ne = Math.max(e, w);
                int nid = nr * C + nc;
                if (ne < effort[nid]) {
                    effort[nid] = ne;
                    pq.add(new int[]{ne, nid});
                }
            }
        }
        // reached only if 1x1 grid
        return effort[target];
    }
}
```

Time O(RC · log(RC)) **Space** O(RC)

LC 787 Cheapest Flights Within K Stops — MEDIUM

Task. Directed flights `[from, to, price]` (price ≥ 1). Return the cheapest price from `src` to `dst` using at most `k` stops (that is, at most k + 1 flights), or −1.

KEY INSIGHT

Run Dijkstra on states `(city, flightsUsed)`. In the expanded graph the cheapest prefix is once again sufficient, because two states with the same city and flight count have identical futures.

JAVA SOLUTION LeetCode 787 · Solution.java

```
import java.util.*;

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src,
      int dst, int k) {
        List<int[]>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        // directed
        for (int[] f : flights) adj[f[0]].add(new int[]{f[1],
          f[2]});
        // k stops = k + 1 edges
        int maxFlights = k + 1;
        int[][] best = new int[n][maxFlights + 1];
        for (int[] row : best) Arrays.fill(row, Integer.MAX_VALUE);
        best[src][0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0],
                  b[0]));
        // {cost, city, flightsUsed}
        pq.add(new int[]{0, src, 0});
        while (!pq.isEmpty()) {
            int[] t = pq.poll();
            int cost = t[0], u = t[1], used = t[2];
            if (cost > best[u][used]) continue;    // stale state
            // cheapest valid state reaching dst
            if (u == dst) return cost;
            // no flights left
            if (used == maxFlights) continue;
            for (int[] e : adj[u]) {
                // <= (k+1) * 1e4: fits in int
                int nc = cost + e[1];
                if (nc < best[e[0]][used + 1]) {
                    best[e[0]][used + 1] = nc;
                    pq.add(new int[]{nc, e[0], used + 1});
                }
            }
        }
        return -1;
    }
}
```

Time O(K·E · log(K·E)) **Space** O(n·K + K·E) states and heap entries

PATTERN 19

Trie / Prefix Tree

Key insight. Use a trie when you need to ask "*can this partial string still become a word?*" while building the string one character at a time. That early "no" is what prunes backtracking in Word Search II.

Concepts

Structural invariant. The path from the root to node x spells a unique string prefix(x). Node x exists if and only if some inserted word has prefix(x) as a prefix. `x.end` (or a stored word) is true if and only if prefix(x) itself was inserted.

```
insert(w), search(w), startsWith(p):  O(L) time, L = query length
Total nodes ≤ 1 + Σ |w_i|          (one node per distinct prefix)
Memory (array children): nodes · alphabet · reference size  ≈ ΣL · 26 · 4–8 bytes
```

End-of-word markers. Prefix and word are different questions. After inserting "apple", `search("app")` is false while `startsWith("app")` is true. Storing the full word at the terminal node (`String word` instead of `boolean end`) avoids rebuilding strings during DFS, which is standard in Word Search II.

Node design choices.

Children store	Pros	Cons
`Node[26]`	O(1) child access, fastest, simplest	26 references per node even when sparse
`HashMap<Character, Node>`	Any alphabet, proportional memory	Hashing and boxing overhead; slower constants
Flat arrays `int[nodes][26]`	Cache-friendly, no object headers	Must pre-size the node count (≤ ΣL + 1)

Counters on nodes. A `pass` count (how many words go through a node) answers "how many words have prefix p" in O(|p|) and supports deletion. Decrement along the path, and a node with `pass == 0` can be detached.

Wildcard search cost. A `'.'` at depth d explores every existing child, so the worst case for a pattern of length L is $O(26^k \cdot L)$ for k wildcards, bounded by the number of trie nodes at those depths. The trie does not make wildcards cheap; it confines branching to prefixes that actually exist.

Template

Template A — array-children trie with prefix counts. Reusable: the node layout and walk. Problem-specific: what is stored at nodes (`end`, `word`, counts).

JAVA TEMPLATE ArrayTrie.java

```java
/** Lowercase-alphabet trie. pass = words through node; endCount = words ending here. */
class ArrayTrie {
    private static final int ALPHA = 26;

    private static final class Node {
        final Node[] next = new Node[ALPHA];
        int pass, endCount;
    }

    private final Node root = new Node();

    void insert(String w) {
        Node cur = root;
        cur.pass++;
        for (int i = 0; i < w.length(); i++) {
            int c = w.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
            cur.pass++;
        }
        cur.endCount++;
    }

    /** Last node of the path spelling s, or null if s is not a prefix of any word. */
    private Node walk(String s) {
        Node cur = root;
        for (int i = 0; i < s.length() && cur != null; i++) {
            cur = cur.next[s.charAt(i) - 'a'];
        }
        return cur;
    }

    boolean search(String w)      { Node n = walk(w); return n != null && n.endCount > 0; }
    // Uses pass, not node existence: the root exists even when the trie is empty.
    boolean startsWith(String p)  { return countPrefix(p) > 0; }
    int countWords(String w)      { Node n = walk(w); return n == null ? 0 : n.endCount; }
    int countPrefix(String p)     { Node n = walk(p); return n == null ? 0 : n.pass; }

    /** Removes one occurrence of w if present; detaches nodes no word passes through. */
    void erase(String w) {
        if (countWords(w) == 0) return;
        Node cur = root;
        cur.pass--;
        for (int i = 0; i < w.length(); i++) {
            int c = w.charAt(i) - 'a';
            // subtree now unused
            if (--cur.next[c].pass == 0) { cur.next[c] = null; return; }
            cur = cur.next[c];
        }
        cur.endCount--;
    }
}
```

Recognition rule. Many words + prefix questions or character-by-character construction → trie. One word or only whole-word membership → hash set.

Template B — map-children trie for arbitrary alphabets, with shortest-prefix lookup.

JAVA TEMPLATE MapTrie.java

```java
import java.util.*;

/** Trie over arbitrary chars. Children in a HashMap: memory proportional to real edges. */
class MapTrie {
    private static final class Node {
        final Map<Character, Node> next = new HashMap<>();
        boolean end;
    }

    private final Node root = new Node();

    void insert(String w) {
        Node cur = root;
        for (char ch : w.toCharArray()) cur = cur.next.computeIfAbsent(ch, k -> new Node());
        cur.end = true;
    }

    /** Length of the shortest inserted word that is a prefix of s, or -1. */
    int shortestPrefixLength(String s) {
        Node cur = root;
        for (int i = 0; i < s.length(); i++) {
            cur = cur.next.get(s.charAt(i));
            // dead prefix: no word can match further
            if (cur == null) return -1;
            // first terminal on the path is the shortest
            if (cur.end) return i + 1;
        }
        return -1;
    }
}
```

Problems

LC 208 Implement Trie (Prefix Tree) — MEDIUM

Task. Implement `insert`, `search` (whole word) and `startsWith` (prefix) for lowercase words.

KEY INSIGHT

Both queries share one walk. They differ only in whether the final node must carry an end marker.

JAVA SOLUTION LeetCode 208 · Trie.java

```java
class Trie {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public Trie() {}

    public void insert(String word) {
        Node cur = root;
        for (int i = 0; i < word.length(); i++) {
            int c = word.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
        }
```

```
        cur.end = true;
    }

    public boolean search(String word) {
        Node n = find(word);
        // node existing only proves a prefix
        return n != null && n.end;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private Node find(String s) {
        Node cur = root;
        for (int i = 0; i < s.length(); i++) {
            cur = cur.next[s.charAt(i) - 'a'];
            if (cur == null) return null;
        }
        return cur;
    }
}
```

Time O(L) per operation **Space** O(ΣL · 26) total nodes

LC 211 Design Add and Search Words Data Structure — MEDIUM

Task. Support `addWord` and `search`, where `'.'` in the query matches any single letter (LeetCode allows at most 2 dots per query).

KEY INSIGHT

A literal character follows one edge. A dot branches over the existing children only, so the trie limits the search to prefixes that are actually present.

JAVA SOLUTION LeetCode 211 · WordDictionary.java

```
class WordDictionary {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public WordDictionary() {}

    public void addWord(String word) {
        Node cur = root;
        for (int i = 0; i < word.length(); i++) {
            int c = word.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
        }
        cur.end = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0);
    }

    private boolean dfs(Node node, String w, int i) {
        if (i == w.length()) return node.end;
        char ch = w.charAt(i);
        if (ch == '.') {
            // branch only over existing prefixes
            for (Node child : node.next) {
                if (child != null && dfs(child, w, i + 1)) return
                  true;
            }
            return false;
        }
        Node child = node.next[ch - 'a'];
        return child != null && dfs(child, w, i + 1);
    }
}
```

Time add O(L); search O(L) without dots, O(26^d · L) with d dots
Space O(ΣL · 26) + O(L) recursion

LC 648 Replace Words — MEDIUM

Task. Replace each word of a sentence with the *shortest* dictionary root that is a prefix of it; leave words with no root unchanged.

KEY INSIGHT

Walking a word down the trie meets candidate roots in increasing length, so the first end marker on the path is the shortest root. The walk stops at that marker or at a dead prefix.

JAVA SOLUTION LeetCode 648 · Solution.java

```
import java.util.*;

class Solution {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
    }

    public String replaceWords(List<String> dictionary, String
      sentence) {
        Node root = new Node();
        for (String r : dictionary) {
            Node cur = root;
            for (int i = 0; i < r.length(); i++) {
                int c = r.charAt(i) - 'a';
                if (cur.next[c] == null) cur.next[c] = new Node();
                cur = cur.next[c];
            }
            cur.end = true;
        }
        StringBuilder sb = new StringBuilder();
        for (String w : sentence.split(" ")) {
            if (sb.length() > 0) sb.append(' ');
            sb.append(w, 0, rootLength(root, w));
        }
        return sb.toString();
    }

    /** Length of the shortest root prefixing w, or w.length() if
      none. */
    private int rootLength(Node root, String w) {
        Node cur = root;
        for (int i = 0; i < w.length(); i++) {
            cur = cur.next[w.charAt(i) - 'a'];
            // no root continues this prefix
            if (cur == null) return w.length();
            // first terminal = shortest root
            if (cur.end) return i + 1;
        }
        return w.length();
    }
}
```

Time O($ΣL_{dict}$ + |sentence|) **Space** O($ΣL_{dict}$ · 26) + output

LC 212 Word Search II — HARD

Task. Return all words from a list that can be traced on an m×n letter board through 4-directionally adjacent cells, without reusing a cell within one word.

KEY INSIGHT

Search the board once for all words together. The DFS carries a trie node, and a missing child ends the branch immediately. Removing words once found, and pruning emptied leaves, keeps later searches short.

JAVA SOLUTION LeetCode 212 · Solution.java

```
import java.util.*;

class Solution {
    private static final class Node {
        final Node[] next = new Node[26];
        // non-null iff a not-yet-found word ends here
        String word;
        // live child count, for leaf pruning
        int children;
    }

    private static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1,
      -1};
    private char[][] board;
    private List<String> found;

    public List<String> findWords(char[][] board, String[] words) {
        Node root = new Node();
        for (String w : words) {
            Node cur = root;
            for (int i = 0; i < w.length(); i++) {
                int c = w.charAt(i) - 'a';
                if (cur.next[c] == null) { cur.next[c] = new Node();
                  cur.children++; }
                cur = cur.next[c];
            }
            // duplicates in words collapse to one entry
            cur.word = w;
        }
        this.board = board;
        this.found = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                if (root.next[board[r][c] - 'a'] != null) dfs(r, c,
                  root);
            }
        }
        return found;
    }

    private void dfs(int r, int c, Node parent) {
        char ch = board[r][c];
        int idx = ch - 'a';
        Node node = parent.next[idx];
        // dead prefix
        if (node == null) return;
        if (node.word != null) {
            found.add(node.word);
            // report each word once
            node.word = null;
```

```
        }
        // choose: cell used on this path
        board[r][c] = '#';
        for (int k = 0; k < 4; k++) {
            int nr = r + DR[k], nc = c + DC[k];
            if (nr < 0 || nr >= board.length || nc < 0 || nc >=
              board[0].length) continue;
            if (board[nr][nc] == '#') continue;
            dfs(nr, nc, node);
        }
        board[r][c] = ch;                                   // undo
```

```
            // exhausted branch: prune it
            if (node.children == 0 && node.word == null) {
                parent.next[idx] = null;
                parent.children--;
            }
        }
    }
```

Time $O(RC \cdot 4 \cdot 3^{L-1})$ worst case, $L \le 10$; build $O(\Sigma L)$

Space $O(\Sigma L \cdot 26)$ trie + $O(L)$ recursion

PATTERN 20

Segment Tree / Fenwick Tree (Binary Indexed Tree)

Key insight. Two questions decide the structure. (1) Is the merge operation invertible (sum, xor, counts)? If yes and updates are point updates, a Fenwick tree is enough. (2) Do you need non-invertible aggregates (max, min) over arbitrary ranges, range assignments, or custom merges? Then use a segment tree, with lazy propagation if updates cover ranges.

Recognition rule. Interleaved updates and range queries over the same array (or value axis) → Fenwick if the operation is invertible and updates are point updates; otherwise a segment tree, lazy if updates cover ranges.

Concepts

Segment tree — array representation. Node 1 is the root and covers the whole input range `[0, n−1]`. Node `x` covering `[l, r]` has children `2x` covering `[l, mid]` and `2x+1` covering `[mid+1, r]`, with `mid = (l + r) >>> 1`.

```
Invariant:  tree[x] = merge(tree[2x], tree[2x+1]),  leaves hold a[i]
Height h = ⌈log₂ n⌉, so indices are < 2^(h+1) ≤ 4n   ⇒  allocate 4n
Query [ql, qr]: at most 2 nodes per level are partially covered ⇒
O(log n) nodes visited
Point update: one root-to-leaf path, then recompute ancestors ⇒ O(log
n)
Build: each of the < 2n real nodes is computed once ⇒ O(n)
```

The segment tree needs only an **associative** merge with an **identity** element for ranges that fall outside the query (0 for sum, `Integer.MIN_VALUE` for max, +∞ for min). Commutativity is not required as long as left and right are merged in order. That is what allows merges such as "max subarray sum within a range" that store a tuple per node.

Bottom-up (iterative) segment tree. Leaves live at `[n, 2n)` and node `x` has children `2x` and `2x+1`. A query over the half-open range `[l, r)` walks both bounds upward. It is short and fast, and its natural convention is *half-open*. The 2407 solution uses it; the recursive Template A uses *inclusive* bounds. Never mix the two conventions inside one implementation.

Lazy propagation (range updates). A range update would touch O(n) leaves, so instead store a pending *tag* on each fully covered node and push it to the children only when a later operation needs to descend. This requires (1) a tag that can be applied to a node aggregate in O(1) (add: `sum += tag · length`; assign: `max = value`), and (2) tags that *compose* (add then add equals one add; assign overrides an earlier assign or add). If you cannot define both, lazy propagation does not apply as written.

Fenwick tree — the low-bit structure. Internally 1-based. `bit[i]` stores the sum of the half-open range `(i − lowbit(i), i]`, where `lowbit(i) = i & −i` is the value of i's lowest set bit.

```
Two's complement:  −i = ~i + 1.  Below the lowest set bit, ~i has 1s
and adding 1 carries
through them, flipping back exactly that bit. Above it, i and ~i
differ, so i & −i keeps
only the lowest set bit.     e.g. i = 12 = 1100₂,  −i = …0100₂,  i &
−i = 4

prefix(i) = bit[i] + prefix(i − lowbit(i))      → strip low bits: ≤
log₂ n steps
update(i, δ): bit[i] += δ; i += lowbit(i)         → climb to every
block containing i
rangeSum(l, r) = prefix(r) − prefix(l − 1)        → requires an
inverse ("−")
Index 0 is unusable: lowbit(0) = 0, so update(0) would loop forever.
Hence 1−based.
```

Zero-based input vs one-based internals. Keep the public API 0-based, matching LeetCode, and shift once at the boundary (`i + 1`) inside `add` and `prefix`. Shifting by hand at call sites is where off-by-one bugs come from.

Why Fenwick fails for range max. With max you cannot recover max(l..r) from max(1..r) and max(1..l−1); the larger prefix hides the window. A Fenwick "prefix max" works only for *prefix* queries with *increase-only* updates. LC 2407 needs a window `[x−k, x−1]`, so it needs a segment tree.

Choosing by update/query type

Updates	Queries	Structure
none	range sum	Prefix sums, O(1) query
point (add / set)	range sum / count / xor	Fenwick (set = add the difference; keep a copy of the array), or a segment tree
range add	point value	Fenwick over the *difference array*: add at l, subtract at r+1; the point value is a prefix sum
range add	range sum	Lazy segment tree, or two Fenwick trees
point set	range max / min / gcd	Segment tree (recursive or bottom-up)
range assign	range max	Lazy segment tree with an assign tag (LC 699)

Template

Template A — recursive segment tree, point set, inclusive range query. Shown for sum. For max or min, change `merge` and `IDENTITY` only.

JAVA TEMPLATE SegmentTree.java

```
/** Array-based segment tree. Public indices are 0-based; ranges [l,
  r] are inclusive. */
class SegmentTree {
    // max: Long.MIN_VALUE; min: MAX
    private static final long IDENTITY = 0;
    // must be associative
    private static long merge(long a, long b) { return a + b; }

    private final int n;
    // node 1 = root over [0, n-1]
    private final long[] tree;

    SegmentTree(int[] a) {
        n = a.length;
        // height ceil(log2 n) => < 4n nodes
        tree = new long[4 * Math.max(1, n)];
        if (n > 0) build(1, 0, n - 1, a);
    }

    private void build(int x, int l, int r, int[] a) {
        if (l == r) { tree[x] = a[l]; return; }
        int mid = (l + r) >>> 1;
        build(2 * x, l, mid, a);
        build(2 * x + 1, mid + 1, r, a);
        tree[x] = merge(tree[2 * x], tree[2 * x + 1]);
    }

    /** a[i] = value. */
    void set(int i, long value) { set(1, 0, n - 1, i, value); }

    private void set(int x, int l, int r, int i, long value) {
        if (l == r) { tree[x] = value; return; }
        int mid = (l + r) >>> 1;
        if (i <= mid) set(2 * x, l, mid, i, value);
        else set(2 * x + 1, mid + 1, r, i, value);
        // recompute along the path
        tree[x] = merge(tree[2 * x], tree[2 * x + 1]);
    }

    /** Aggregate of a[ql..qr], inclusive. */
    long query(int ql, int qr) { return query(1, 0, n - 1, ql, qr);
      }

    private long query(int x, int l, int r, int ql, int qr) {
        // disjoint
        if (qr < l || r < ql) return IDENTITY;
        // fully covered: canonical block
        if (ql <= l && r <= qr) return tree[x];
        int mid = (l + r) >>> 1;
        return merge(query(2 * x, l, mid, ql, qr), query(2 * x + 1,
          mid + 1, r, ql, qr));
    }
```

```
}
```

Template B — Fenwick tree (BIT): point add, prefix and range sums.

JAVA TEMPLATE Fenwick.java

```java
/** Fenwick tree for sums. Public API is 0-based; internal array is
  1-based. */
class Fenwick {
    // bit[i] = sum of (i - lowbit(i), i]
    private final long[] bit;

    Fenwick(int n) { bit = new long[n + 1]; }

    /** O(n) build: push each node's value to its immediate parent
      block. */
    Fenwick(int[] a) {
        this(a.length);
        for (int i = 1; i <= a.length; i++) {
            bit[i] += a[i - 1];
            int parent = i + (i & -i);
            if (parent <= a.length) bit[parent] += bit[i];
        }
    }

    /** a[i0] += delta. */
    void add(int i0, long delta) {
        for (int i = i0 + 1; i < bit.length; i += i & -i) bit[i] +=
          delta;
    }

    /** Sum of a[0..i0] inclusive; prefix(-1) == 0. */
    long prefix(int i0) {
        long s = 0;
        for (int i = i0 + 1; i > 0; i -= i & -i) s += bit[i];
        return s;
    }

    /** Sum of a[l..r] inclusive, via prefix difference (needs
      subtraction). */
    long rangeSum(int l, int r) { return prefix(r) - prefix(l - 1);
      }
}
```

Problems

LC 307 Range Sum Query - Mutable — MEDIUM

Task. Implement `NumArray(nums)`, `update(index, val)` (sets a value) and `sumRange(left, right)` (inclusive), with up to $3 \cdot 10^4$ calls.

KEY INSIGHT

Static prefix sums make each update O(n). Storing O(log n)-sized block sums makes both operations logarithmic. Sum is invertible, so either structure works; this is the problem to practise both.

JAVA SOLUTION LeetCode 307 · NumArray.java (segment tree)

```java
class NumArray {
    private final int n;
    // node 1 covers [0, n-1]; sums fit in int
    private final int[] tree;

    public NumArray(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(1, 0, n - 1, nums);
    }

    private void build(int x, int l, int r, int[] a) {
        if (l == r) { tree[x] = a[l]; return; }
        int mid = (l + r) >>> 1;
        build(2 * x, l, mid, a);
        build(2 * x + 1, mid + 1, r, a);
        tree[x] = tree[2 * x] + tree[2 * x + 1];
    }

    public void update(int index, int val) { update(1, 0, n - 1,
      index, val); }

    private void update(int x, int l, int r, int i, int val) {
        if (l == r) { tree[x] = val; return; }
        int mid = (l + r) >>> 1;
        if (i <= mid) update(2 * x, l, mid, i, val);
        else update(2 * x + 1, mid + 1, r, i, val);
        tree[x] = tree[2 * x] + tree[2 * x + 1];
    }

    public int sumRange(int left, int right) { return query(1, 0, n
      - 1, left, right); }

    private int query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[x];
        int mid = (l + r) >>> 1;
        return query(2 * x, l, mid, ql, qr) + query(2 * x + 1, mid +
          1, r, ql, qr);
    }
}
```

Time build O(n); update and query O(log n)
Space O(n) (4n segment tree, n + 1 Fenwick)

LC 315 Count of Smaller Numbers After Self — HARD

Task. For each i, count the indices j > i with `nums[j] < nums[i]`; n ≤ 10^5, values in $[-10^4, 10^4]$.

KEY INSIGHT

Sweep from right to left while maintaining frequencies of the values already seen (exactly those with j > i). The answer for i is the count of seen values whose rank is smaller than `rank(nums[i])`: a prefix sum over the value axis.

JAVA SOLUTION LeetCode 315 · Solution.java

```java
import java.util.*;

class Solution {
    public List<Integer> countSmaller(int[] nums) {
        int n = nums.length;
        int[] sorted = nums.clone();
        Arrays.sort(sorted);
        // compress to distinct ascending
        int m = 0;
        for (int i = 0; i < n; i++) {
            if (i == 0 || sorted[i] != sorted[i - 1]) sorted[m++] =
              sorted[i];
        }
        // Fenwick over ranks, 1-based inside
        int[] bit = new int[m + 1];

        Integer[] ans = new Integer[n];
        for (int i = n - 1; i >= 0; i--) {
            // 0-based rank
            int rank = Arrays.binarySearch(sorted, 0, m, nums[i]);
            int count = 0;
            // ranks 0..rank-1
            for (int k = rank; k > 0; k -= k & -k) count += bit[k];
            ans[i] = count;
            // add value at rank
            for (int k = rank + 1; k <= m; k += k & -k) bit[k]++;
        }
        return Arrays.asList(ans);
    }
}
```

Time O(n log n) **Space** O(n)

LC 2407 Longest Increasing Subsequence II — HARD

Task. Find the longest strictly increasing subsequence in which adjacent elements differ by at most `k`; n, values, k ≤ 10^5.

KEY INSIGHT

Let `best[v]` be the longest valid subsequence seen so far that ends with value v. Processing left to right, `len(x) = 1 + max(best[v] for v in [x - k, x - 1])`. That is a range-max query on the value axis followed by a point update `best[x] = max(best[x], len)`.

JAVA SOLUTION LeetCode 2407 · Solution.java

```java
class Solution {
    public int lengthOfLIS(int[] nums, int k) {
        int maxV = 0;
        for (int x : nums) maxV = Math.max(maxV, x);
        int size = 1;
        // leaves for values 0..maxV
        while (size < maxV + 1) size <<= 1;
        // bottom-up max tree, identity 0
        int[] tree = new int[2 * size];
        int ans = 0;
        for (int x : nums) {
            // values [x-k, x-1]
            int len = query(tree, size, Math.max(1, x - k), x) + 1;
            ans = Math.max(ans, len);
            // point update: best[x] = max(., len)
            int i = x + size;
            tree[i] = Math.max(tree[i], len);
            for (i >>= 1; i >= 1; i >>= 1) tree[i] = Math.max(tree[2
              * i], tree[2 * i + 1]);
        }
        return ans;
    }

    /** Max over leaf positions [l, r) (half-open), 0 if empty. */
    private int query(int[] tree, int size, int l, int r) {
        int res = 0;
        for (l += size, r += size; l < r; l >>= 1, r >>= 1) {
            // l is a right child
            if ((l & 1) == 1) res = Math.max(res, tree[l++]);
            // r-1 is a left child
            if ((r & 1) == 1) res = Math.max(res, tree[--r]);
        }
        return res;
    }
}
```

Time O(n log V) **Space** O(V)

LC 699 Falling Squares — HARD

Task. Squares `[left, side]` fall one by one and land on the highest square they overlap (touching edges do not count). After each drop, report the tallest stack height. Coordinates go up to 10^8; up to 1000 squares.

KEY INSIGHT

A square occupies the half-open interval `[left, left + side)`. Its landing height is `max(height over that interval) + side`, and afterwards the whole interval has exactly that height. That is a range-max query followed by a *range assignment*, on a compressed axis.

JAVA SOLUTION LeetCode 699 · Solution.java

```java
import java.util.*;

class Solution {
    // tag[x] = pending assignment, -1 if none
    private int[] mx, tag;

    public List<Integer> fallingSquares(int[][] positions) {
        int[] xs = new int[2 * positions.length];
        int p = 0;
        for (int[] s : positions) { xs[p++] = s[0]; xs[p++] = s[0] +
          s[1]; }
        Arrays.sort(xs);
        int m = 0;
        for (int i = 0; i < xs.length; i++) {
            if (i == 0 || xs[i] != xs[i - 1]) xs[m++] = xs[i];
        }
        // slot i = cell [xs[i], xs[i+1]); m - 1 cells are
          meaningful, m slots allocated
        mx = new int[4 * m];
        tag = new int[4 * m];
        Arrays.fill(tag, -1);

        List<Integer> res = new ArrayList<>();
        int best = 0;
        for (int[] s : positions) {
            int l = Arrays.binarySearch(xs, 0, m, s[0]);
            // half-open -> inclusive
            int r = Arrays.binarySearch(xs, 0, m, s[0] + s[1]) - 1;
            int h = query(1, 0, m - 1, l, r) + s[1];
            assign(1, 0, m - 1, l, r, h);
            best = Math.max(best, h);
            res.add(best);
        }
        return res;
    }

    // later assign wins
    private void applyAssign(int x, int v) { mx[x] = v; tag[x] = v;
      }

    private void push(int x) {
        if (tag[x] < 0) return;
        applyAssign(2 * x, tag[x]);
        applyAssign(2 * x + 1, tag[x]);
        tag[x] = -1;
    }

    private void assign(int x, int l, int r, int ql, int qr, int v)
      {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { applyAssign(x, v); return; }
        push(x);
        int mid = (l + r) >>> 1;
        assign(2 * x, l, mid, ql, qr, v);
        assign(2 * x + 1, mid + 1, r, ql, qr, v);
        mx[x] = Math.max(mx[2 * x], mx[2 * x + 1]);
    }

    private int query(int x, int l, int r, int ql, int qr) {
        // heights are >= 0: 0 is the identity
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return mx[x];
        push(x);
        int mid = (l + r) >>> 1;
        return Math.max(query(2 * x, l, mid, ql, qr), query(2 * x +
          1, mid + 1, r, ql, qr));
    }
}
```

Time O(n log n) **Space** O(n)

ALGORITHM A1

Monotonic Stack

A stack whose contents stay sorted. Each element is pushed once and popped once, and the moment it is popped is exactly when you learn its nearest greater (or smaller) neighbour.

Concepts

Goal (scanning left → right)	Stack order bottom → top	Pop while…
Next greater element	Decreasing	`a[top] < a[i]`: i is top's next strictly greater
Next greater or equal	Strictly decreasing	`a[top] <= a[i]`
Next smaller element	Increasing	`a[top] > a[i]`
Previous smaller (after pops)	Increasing	Pop `a[top] >= a[i]`; the new top is the previous strictly smaller element

Invariant (next-greater form). After processing index i, the stack holds exactly the indices j ≤ i that do not yet have a next greater element, and their values are non-increasing from bottom to top. A popped index has found its answer, and that answer is final: i is the first index after it with a larger value, because every index between them was either smaller (popped earlier, or never above it) or is still on the stack above it, which contradicts the order.

Index vs value storage. Store *indices*. They give you the distance (`i - top`), the value (`a[top]`) and a write position in the answer array. Store values only when positions never matter (e.g. remove-k-digits builds the output directly).

Amortised O(n). Each index is pushed exactly once and popped at most once, so the inner while loop runs at most n times *in total* across all iterations, even though a single iteration may pop many.

For contribution counting, let `L[i]` be the index of the previous element *strictly smaller* than a[i] (or −1), and `R[i]` be the index of the next element *smaller or equal* (or n). Then a[i] is the minimum of every subarray [l, r] with L[i] < l ≤ i ≤ r < R[i]:

```
count(i) = (i - L[i]) · (R[i] - i)
Σ over subarrays of min = Σᵢ a[i] · (i - L[i]) · (R[i] - i)
```

Why one side strict and the other non-strict: with duplicates, a subarray such as [2, 2] has two
positions holding its minimum. Making exactly one side non-strict assigns it to exactly one of them
(here the rightmost occurrence), so nothing is double-counted or missed.

Template

Template A — next greater (right) and previous smaller (left). `ArrayDeque` used as a stack: `push` / `peek` / `pop` act on the head.

JAVA TEMPLATE MonotonicStack.java

```java
import java.util.*;

class MonotonicStack {
    /** nge[i] = first j > i with a[j] > a[i], or -1. Stack values
      non-increasing. */
    static int[] nextGreater(int[] a) {
        int n = a.length;
        int[] nge = new int[n];
        Arrays.fill(nge, -1);
        // holds indices still waiting
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            // strict: equal values wait
            while (!st.isEmpty() && a[st.peek()] < a[i]) {
                nge[st.pop()] = i;
            }
            st.push(i);
        }
        return nge;
    }

    /** pse[i] = last j < i with a[j] < a[i], or -1. Stack values
      strictly increasing. */
    static int[] previousSmaller(int[] a) {
        int n = a.length;
        int[] pse = new int[n];
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
```

```
            // drop not-smaller
            while (!st.isEmpty() && a[st.peek()] >= a[i]) st.pop();
            pse[i] = st.isEmpty() ? -1 : st.peek();
            st.push(i);
        }
        return pse;
    }
}
```

Template B — contribution boundaries in one pass. An index is popped by the first element to its right that is ≤ it (right boundary, non-strict), and the element below it on the stack is its previous strictly smaller element (left boundary). A sentinel of −∞ at position n flushes the stack.

JAVA TEMPLATE MinContribution.java

```
import java.util.*;

class MinContribution {
    /** Sum of min over all subarrays, exact (no modulus), using
      long. */
    static long sumOfSubarrayMins(int[] a) {
        int n = a.length;
        long total = 0;
        // indices; values strictly increasing
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i <= n; i++) {
            // sentinel pops everything
            long cur = (i == n) ? Long.MIN_VALUE : a[i];
            while (!st.isEmpty() && a[st.peek()] >= cur) {
                int mid = st.pop();
                // previous strictly smaller
                int left = st.isEmpty() ? -1 : st.peek();
                // i: next <= a[mid]
                total += (long) a[mid] * (mid - left) * (i - mid);
            }
            st.push(i);
        }
        return total;
    }
}
```

Problems

LC 739 Daily Temperatures — MEDIUM

Task. For each day, return how many days you must wait for a strictly warmer temperature (0 if never).

KEY INSIGHT

"Next strictly greater element to the right", with the answer expressed as an index distance. A day is popped exactly when its first warmer day arrives.

JAVA SOLUTION LeetCode 739 · Solution.java

```
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        // default 0 = no warmer day
        int[] ans = new int[n];
        // array-backed stack of indices
        int[] stack = new int[n];
        int top = -1;
        for (int i = 0; i < n; i++) {
            while (top >= 0 && temperatures[stack[top]] <
              temperatures[i]) {
                int j = stack[top--];
                ans[j] = i - j;
            }
            stack[++top] = i;
        }
        return ans;
    }
}
```

Time O(n) amortised **Space** O(n) stack

LC 907 Sum of Subarray Minimums — MEDIUM

Task. Return the sum of `min(sub)` over every contiguous subarray, modulo 10^9+7.

KEY INSIGHT

Instead of enumerating $O(n^2)$ subarrays, count how many subarrays each element is the minimum of: `(i − L[i]) · (R[i] − i)`, with one strict and one non-strict boundary so that duplicates are counted once.

JAVA SOLUTION LeetCode 907 · Solution.java

```
class Solution {
    private static final int MOD = 1_000_000_007;

    public int sumSubarrayMins(int[] arr) {
        int n = arr.length;
        // indices; values strictly increasing bottom -> top
        int[] stack = new int[n + 1];
        int top = -1;
        long total = 0;
        for (int i = 0; i <= n; i++) {
            // arr[i] >= 1, so 0 is a -infinity sentinel
            int cur = (i == n) ? 0 : arr[i];
            while (top >= 0 && arr[stack[top]] >= cur) {
                int mid = stack[top--];
                // previous strictly smaller
                int left = (top >= 0) ? stack[top] : -1;
                // i = next smaller-or-equal
                long count = (long) (mid - left) * (i - mid);
                total = (total + count % MOD * arr[mid]) % MOD;
            }
            stack[++top] = i;
        }
        return (int) total;
    }
}
```

Time O(n) **Space** O(n)

ALGORITHM A2

Dijkstra's Algorithm

> Greedy single-source shortest paths for non-negative weights: always finalise the closest unfinished vertex. Chapter 18 covers the pattern and its problem shapes. This chapter covers the proof, the exact conditions under which the greedy step stays valid, and two generalisations: multiplicative path values and counting shortest paths.

Concepts

Relaxation. For an edge u → v with weight w: `if dist[u] + w < dist[v] then dist[v] = dist[u] + w`. Every shortest-path algorithm is a strategy for ordering relaxations. Dijkstra orders them by settling vertices in increasing distance, so each edge is relaxed once from its final source distance.

Invariant. Let S be the set of settled vertices. (1) For every v ∈ S, `dist[v]` equals the true distance δ(s, v). (2) For every v ∉ S, `dist[v]` is the shortest path to v that uses only vertices of S before its last edge.

Why negative weights break it. Edges s→a (2), s→b (3), b→a (−2). Dijkstra settles a at distance 2 before it settles b. The true distance is 3 + (−2) = 1, found only after a has been finalised. With lazy re-insertion some implementations happen to recover in this case, but then the "settled once" guarantee and the complexity bound both disappear, and negative cycles make the answer undefined. Use Bellman–Ford (A7).

Stale entries. Java's `PriorityQueue` has no decrease-key. Push a new `(dist, v)` entry on every improvement and, when popping, skip any entry whose distance is larger than the current `dist[v]`. The heap holds at most E + 1 entries.

```
Generalised condition: path value f(P · e) is computed from f(P) and
w(e) by an operation ⊕ that is
  (a) monotone:            a ≤ b ⇒ a ⊕ w ≤ b ⊕ w
  (b) non-improving:       a ⊕ w ≥ a          (for minimisation)
Sum with w ≥ 0           : a ⊕ w = a + w              ✓
Bottleneck (LC 1631)     : a ⊕ w = max(a, w)          ✓
Reliability (LC 1514)    : a ⊕ p = a · p, p ∈ [0, 1]  ✓ (maximise;
value never increases)
Sum with w < 0           : violates (b)               ✗  → Bellman–
Ford

Counting shortest paths (weights > 0):
  ways[s] = 1
  ways[v] = Σ ways[u]  over edges u → v with dist[u] + w(u, v) =
dist[v]
```

The counting recurrence is evaluated correctly in pop order because every predecessor u on a shortest path to v has `dist[u] < dist[v]` when weights are strictly positive. So all of v's contributors are settled, and their counts are final, before v itself is popped. With zero-weight edges this ordering argument fails and the counting needs extra care: zero-weight ties may be popped in either order.

Template

Template A — lazy Dijkstra with path reconstruction. Directed edges `{u, v, w}` with w ≥ 0; add the reverse edge for undirected graphs. Distances are `long` because sums of int weights overflow.

JAVA TEMPLATE Dijkstra.java

```java
import java.util.*;

class Dijkstra {
    // headroom: INF + w never overflows
    static final long INF = Long.MAX_VALUE / 4;

    long[] dist;
    int[] parent;

    /** n vertices 0..n-1, directed edges {u, v, w}, w >= 0. */
    Dijkstra(int n, int[][] edges, int src) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) adj.get(e[0]).add(new int[]{e[1],
          e[2]});

        dist = new long[n];
        parent = new int[n];
        Arrays.fill(dist, INF);
        Arrays.fill(parent, -1);
        dist[src] = 0;
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
          Long.compare(a[0], b[0]));
        pq.add(new long[]{0, src});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
            long d = top[0];
            int u = (int) top[1];
            // stale entry: u already settled cheaper
            if (d > dist[u]) continue;
            for (int[] e : adj.get(u)) {
                int v = e[0];
                long nd = d + e[1];
                // relaxation
                if (nd < dist[v]) {
                    dist[v] = nd;
                    parent[v] = u;
                    pq.add(new long[]{nd, v});
                }
            }
        }
    }

    /** Vertices on a shortest src -> t path, or an empty list if t
      is unreachable. */
    List<Integer> path(int t) {
        LinkedList<Integer> p = new LinkedList<>();
        if (dist[t] >= INF) return p;
        for (int v = t; v != -1; v = parent[v]) p.addFirst(v);
        return p;
    }
}
```

Template B — dense-graph Dijkstra, O(V²), no heap. When E ≈ V² (a complete graph or adjacency matrix), a linear scan for the minimum beats a heap's O(E log V).

JAVA TEMPLATE DenseDijkstra.java

```java
import java.util.*;

class DenseDijkstra {
    /** w[u][v] >= 0 is the edge weight, or -1 if there is no edge
      (directed). */
    static long[] run(int[][] w, int src) {
        int n = w.length;
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        boolean[] done = new boolean[n];
        dist[src] = 0;
        for (int iter = 0; iter < n; iter++) {
            int u = -1;
            for (int v = 0; v < n; v++) {
                if (done[v] || dist[v] == Long.MAX_VALUE) continue;
                if (u == -1 || dist[v] < dist[u]) u = v;
            }
            // remaining vertices unreachable
            if (u == -1) break;
            done[u] = true;
            for (int v = 0; v < n; v++) {
                if (w[u][v] >= 0 && !done[v] && dist[u] + w[u][v] <
                  dist[v]) {
                    dist[v] = dist[u] + w[u][v];
                }
            }
        }
        // Long.MAX_VALUE = unreachable
        return dist;
    }
}
```

Problems

LC 1514 Path with Maximum Probability — MEDIUM

Task. In an undirected graph whose edges carry success probabilities in [0, 1], return the maximum probability of a path from `start_node` to `end_node`, or 0 if no path exists.

KEY INSIGHT

Path value is a product of factors in [0, 1], so extending a path never increases its value. That is the mirror image of "never decreases the cost", which makes greedy settle-the-most-probable-first valid with a *max*-heap.

JAVA SOLUTION LeetCode 1514 · Solution.java

```java
import java.util.*;

class Solution {
    public double maxProbability(int n, int[][] edges, double[]
      succProb,
                                 int start_node, int end_node) {
        // {neighbour, probability}
        List<List<double[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        // undirected
        for (int i = 0; i < edges.length; i++) {
            adj.get(edges[i][0]).add(new double[]{edges[i][1],
              succProb[i]});
            adj.get(edges[i][1]).add(new double[]{edges[i][0],
              succProb[i]});
        }
        // 0 = unreachable so far
        double[] best = new double[n];
        best[start_node] = 1.0;
        // max-heap on probability
        PriorityQueue<double[]> pq =
            new PriorityQueue<>((a, b) -> Double.compare(b[0],
              a[0]));
        pq.add(new double[]{1.0, start_node});
        while (!pq.isEmpty()) {
            double[] top = pq.poll();
            double p = top[0];
            int u = (int) top[1];
            // stale
            if (p < best[u]) continue;
            // popped = final
            if (u == end_node) return p;
            for (double[] e : adj.get(u)) {
                int v = (int) e[0];
                double np = p * e[1];
                if (np > best[v]) {
                    best[v] = np;
                    pq.add(new double[]{np, v});
                }
            }
        }
        return 0.0;
    }
}
```

Time O((V + E) log E) **Space** O(V + E)

LC 1976 Number of Ways to Arrive at Destination — MEDIUM

Task. In an undirected graph with positive travel times, count the shortest-time paths from intersection 0 to n − 1, modulo 10^9+7.

KEY INSIGHT

Shortest-path counting is DP over the shortest-path DAG, and Dijkstra's pop order is a topological order of that DAG when weights are strictly positive. Count while relaxing: a strict improvement *resets* the count, and an equal distance *adds* to it.

JAVA SOLUTION LeetCode 1976 · Solution.java

```java
import java.util.*;

class Solution {
    private static final int MOD = 1_000_000_007;

    public int countPaths(int n, int[][] roads) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        // undirected, time >= 1
        for (int[] r : roads) {
            adj.get(r[0]).add(new int[]{r[1], r[2]});
            adj.get(r[1]).add(new int[]{r[0], r[2]});
        }
        long[] dist = new long[n];
        long[] ways = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        dist[0] = 0;
        ways[0] = 1;
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
          Long.compare(a[0], b[0]));
        pq.add(new long[]{0, 0});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
```

```
            long d = top[0];
            int u = (int) top[1];
            // stale: would double-count ways
            if (d > dist[u]) continue;
            for (int[] e : adj.get(u)) {
                int v = e[0];
                long nd = d + e[1];
                // strictly shorter: old ways are obsolete
                if (nd < dist[v]) {
                    dist[v] = nd;
                    ways[v] = ways[u];
                    pq.add(new long[]{nd, v});
                // another shortest route into v
                } else if (nd == dist[v]) {
                    ways[v] = (ways[v] + ways[u]) % MOD;
                }
            }
        }
        return (int) ways[n - 1];
    }
}
```

Time O((V + E) log E) **Space** O(V + E)

ALGORITHM A3

Union-Find / Disjoint Set Union

A forest of parent pointers that answers "are x and y in the same group?" while groups only ever merge. Chapter 17 applies it to connectivity problems. This chapter covers why the two optimisations matter, what the amortised bound really means, a weighted extension, and the counting arguments that recur in DSU problems.

Concepts

Representation. `parent[x]` points toward the root of x's tree, and a root has `parent[r] = r`. Two elements are in the same set exactly when their roots coincide. The invariant to preserve: *every element's parent chain ends at its own set's root*. A union only re-points one root, so it never breaks the chains of other elements.

- **Path compression**: during `find`, point visited nodes directly (or every other node, for path halving) at the root. Membership is unchanged; only the shape changes.
- **Union by size / rank**: attach the root of the smaller tree under the root of the larger. A node's depth grows only when its tree at least doubles in size, so depth ≤ $\log_2 n$ even without compression.

```
Union by size alone        : height ≤ ⌊log₂ n⌋            ⇒ find is
O(log n) worst case
Path compression alone     : O(log n) amortised per operation (m ≥ n
operations)
Both together (Tarjan)     : O(m · α(n)) for m operations; α(n) ≤ 4
for any n < 2^65536

Components after processing edges: components = n − (number of
successful unions)
Redundant edges              : redundant = E − (number of successful
unions)
Connecting k components needs exactly k − 1 additional edges.
```

Why size beats rank in interviews. Both give the same bound. Size has a direct meaning: it answers "how large is x's group?" with `size[find(x)]` at no extra cost.

Template

Template A — DSU with path halving and union by size. The iterative find avoids recursion depth issues for n up to 10^6.

JAVA TEMPLATE DSU.java

```
class DSU {
    private final int[] parent, size;
    private int components;

    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int x) {
        while (parent[x] != x) {
            // path halving: shortcut every other node
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    /** Returns false if x and y were already connected (edge (x, y)
      closes a cycle). */
    boolean union(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        // rx is the larger root
        if (size[rx] < size[ry]) { int t = rx; rx = ry; ry = t; }
        parent[ry] = rx;
        size[rx] += size[ry];
        components--;
        return true;
    }

    boolean connected(int x, int y) { return find(x) == find(y); }
    int componentSize(int x) { return size[find(x)]; }
    int components() { return components; }
}
```

Template B — weighted DSU (potential to the root). This stores `w[x] = value(x) − value(parent[x])`, so constraints such as `value(b) − value(a) = d` can be merged and checked. It is the basis of LC 399 (Evaluate Division, in multiplicative form) and of parity problems.

JAVA TEMPLATE WeightedDSU.java

```
class WeightedDSU {
    private final int[] parent;
    // diff[x] = value(x) − value(parent[x])
    private final long[] diff;

    WeightedDSU(int n) {
        parent = new int[n];
        diff = new long[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    /** Returns the root; afterwards diff[x] = value(x) −
      value(root). */
    int find(int x) {
        if (parent[x] == x) return x;
        int root = find(parent[x]);
        // parent[x] is now a root child: full offset
        diff[x] += diff[parent[x]];
        parent[x] = root;
        return root;
    }

    /** Adds value(b) − value(a) = d. Returns false if it
      contradicts earlier facts. */
    boolean relate(int a, int b, long d) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return diff[b] − diff[a] == d;
        // value(rb) − value(ra) = d + diff[a] − diff[b]
        parent[rb] = ra;
        diff[rb] = d + diff[a] − diff[b];
        return true;
    }

    /** value(b) − value(a) if known, else null. */
    Long difference(int a, int b) {
        if (find(a) != find(b)) return null;
        return diff[b] − diff[a];
    }
}
```

Template B uses recursion and no union by size, to keep the offset algebra readable. Its depth is O(log n) amortised but can be O(n) on a single first call, so add union by size (the offset formula is symmetric) when n is large.

Problems

LC 1319 Number of Operations to Make Network Connected

MEDIUM

Task. n computers, undirected cables `connections`. You may move any cable to any other pair of computers. Return the minimum number of moves to connect every computer, or −1 if impossible.

KEY INSIGHT

A connected graph on n vertices needs at least n − 1 edges. If there are at least n − 1 cables, the redundant cables (those whose endpoints were already connected) are always enough to link the k components with k − 1 moves.

JAVA SOLUTION LeetCode 1319 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int makeConnected(int n, int[][] connections) {
        // not enough cables in total
        if (connections.length < n - 1) return -1;
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
        int components = n;
        for (int[] c : connections) {
            // undirected edge
            if (union(c[0], c[1])) components--;
        }
        // E >= n-1 guarantees >= components-1 redundant cables
        return components - 1;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(n + E · α(n)) **Space** O(n)

LC 990 Satisfiability of Equality Equations — MEDIUM

Task. Given equations like `"a==b"` and `"b!=c"` over single lowercase-letter variables, decide whether integers can be assigned so that all hold.

KEY INSIGHT

Equality is an equivalence relation. Its classes are exactly the DSU components after merging every `==`. A `!=` is violated exactly when both variables fall in the same class. Any assignment of distinct values per class satisfies everything else.

JAVA SOLUTION LeetCode 990 · Solution.java

```java
class Solution {
    private final int[] parent = new int[26];

    public boolean equationsPossible(String[] equations) {
        for (int i = 0; i < 26; i++) parent[i] = i;
        // phase 1: all equalities
        for (String e : equations) {
            if (e.charAt(1) == '=') union(e.charAt(0) - 'a',
              e.charAt(3) - 'a');
        }
        // phase 2: check inequalities
        for (String e : equations) {
            if (e.charAt(1) == '!' && find(e.charAt(0) - 'a') ==
              find(e.charAt(3) - 'a')) {
                // includes "a!=a"
                return false;
            }
        }
        return true;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        // 26 elements: union by size is unnecessary
        parent[find(a)] = find(b);
    }
}
```

Time O(m · α(26)) ≈ O(m) for m equations **Space** O(1) (26-entry array)

ALGORITHM A4

Minimum Spanning Tree: Kruskal and Prim

> Connect every vertex of an undirected, weighted graph with the minimum total edge weight. Both classic algorithms are greedy, and both are justified by the same *cut property*.

Concepts

Invariant (both algorithms). The chosen edge set F is always a subset of *some* minimum spanning tree.

Cycle property (the dual). The heaviest edge on any cycle (if strictly heaviest) is in no MST. That is why Kruskal may safely reject an edge whose endpoints are already connected.

```
A spanning tree of V vertices has exactly V − 1 edges.
Connected ⇔ Kruskal accepts exactly V − 1 edges.
Spanning forest of a graph with c components has V − c edges.

Kruskal : O(E log E) sort + O(E · α(V)) DSU            = O(E log V)
Prim    : binary heap, lazy   O(E log E)                 = O(E log V)
          adjacency matrix / complete graph, array scan   O(V²)
Dense complete graph (E = V(V−1)/2): Prim O(V²) beats Kruskal O(V²
log V)
```

If all edge weights are distinct the MST is unique. With ties there may be several MSTs, all with the same total weight.

Template

Template A — Kruskal with DSU. Undirected edges `{u, v, w}`, vertices 0..n−1. Returns the forest weight and reports connectivity.

JAVA TEMPLATE Kruskal.java

```java
import java.util.*;

class Kruskal {
    /** Returns MST weight, or -1 if the graph is disconnected.
      Edges are undirected. */
    static long mstWeight(int n, int[][] edges) {
        // do not reorder the caller's array
        int[][] es = edges.clone();
        Arrays.sort(es, (a, b) -> Integer.compare(a[2], b[2]));
        int[] parent = new int[n], size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        long total = 0;
        int taken = 0;
        for (int[] e : es) {
            int ru = find(parent, e[0]), rv = find(parent, e[1]);
            // would close a cycle: cycle property
            if (ru == rv) continue;
            if (size[ru] < size[rv]) { int t = ru; ru = rv; rv = t;
              }
            parent[rv] = ru;
            size[ru] += size[rv];
            total += e[2];
            // tree complete
            if (++taken == n - 1) break;
        }
        // n - taken = number of components
        return taken == n - 1 ? total : -1;
    }

    private static int find(int[] p, int x) {
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    }
}
```

Template B — Prim with a lazy binary heap. Adjacency list; a vertex is added to the tree when it is first popped.

JAVA TEMPLATE PrimHeap.java

```java
import java.util.*;

class PrimHeap {
    /** Undirected edges {u, v, w}. Returns MST weight, or -1 if
      disconnected. */
    static long mstWeight(int n, int[][] edges) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) {
            adj.get(e[0]).add(new int[]{e[1], e[2]});
            adj.get(e[1]).add(new int[]{e[0], e[2]});
        }
        boolean[] inTree = new boolean[n];
```

```
        PriorityQueue<int[]> pq =
            new PriorityQueue<>((a, b) -> Integer.compare(a[1],
              b[1]));
        // {vertex, weight of edge into tree}
        pq.add(new int[]{0, 0});
        long total = 0;
        int count = 0;
        while (!pq.isEmpty() && count < n) {
            int[] top = pq.poll();
            int u = top[0];
            // stale: u joined via a cheaper edge
            if (inTree[u]) continue;
            inTree[u] = true;
            total += top[1];
            count++;
            for (int[] e : adj.get(u)) {
                // crossing edges
                if (!inTree[e[0]]) pq.add(new int[]{e[0], e[1]});
            }
        }
        return count == n ? total : -1;
    }
}
```

Prim's heap holds *edge weights* (the cost to attach a vertex), not path distances. Writing `dist[u] + w`, the Dijkstra habit, computes a shortest-path tree instead.

Problems

LC 1584 Min Cost to Connect All Points — MEDIUM

Task. Given up to 1000 points, connect them all with minimum total Manhattan distance `|xi − xj| + |yi − yj|`.

KEY INSIGHT

The graph is complete and implicit: every pair is an edge. With V = 1000, E ≈ $5 \cdot 10^5$. Array-based Prim runs in $O(V^2) = 10^6$ steps and never builds an edge list.

JAVA SOLUTION LeetCode 1584 · Solution.java

```
import java.util.*;

class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        // cheapest edge from the tree to each vertex
        int[] minDist = new int[n];
        boolean[] inTree = new boolean[n];
        Arrays.fill(minDist, Integer.MAX_VALUE);
        minDist[0] = 0;
        long total = 0;
        for (int iter = 0; iter < n; iter++) {
            int u = -1;
            // O(V) scan replaces the heap
            for (int v = 0; v < n; v++) {
                if (!inTree[v] && (u == -1 || minDist[v] <
                  minDist[u])) u = v;
            }
            inTree[u] = true;
            total += minDist[u];
            for (int v = 0; v < n; v++) {
                if (!inTree[v]) {
                    int d = Math.abs(points[u][0] - points[v][0])
                          // <= 4e6, fits in int
                          + Math.abs(points[u][1] - points[v][1]);
                    if (d < minDist[v]) minDist[v] = d;
                }
            }
        }
        return (int) total;
    }
}
```

Time $O(V^2)$ **Space** $O(V)$

LC 1135 Connecting Cities With Minimum Cost — MEDIUM

Premium

Task. Cities are labelled 1..n; `connections[i] = [x, y, cost]` is an undirected candidate road. Return the minimum cost to connect all cities, or −1 if impossible.

KEY INSIGHT

The input is a sparse edge list with a possible disconnected answer, which is exactly Kruskal's natural input. Connectivity is checked for free by counting accepted edges.

JAVA SOLUTION LeetCode 1135 · Solution.java

```
import java.util.*;

class Solution {
    private int[] parent, size;

    public int minimumCost(int n, int[][] connections) {
        // cities are 1-indexed
        parent = new int[n + 1];
        size = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
        Arrays.sort(connections, (a, b) -> Integer.compare(a[2],
          b[2]));
        int total = 0, used = 0;
        for (int[] c : connections) {
            // undirected road
            if (union(c[0], c[1])) {
                total += c[2];
                // spanning tree complete
                if (++used == n - 1) return total;
            }
        }
        // fewer than n-1 edges: disconnected
        return n == 1 ? 0 : -1;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(E log E) **Space** O(n) DSU + O(log E) sort stack

ALGORITHM A5

Kadane's Algorithm

> A one-pass dynamic programme for the best contiguous subarray. The best subarray ending at index i either extends the best one ending at i − 1 or starts fresh at i.

Concepts

Let `end(i)` be the maximum sum of a non-empty subarray that ends exactly at index i, and `best(i)` the maximum over all subarrays within a[0..i].

```
Invariant after processing index i:
  cur  = end(i)  = max sum of a[j..i] over 0 ≤ j ≤ i
  best = max(end(0), …, end(i))
```

Every subarray ends somewhere, so the answer is `max over i of end(i)`. That is the whole correctness argument: local best (ending here) feeds the global best (anywhere so far).

```
end(0) = a[0]
end(i) = a[i] + max(0, end(i-1))  =  max(a[i], end(i-1) + a[i])
answer = max over i of end(i)
```

Derivation: a subarray ending at i is either [i] alone, or a[j..i−1] followed by a[i]. The best of the second kind is end(i−1) + a[i]. So keep the previous run only if it is positive.

Prefix-sum view. sum(a[j..i]) = pre[i+1] − pre[j], so end(i) = pre[i+1] − min(pre[0..i]). Kadane is the same as tracking the running minimum prefix. This view generalises to length-constrained versions: use a minimum over a sliding range of prefixes (monotonic deque).

Circular case. A wrapping subarray takes a suffix and a prefix, which is the complement of a middle block a[j..i]. Its sum is `total − sum(a[j..i])`, maximised when the middle block is the minimum subarray:

```
circular = max( maxSubarray(a),  total − minSubarray(a) )
```
Exception: if every element is negative, minSubarray = total, so total − min = 0 corresponds to the empty subarray, which is not allowed. Return maxSubarray(a) in that case (detect it with maxSubarray < 0).

Empty-subarray conventions. If the problem allows an empty subarray (answer ≥ 0), start with `cur = 0, best = 0` and clamp `cur = max(0, cur + a[i])`. If subarrays must be non-empty (LeetCode 53, 918), initialise both with `a[0]`. Initialising `best = 0` returns 0 instead of −1 on `[-3,-1,-2]`.

Template

JAVA TEMPLATE Kadane.java

```java
/** Kadane variants. Sums are accumulated in long so n * |a[i]|
  cannot overflow. */
class Kadane {
    /** Maximum sum of a NON-EMPTY contiguous subarray. Requires
      a.length >= 1. */
    static long maxSubarray(int[] a) {
        // not 0: all-negative input must return max
        long cur = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) {
            // extend or restart
            cur = Math.max(a[i], cur + a[i]);
            best = Math.max(best, cur);
        }
        return best;
    }

    /** Maximum sum when the EMPTY subarray (sum 0) is allowed. */
    static long maxSubarrayAllowEmpty(int[] a) {
        long cur = 0, best = 0;
        for (int x : a) {
            cur = Math.max(0, cur + x);
            best = Math.max(best, cur);
        }
        return best;
    }

    /** Returns {bestSum, start, end} (inclusive) of a non-empty
      maximum subarray. */
    static long[] maxSubarrayWithIndices(int[] a) {
        long cur = a[0], best = a[0];
        int start = 0, bestL = 0, bestR = 0;
        for (int i = 1; i < a.length; i++) {
            // previous run can only hurt: restart at i
            if (cur < 0) {
                cur = a[i];
                start = i;
            } else {
                cur += a[i];
            }
            if (cur > best) {
                best = cur;
                bestL = start;
                bestR = i;
            }
        }
        return new long[]{best, bestL, bestR};
    }

    /** Minimum sum of a non-empty subarray (mirror image). */
    static long minSubarray(int[] a) {
        long cur = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) {
            cur = Math.min(a[i], cur + a[i]);
            best = Math.min(best, cur);
        }
        return best;
    }
}
```

Problems

LC 53 **Maximum Subarray** — MEDIUM

Task. Return the largest sum of a non-empty contiguous subarray.

KEY INSIGHT

A negative running sum is a liability to every later subarray, so restart whenever extending is worse than starting at the current element.

JAVA SOLUTION LeetCode 53 · Solution.java

```java
class Solution {
    public int maxSubArray(int[] nums) {
        // non-empty: start from the first element
        int cur = nums[0], best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            // |sum| <= 1e9: fits in int
            cur = Math.max(nums[i], cur + nums[i]);
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

LC 918 **Maximum Sum Circular Subarray** — MEDIUM

Task. Return the maximum sum of a non-empty subarray of a circular array, where each element may be used at most once.

KEY INSIGHT

A wrapping subarray is the complement of a non-wrapping middle block, so its best sum is `total − minSubarray`. The answer is the larger of the ordinary Kadane maximum and that complement, except when everything is negative.

JAVA SOLUTION LeetCode 918 · Solution.java

```java
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0;
        int curMax = 0, maxSum = nums[0];
        int curMin = 0, minSum = nums[0];
        for (int x : nums) {
            // best non-wrapping subarray ending here
            curMax = Math.max(x, curMax + x);
            maxSum = Math.max(maxSum, curMax);
            // worst subarray ending here
            curMin = Math.min(x, curMin + x);
            minSum = Math.min(minSum, curMin);
            // |total| <= 3e4 * 3e4 = 9e8: fits in int
            total += x;
        }
        // All negative: total − minSum == 0 would be the
          (forbidden) empty subarray.
        if (maxSum < 0) return maxSum;
        return Math.max(maxSum, total − minSum);
    }
}
```

Time O(n) **Space** O(1)

ALGORITHM A6

KMP String Matching

Knuth–Morris–Pratt finds every occurrence of a pattern in linear time by precomputing, for each prefix of the pattern, its longest proper border. A mismatch then jumps straight to the next viable alignment.

Concepts

Prefix function. For a string P, `pi[i]` = the length of the longest *proper* prefix of P[0..i] that is also a suffix of P[0..i]. Proper means shorter than i + 1.

```
P   = a b a b c a b a b a
pi  = 0 0 1 2 0 1 2 3 4 3
```
pi[8] = 4: "abab" is both a prefix and a suffix of "ababcabab". pi[9] = 3: "aba".

Search invariant. After reading T[i], `j` = the length of the longest prefix of P that is a suffix of T[0..i]. A match is reported when j = m, after which j becomes pi[m − 1] so overlapping matches are found.

Why matched characters need not be rechecked. Suppose P[0..j−1] equals T[i−j..i−1] and P[j] ≠ T[i]. Any shift that keeps fewer characters aligned must line a prefix of P up against a suffix of T[i−j..i−1] = P[0..j−1], which means a border of P[0..j−1]. Shifts that do not correspond to a border are mismatched on characters already known, so they are impossible. Trying borders in decreasing length (pi[j−1], pi[pi[j−1]−1], …) visits every viable alignment and skips the impossible ones without reading the text again.

Construction phase (pattern against itself). Borders of P[0..i] are obtained by extending borders of P[0..i−1]:

```
pi[0] = 0
for i = 1..m-1:
    k = pi[i-1]                          longest border of P[0..i−1]
    while k > 0 and P[i] ≠ P[k]:  k = pi[k-1]    next shorter border
    if P[i] == P[k]:  k++
    pi[i] = k
```

This is correct because the borders of a string are exactly pi[i−1], pi[pi[i−1]−1], …, so the loop tries every candidate border in decreasing order and takes the first that extends by P[i].

Search phase. This is the same loop with T[i] in place of P[i], continuing to compare against P[j].

Linear-time (amortised) proof. In each phase, k (or j) increases by at most 1 per character, so by at most n in total. Each iteration of the inner `while` loop decreases it by at least 1, and it never goes below 0. Therefore the total number of fallbacks is at most the total number of increases, and the work is O(m) for construction plus O(n) for search.

Useful identities.

```
Longest border of S                = pi[|S| − 1]
Smallest period of S               = p = |S| − pi[|S| − 1]
S is a repetition of a block       ⇔  pi[|S|−1] > 0  and  |S| mod p
== 0
Longest palindromic prefix of S    = pi[last] of  S + '#' +
reverse(S)
```

Template

JAVA TEMPLATE KMP.java

```java
import java.util.*;

/** Knuth–Morris–Pratt. pi[i] = longest proper border of p[0..i]. */
class KMP {
    static int[] prefixFunction(String p) {
        int m = p.length();
        int[] pi = new int[m];
        for (int i = 1; i < m; i++) {
            int k = pi[i − 1];
            // shorter border
            while (k > 0 && p.charAt(i) != p.charAt(k)) k = pi[k −
              1];
            if (p.charAt(i) == p.charAt(k)) k++;
            pi[i] = k;
        }
        return pi;
    }

    /** All start indices of pattern in text (overlapping matches
      included). */
    static List<Integer> findAll(String text, String pattern) {
        List<Integer> res = new ArrayList<>();
        int m = pattern.length();
        // convention: caller decides empty-pattern rule
        if (m == 0) return res;
        int[] pi = prefixFunction(pattern);
        // length of matched prefix of pattern
        int j = 0;
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            // never move i back
            while (j > 0 && c != pattern.charAt(j)) j = pi[j − 1];
            if (c == pattern.charAt(j)) j++;
            if (j == m) {
                res.add(i − m + 1);
                // continue for overlapping matches
                j = pi[m − 1];
            }
        }
        return res;
    }

    /** Smallest period of s (s.length() if s is not a repetition of
      a shorter block). */
    static int smallestPeriod(String s) {
        int n = s.length();
        if (n == 0) return 0;
        int p = n − prefixFunction(s)[n − 1];
        return n % p == 0 ? p : n;
    }
}
```

Problems

LC 28 Find the Index of the First Occurrence in a String — EASY

Task. Return the index of the first occurrence of `needle` in `haystack`, or −1. Both strings have length ≥ 1.

KEY INSIGHT

After a partial match, the prefix function tells you how much of the match survives a shift, so the text pointer never moves backward.

JAVA SOLUTION LeetCode 28 · Solution.java

```java
class Solution {
    public int strStr(String haystack, String needle) {
        int m = needle.length();
        // defensive: constraints say m >= 1
        if (m == 0) return 0;
        int[] pi = new int[m];
        // construction phase
        for (int i = 1; i < m; i++) {
            int k = pi[i − 1];
            while (k > 0 && needle.charAt(i) != needle.charAt(k)) k
              = pi[k − 1];
            if (needle.charAt(i) == needle.charAt(k)) k++;
            pi[i] = k;
        }
        int j = 0;
        // search phase
        for (int i = 0; i < haystack.length(); i++) {
            char c = haystack.charAt(i);
            while (j > 0 && c != needle.charAt(j)) j = pi[j − 1];
            if (c == needle.charAt(j)) j++;
            if (j == m) return i − m + 1;
        }
        return −1;
    }
}
```

Time O(n + m) **Space** O(m)

LC 214 Shortest Palindrome — HARD

Task. Add the fewest characters to the *front* of `s` to make it a palindrome; return the result. ($0 \le |s| \le 5 \cdot 10^4$, lowercase letters.)

KEY INSIGHT

Keep the longest palindromic prefix of s and prepend the reverse of the remaining suffix. A prefix of s is a palindrome exactly when it equals a suffix of reverse(s), so the longest one is the final prefix-function value of `s + '#' + reverse(s)`.

JAVA SOLUTION LeetCode 214 · Solution.java

```java
class Solution {
    public String shortestPalindrome(String s) {
        int n = s.length();
        String rev = new StringBuilder(s).reverse().toString();
        // '#' never occurs in s (lowercase only)
        String t = s + "#" + rev;
        int[] pi = new int[t.length()];
        for (int i = 1; i < t.length(); i++) {
            int k = pi[i − 1];
            while (k > 0 && t.charAt(i) != t.charAt(k)) k = pi[k −
              1];
            if (t.charAt(i) == t.charAt(k)) k++;
            pi[i] = k;
        }
        // longest palindromic prefix of s
        int pal = pi[t.length() − 1];
        // prepend reverse of the remaining suffix
        return rev.substring(0, n − pal) + s;
    }
}
```

Time O(n) **Space** O(n)

ALGORITHM A7

Bellman–Ford Algorithm

Relax every edge, repeatedly. After round i, every shortest path that uses at most i edges has been found. This one fact gives negative-weight support, negative-cycle detection and edge-budgeted shortest paths.

Concepts

Relaxation. `if dist[u] + w(u, v) < dist[v] then dist[v] = dist[u] + w(u, v)`. A relaxation never sets dist[v] below the weight of some real path to v, so distances are always upper bounds (or ∞).

```
Invariant after round i (snapshot version):
  dist_i[v] = min weight over walks src ⇝ v using at most i edges

dist_0[src] = 0, dist_0[v] = ∞ otherwise
dist_i[v]   = min( dist_{i-1}[v],  min over edges (u,v,w) of
dist_{i-1}[u] + w )
```

This is a DP over "number of edges used". With in-place updates (the classical algorithm), dist after round i is *at most* dist_i: it may already include longer paths. That does not affect correctness for unconstrained shortest paths, but it

breaks any edge budget.

Why V − 1 rounds suffice. If no negative cycle is reachable from the source, every shortest walk can have its cycles removed (each has weight ≥ 0) without increasing its cost. The result is a simple path with at most V − 1 edges. By the invariant, round V − 1 has considered every path with ≤ V − 1 edges, so the distances are final.

Negative-cycle detection. If after V − 1 rounds some edge (u, v) can still be relaxed, with dist[u] finite, then a shortest path would need ≥ V edges, so it repeats a vertex. That repeated section must be a negative cycle, and it is reachable from src (dist[u] is finite). Conversely, a reachable negative cycle always leaves at least one relaxable edge: summing `dist[v] ≤ dist[u] + w` around the cycle would give 0 ≤ (cycle weight) < 0.

```
Detection:  ∃ (u,v,w): dist[u] ≠ ∞ and dist[u] + w < dist[v]  after
V−1 rounds
         ⇔  a negative cycle is reachable from src

Vertices whose distance is −∞ : every vertex reachable FROM a vertex
on (or fed by) that cycle.
Propagate −∞ with V more rounds, or BFS from every vertex that still relaxes.
```

Early termination. If a full round changes nothing, no later round can change anything, because each round depends only on the previous one. Stop early. When distances stabilise, no reachable negative cycle can exist.

Template

Template A — classical Bellman–Ford with early exit, detection and −∞ propagation. Directed edges `{u, v, w}`; weights may be negative.

JAVA TEMPLATE BellmanFord.java

```java
import java.util.*;

/** Single-source shortest paths, weights may be negative. Directed
  edges {u, v, w}. */
class BellmanFord {
    // sentinel; INF + w cannot overflow
    static final long INF = Long.MAX_VALUE / 4;
    // marks "arbitrarily small" distances
    static final long NEG_INF = Long.MIN_VALUE;

    /**
     * Returns dist[] where dist[v] == INF if v is unreachable and
       dist[v] == NEG_INF if a
     * negative cycle reachable from src can reach v (no finite
       shortest path exists).
     */
    static long[] shortestPaths(int n, int[][] edges, int src) {
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 1; round <= n - 1; round++) {
            boolean changed = false;
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                // skip unreachable u
                if (dist[u] != INF && dist[u] + e[2] < dist[v]) {
                    dist[v] = dist[u] + e[2];
                    changed = true;
                }
            }
            // stable: no reachable negative cycle
            if (!changed) return dist;
        }
        // Any edge still relaxable lies on or after a reachable
          negative cycle.
        // n more rounds propagate NEG_INF along every path leaving
          such edges.
        for (int round = 0; round < n; round++) {
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                if (dist[u] == INF) continue;
                if (dist[u] == NEG_INF || dist[u] + e[2] < dist[v])
                  dist[v] = NEG_INF;
            }
        }
        return dist;
    }

    /** True iff some negative cycle is reachable from src. */
    static boolean hasReachableNegativeCycle(int n, int[][] edges,
      int src) {
        for (long d : shortestPaths(n, edges, src)) if (d ==
          NEG_INF) return true;
        return false;
    }
}
```

Template B — shortest path with at most K edges (snapshot per round).

JAVA TEMPLATE BoundedEdgesBellmanFord.java

```java
import java.util.*;

/** min-cost walk src -> every v using AT MOST k edges. Directed
  edges {u, v, w}. */
class BoundedEdgesBellmanFord {
    static final long INF = Long.MAX_VALUE / 4;

    static long[] atMostKEdges(int n, int[][] edges, int src, int k)
      {
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 1; round <= k; round++) {
            // reads use round-(i-1) values only
            long[] next = dist.clone();
            boolean changed = false;
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                if (dist[u] != INF && dist[u] + e[2] < next[v]) {
                    next[v] = dist[u] + e[2];
                    changed = true;
                }
            }
            dist = next;
            // further rounds cannot help
            if (!changed) break;
        }
        return dist;
    }
}
```

Why the snapshot matters. With in-place updates, one round can relax a → b and then b → c using the *new* dist[b], so it uses two edges in a single round. If the edge list happens to be in path order, one round can traverse a whole chain, and the "≤ i edges after round i" guarantee disappears.

Problems

LC 787 Cheapest Flights Within K Stops — MEDIUM

Task. Directed flights `[from, to, price]` with price ≥ 1. Return the cheapest price from `src` to `dst` using at most K intermediate stops (so at most K + 1 flights), or −1.

KEY INSIGHT

"At most K stops" is "at most K + 1 edges". After round i of snapshot Bellman–Ford, `dist[v]` is exactly the cheapest walk with ≤ i edges, so run K + 1 rounds and read `dist[dst]`.

JAVA SOLUTION LeetCode 787 · Solution.java

```java
import java.util.*;

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src,
      int dst, int k) {
        final int INF = Integer.MAX_VALUE;
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        // k stops = k + 1 edges = k + 1 rounds
        for (int round = 0; round <= k; round++) {
            // snapshot: one more edge per round
            int[] next = dist.clone();
            boolean changed = false;
            for (int[] f : flights) {
                int u = f[0], v = f[1], price = f[2];
                // Max cost <= 101 flights * 1e4 = ~1e6, so dist[u]
                  + price cannot overflow.
                if (dist[u] != INF && dist[u] + price < next[v]) {
                    next[v] = dist[u] + price;
                    changed = true;
                }
            }
            dist = next;
            if (!changed) break;
        }
        return dist[dst] == INF ? -1 : dist[dst];
    }
}
```

Time O((K + 1) · E) **Space** O(V)